None.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for Controls for View	Defines the data that is displayed by the control.

Text Value

Specify the script whose value is displayed by the control.

Remarks

See Also

ExpressionBinding Element for CustomItem for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Frame Element for CustomItem for Controls for View

Defines how the data is displayed, such as shifting the data to the left or right. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element

Syntax

XML

```
<Frame>
  <LeftIndent>NumberOfCharactersToShift</LeftIndent>
  <RightIndent>NumberOfCharactersToShift</RightIndent>
  <FirstLineHanging>NumberOfCharactersToShift</FirstLineHanging>
  <FirstLineIndent>NumberOfCharactersToShift</FirstLineIndent>
  <CustomItem>...</CustomItem>
</Frame>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `Frame` element.

Attributes

None.

Child Elements

Expand table

Element	Description
`CustomItem Element`	Required Element
FirstLineHanging Element of Frame of Controls of View	Optional element. Specifies how many characters the first line is shifted to the left.
FirstLineIndent Element of Frame of Controls of View	Optional element. Specifies how many characters the first line is shifted to the right.
LeftIndent Element of Frame of Controls of View	Optional element. Specifies how many characters the data is shifted away from the left margin.
RightIndent Element of Frame of Controls of View	Optional element. Specifies how many characters the data is shifted away from the right margin.

Parent Elements

Expand table

Element	Description
CustomItem Element for CustomEntry for Controls for View	Defines what data is displayed by the control and how it is displayed.

Remarks

You cannot specify the FirstLineHanging and the FirstLineIndent elements in the same `Frame` element.

See Also

FirstLineHanging Element of Frame of Controls of View

FirstLineIndent Element of Frame of Controls of View

LeftIndent Element of Frame of Controls of View

RightIndent Element of Frame of Controls of View

CustomItem Element for CustomEntry for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

FirstLineHanging Element for Frame for Controls for View

Specifies how many characters the first line of data is shifted to the left. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element
- FirstLineHanging Element

Syntax

XML

```
<FirstLineHanging>NumberOfCharactersToShift</FirstLineHanging>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `FirstLineHanging` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Frame Element for CustomItem for Controls for View	Defines how the data is displayed, such as shifting the data to the left or right.

Text Value

Specify the number of characters that you want to shift the first line of the data.

Remarks

If this element is specified, you cannot specify the FirstLineIndent element.

See Also

FirstLineIndent Element for Frame for Controls for View

Frame Element for CustomItem for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

FirstLineIndent Element for Frame for Controls for View

Specifies how many characters the first line of data is shifted to the right. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element
- FirstLineIndent Element

Syntax

XML

```
<FirstLineIndent>NumberOfCharactersToShift</FirstLineIndent>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `FirstLineIndent` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Frame Element for CustomItem for Controls for View	Defines how the data is displayed, such as shifting the data to the left or right.

Text Value

Specify the number of characters that you want to shift the first line of the data.

Remarks

If this element is specified, you cannot specify the FirstLineHanging element.

See Also

FirstLineHanging Element for Frame for Controls for View

Frame Element for CustomItem for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

LeftIndent Element for Frame for Controls for View

Specifies how many characters the data is shifted away from the left margin. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element
- LeftIndent Element

Syntax

XML

```
<LeftIndent>CharactersToShift</LeftIndent>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `LeftIndent` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Frame Element for CustomItem for Controls for View	Defines how the data is displayed, such as shifting the data to the left or right.

Text Value

Specify the number of characters that you want to shift the data to the left.

Remarks

See Also

Frame Element for CustomItem for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

RightIndent Element for Frame for Controls for View

Specifies how many characters the data is shifted away from the right margin. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element
- RightIndent Element

Syntax

XML

```
<RightIndent>CharactersToShift</RightIndent>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `RightIndent` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Frame Element for CustomItem for Controls for View	Defines how the data is displayed, such as shifting the data to the left or right.

Text Value

Specify the number of characters that you want to shift the data to the right.

Remarks

See Also

Frame Element for CustomItem for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

NewLine Element for CustomItem for Controls for View

Adds a blank line to the display of the control. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- NewLine Element

Syntax

XML

```
<NewLine/>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `NewLine` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
CustomItem Element for CustomEntry for Controls for View	Defines what data is displayed by the control and how it is displayed.

Remarks

See Also

CustomItem Element for CustomEntry for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Text Element for CustomItem for Controls for View

Specifies text that is added to the data that is displayed by the control, such as a label, brackets to enclose the data, and spaces to indent the data. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element

Syntax

XML

```
<Text>TextToDisplay</Text>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `Text` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
CustomItem Element for CustomEntry for Controls for View	Defines what data is displayed by the control and how it is displayed.

Text Value

Specify the text of a control for data that you want to display.

Remarks

See Also

CustomItem Element for CustomEntry for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

EntrySelectedBy Element for CustomEntry for Controls for View

Defines the .NET types that use this control definition or the condition that must exist for this definition to be used. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element

Syntax

XML

```xml
<EntrySelectedBy>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>NameofSelectionSet</SelectionSetName>
  <SelectionCondition>...</SelectionCondition>
</EntrySelectedBy>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `EntrySelectedBy` element. You must specify at least one type, selection set, or selection condition for a definition. There is no maximum limit to the number of child elements that you can use.

Attributes

None.

Child Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for Controls for View	Optional element. Defines the condition that must exist for this definition to be used.
SelectionSetName Element for EntrySelectedBy for Controls for View	Optional element. Specifies a set of .NET types that use this definition of the control.
TypeName Element for EntrySelectedBy for Controls for View	Optional element. Specifies a .NET type that uses this definition of the control.

Parent Elements

Expand table

Element	Description
CustomEntry Element for CustomEntries for Controls for View	Provides a definition of the control.

Remarks

Selection conditions are used to define a condition that must exist for the definition to be used, such as when an object has a specific property or when a specific property value or script evaluates to `true`. For more information about selection conditions, see Defining Conditions for when a View Entry or Item is Used.

See Also

CustomEntry Element for CustomEntries for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionCondition Element for EntrySelectedBy for Controls for View

Defines a condition that must exist for the control definition to be used. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionCondition Element

Syntax

XML

```xml
<SelectionCondition>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>NameofSelectionSet</SelectionSetName>
  <PropertyName>.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate</ScriptBlock>
</SelectionCondition>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionCondition` element.

Attributes

None.

Child Elements

Expand table

Element	Description
PropertyName Element for SelectionCondition for Controls for View	Optional element. Specifies a .NET property that triggers the condition.
ScriptBlock Element for SelectionCondition for Controls for View	Optional element. Specifies the script that triggers the condition.
SelectionSetName Element for SelectionCondition for Controls for View	Optional element. Specifies the set of .NET types that triggers the condition.
TypeName Element for SelectionCondition for Controls for View	Optional element. Specifies a .NET type that triggers the condition.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for CustomEntry for Controls for View	Defines the .NET types that use this control definition or the condition that must exist for this definition to be used.

Remarks

When you are defining a selection condition, the following requirements apply:

- The selection condition must specify a least one property name or a script block, but cannot specify both.
- The selection condition can specify any number of .NET types or selection sets, but cannot specify both.

For more information about how to use selection conditions, see Defining Conditions for when Data is Displayed.

See Also

PropertyName Element for SelectionCondition for Controls for View

ScriptBlock Element for SelectionCondition for Controls for View

SelectionSetName Element for SelectionCondition for Controls for View

TypeName Element for SelectionCondition for Controls for View

EntrySelectedBy Element for CustomEntry for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

PropertyName Element for SelectionCondition for CustomControl for View

Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the definition is used. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- EntrySelectedBy Element
- SelectionCondition Element
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for CustomControl for View	Defines a condition that must exist for the control definition to be used.

Text Value

Specify the .NET property name.

Remarks

The selection condition must specify a least one property name or a script, but cannot specify both. For more information about how selection conditions can be used, see Defining Conditions for Displaying Data.

See Also

SelectionCondition Element for EntrySelectedBy for CustomControl for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for SelectionCondition for CustomControl for View

Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the definition is used. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- SelectionCondition Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for CustomControl for View	Defines a condition that must exist for the control definition to be used.

Text Value

Specify the script that is evaluated.

Remarks

The selection condition must specify a least one script or property name to evaluate, but cannot specify both. For more information about how selection conditions can be used, see Defining Conditions for Displaying Data.

See Also

SelectionCondition Element for EntrySelectedBy for CustomControl for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSetName Element for SelectionCondition for Controls for View

Specifies the set of .NET types that trigger the condition. When any of the types in this set are present, the condition is met and the object is displayed using this control. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- SelectionSetName Element

Syntax

XML

```
<SelectionSetName>NameofSelectionSet</SelectionSetName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionSetName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for Controls for View	Defines a condition that must exist for the control definition to be used.

Text Value

Specify the name of the selection set.

Remarks

Selection sets are common groups of .NET objects that can be used by any view that the formatting file defines. For more information about creating and referencing selection sets, see Defining Selection Sets.

The selection condition can specify a selection set or .NET type, but cannot specify both. For more information about how to use selection conditions, see Defining Conditions for Displaying Data.

See Also

SelectionCondition Element for EntrySelectedBy for Controls for View

Defining Conditions for When Data Is Displayed

Defining Selection Sets

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for SelectionCondition for Controls for View

Specifies a .NET type that triggers the condition. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- TypeName Element

Syntax

XML

```
<TypeName>Nameof.NetType</TypeName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `TypeName` Element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for Controls for View	Defines a condition that must exist for the control definition to be used.

Text Value

Specify the fully qualified name of the .NET type, such as `System.IO.DirectoryInfo`.

Remarks

See Also

SelectionCondition Element for EntrySelectedBy for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSetName Element for EntrySelectedBy for Controls for View

Specifies a set of .NET types that use this definition of the control. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionSetName Element

Syntax

XML

```
<SelectionSetName>NameofSelectionSet</SelectionSetName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionSetName` element.

Attributes

None

Child Elements

None.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for CustomEntry for Controls for View	Defines the .NET types that use this control definition or the condition that must exist for this definition to be used.

Text Value

Specify the name of the selection set.

Remarks

Each control definition must have at least one type name, selection set, or selection condition defined.

Selection sets are typically used when you want to define a group of objects that are used in multiple views. For more information about defining selection sets, see Defining Selection Sets.

See Also

EntrySelectedBy Element for CustomEntry for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for EntrySelectedBy for Controls for View

Specifies a .NET type that uses this definition of the control. This element is used when defining controls that can be used by a view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- TypeName Element

Syntax

XML

```
<TypeName>Nameof.NetType</TypeName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `TypeName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for CustomEntry for Controls for View	Defines the .NET types that use this control definition or the condition that must exist for this definition to be used.

Text Value

Specify the fully qualified name of the .NET type, such as `System.IO.DirectoryInfo`.

Remarks

See Also

EntrySelectedBy Element for CustomEntry for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Name Element for Control for Controls for View

Specifies the name of the control.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Controls Element
- Control Element
- Name Element

Syntax

XML

```
<Name>ControlName</Name>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `Name` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Control Element for Controls for View	Defines a control that can be used by the view and the name that is used to reference the control.

Text Value

Specify the name that is used to reference the control.

Remarks

The name specified here can be used in the following elements to reference this control.

- When creating a table, list, wide or custom control view, the control can be specified by the following element: GroupBy Element for View
- When creating another control that can be used by a view, this control can be specified by the following element: ExpressionBinding Element for CustomItem for Controls for View

See Also

GroupBy Element for View

ExpressionBinding Element for CustomItem for Controls for View

Control Element for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomControl Element for View

Defines a custom control format for the view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element

Syntax

XML

```
<CustomControl>
  <CustomEntries>...</CustomEntries>
</CustomControl>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `CustomControl` element. You must specify one child element.

Attributes

None.

Child Elements

Expand table

Element	Description
CustomEntries Element for CustomControl for View	Required element. Provides the definitions of the custom control view.

Parent Elements

Expand table

Element	Description
View Element	Defines a view that is used to display one or more .NET objects.

Remarks

In most cases, only one definition is required for each control view, but it is possible to provide multiple definitions if you want to use the same view to display different .NET objects. In those cases, you can provide a separate definition for each object or set of objects.

See Also

CustomEntries Element for CustomControl for View

View Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomEntries Element for CustomControl for View

Provides the definitions of the custom control view. The custom control view must specify one or more definitions.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element

Syntax

XML

```
<CustomEntries>
  <CustomEntry>...</CustomEntry>
</CustomEntries>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `CustomControlEntries` element. You must specify one or more child elements.

Attributes

None.

Child Elements

Expand table

Element	Description
CustomEntry Element for CustomEntries for View	Required element.

Element	Description
	Provides a definition of the custom control view.

Parent Elements

Expand table

Element	Description
CustomControl Element for View	Required element. Defines a custom control format for the view.

Remarks

In most cases, a control has only one definition, which is defined in a single `CustomEntry` element. However it is possible to have multiple definitions if you want to use the same control to display different .NET objects. In those cases, you can define a `CustomEntry` element for each object or set of objects.

See Also

CustomControl Element for View

CustomEntry Element for CustomEntries for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomEntry Element for CustomEntries for CustomControl for View

Provides a definition of the custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element

Syntax

XML

```
<CustomEntry>
  <EntrySelectedBy>...</EntrySelectedBy>
  <CustomItem>...</CustomItem>
</CustomEntry>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `CustomEntry` element. You must specify the items displayed by the definition.

Attributes

None.

Child Elements

Expand table

Element	Description
EntrySelectedBy Element for CustomEntry for View	Optional element. Defines the .NET types that use the definition of the custom control view or the condition that must exist for this definition to be used.
CustomItem Element for CustomEntry for View	Defines a control for the custom control definition.

Parent Elements

Expand table

Element	Description
CustomEntries Element for CustomControl for View	Provides the definitions of the custom control view. The custom control view must specify one or more definitions.

Remarks

In most cases, only one definition is required for each custom control view, but it is possible to have multiple definitions if you want to use the same view to display different .NET objects. In those cases, you can provide a separate definition for each object or set of objects.

See Also

CustomControl Element for View

CustomItem Element for CustomEntry for View

EntrySelectedBy Element for CustomEntry for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomItem Element for CustomEntry for CustomControl for View

Defines what data is displayed by the custom control view and how it is displayed. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element

Syntax

XML

```
<CustomItem>
  <ExpressionBinding>...</ExpressionBinding>
  <Frame>...</Frame>
  <NewLine/>
  <Text>TextToDisplay</Text>
</CustomItem>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `CustomItem` element.

Attributes

None.

Child Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for CustomControl for View	Optional element. Defines the data that is displayed by the control.
Frame Element for CustomItem for CustomControl for View	Optional element. Defines what data is displayed by the custom control view and how it is displayed.
NewLine Element for CustomItem for Custom Control for View	Optional element. Adds a blank line to the display of the control.
Text Element for CustomItem for CustomControl for View	Optional element. Specifies additional text to the data displayed by the control.

Parent Elements

Expand table

Element	Description
CustomEntry Element for CustomEntries for CustomControl for View	Provides a definition of the custom control view.

Remarks

See Also

CustomEntry Element for CustomEntries for View

ExpressionBinding Element for CustomItem for CustomControl for View

Frame Element for CustomItem for CustomControl for View

NewLine Element for CustomItem for CustomControl for View

Text Element for CustomItem for CustomControl for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ExpressionBinding Element for CustomItem for CustomControl for View

Defines the data that is displayed by the control. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element

Syntax

XML

```
<ExpressionBinding>
  <CustomControl>...</CustomControl>
  <CustomControlName>NameofCommonCustomControl</CustomControlName>
  <EnumerateCollection/>
  <ItemSelectionCondition>...</ItemSelectionCondition>
  <PropertyName>Nameof.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate</ScriptBlock>
</ExpressionBinding>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ExpressionBinding` element.

Attributes

None.

Child Elements

Expand table

Element	Description
`CustomControl Element`	Optional element. Defines a control that is used by this control.
CustomControlName Element for ExpressionBinding for CustomControl for View	Optional element. Specifies the name of a common control or a view control.
EnumerateCollection Element for ExpressionBinding for CustomControl for View	Optional element. Specified that the elements of collections are displayed.
ItemSelectionCondition Element for ExpressionBinding for CustomControl for View	Optional element. Defines the condition that must exist for this control to be used.
PropertyName Element for ExpressionBinding for CustomControl for View	Optional element. Specifies the .NET property whose value is displayed by the control.
ScriptBlock Element for ExpressionBinding for CustomCustomControl for View	Optional element. Specifies the script whose value is displayed by the control.

Parent Elements

Expand table

Element	Description
CustomItem Element for CustomEntry for CustomControl for View	Defines what data is displayed by the custom control view and how it is displayed.

Remarks

See Also

CustomControlName Element for ExpressionBinding for CustomControl for View

EnumerateCollection Element for ExpressionBinding for CustomControl for View

ItemSelectionCondition Element for ExpressionBinding for CustomControl for View

PropertyName Element for ExpressionBinding for CustomControl for View

ScriptBlock Element for ExpressionBinding for CustomControl for View

CustomItem Element for CustomEntry for CustomControl for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomControlName Element for ExpressionBinding for CustomControl for View

Specifies the name of a common control or a view control. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- CustomControlName Element

Syntax

XML

```
<CustomControlName>NameofCustomControl</CustomControlName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `CustomControlName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem	Defines the data that is displayed by the control.

Text Value

Specify the name of the control.

Remarks

You can create common controls that can be used by all the views of a formatting file and you can create view controls that can be used by a specific view. The names of these controls are specified by the following elements.

- Name Element for Control for Controls for Configuration
- Name Element for Control for Controls for View

See Also

Name Element for Control for Controls for Configuration

Name Element for Control for Controls for View

ExpressionBinding Element for CustomItem

Writing a PowerShell Formatting File

Last updated on 05/20/2025

EnumerateCollection Element for ExpressionBinding for CustomControl for View

Specifies that the elements of collections are displayed. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- EnumerateCollection Element

Syntax

XML

```
<EnumerateCollection/>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `EnumerateCollection` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem	Defines the data that is displayed by the control.

Remarks

See Also

ExpressionBinding Element for CustomItem

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ItemSelectionCondition Element for ExpressionBinding for CustomControl

Defines the condition that must exist for this control to be used. There is no limit to the number of selection conditions that can be specified for a control item. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- ItemSelectionCondition Element

Syntax

XML

```xml
<ItemSelectionCondition>
  <PropertyName>.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate</ScriptBlock>
</ItemSelectionCondition>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ItemSelectionCondition` element.

Attributes

None.

Child Elements

Expand table

Element	Description
PropertyName Element for ItemSelectionCondition for CustomControl for View (Format	Optional element. Specifies the .NET property that triggers the condition.
ScriptBlock Element for ItemSelectionCondition for CustomControl for View	Optional element. Specifies the script that triggers the condition.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for CustomControl for View	Defines the data that is displayed by the control.

Remarks

You can specify one property name or a script for this condition but cannot specify both.

See Also

Writing a PowerShell Formatting File

ExpressionBinding Element for CustomItem for CustomControl for View

Last updated on 05/20/2025

PropertyName Element for ItemSelectionCondition for CustomControl for View

Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the control is used. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element for CustomControl for View
- CustomEntry Element for CustomEntries for View
- CustomItem Element for CustomEntry for View
- ExpressionBinding Element for CustomItem for CustomControl for View
- ItemSelectionCondition Element for Expression Binding for CustomControl for View
- PropertyName Element for ItemSelectionCondition for CustomControl for View (Format

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ItemSelectionCondition Element for Expression Binding for CustomControl for View	Defines the condition that must exist for this control to be used.

Text Value

Specify the name of the .NET property that triggers the condition.

Remarks

If this element is used, you cannot specify the ScriptBlock element when defining the selection condition.

See Also

ScriptBlock Element for ItemSelectionCondition for CustomControl for View

ItemSelectionCondition Element for Expression Binding for CustomControl for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for ItemSelectionCondition for CustomControl for View

Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the control is used. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- ItemSelectionCondition Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ItemSelectionCondition Element for Expression Binding for CustomControl for View	Defines the condition that must exist for this control to be used.

Text Value

Specify the script that is evaluated.

Remarks

If this element is used, you cannot specify the PropertyName element when defining the selection condition.

See Also

PropertyName Element for ItemSelectionCondition for CustomControl for View

ItemSelectionCondition Element for Expression Binding for CustomControl for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

PropertyName Element for ExpressionBinding for CustomControl for View

Specifies the .NET property whose value is displayed by the control. This element is used when defining a custom control view

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for CustomControl for View	Defines the data that is displayed by the control.

Text Value

Specify the name of the .NET property whose value is displayed by the control.

Remarks

See Also

ExpressionBinding Element for CustomItem for CustomControl for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for ExpressionBinding for CustomControl for View

Specifies the script whose value is displayed by the control. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for CustomControl for View	Defines the data that is displayed by the control.

Text Value

Specify the script whose value is displayed by the control.

Remarks

See Also

ExpressionBinding Element for CustomItem for CustomControl for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Frame Element for CustomItem for CustomControl for View

Defines how the data is displayed, such as shifting the data to the left or right. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element

Syntax

XML

```
<Frame>
  <LeftIndent>NumberOfCharactersToShift</LeftIndent>
  <RightIndent>NumberOfCharactersToShift</RightIndent>
  <FirstLineHanging>NumberOfCharactersToShift</FirstLineHanging>
  <FirstLineIndent>NumberOfCharactersToShift</FirstLineIndent>
  <CustomItem>...</CustomItem>
</Frame>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `Frame` element.

Attributes

None.

Child Elements

Expand table

Element	Description
`CustomItem Element`	Required Element
FirstLineHanging Element	Optional element. Specifies how many characters the first line of data is shifted to the left.
FirstLineIndent Element	Optional element. Specifies how many characters the first line of data is shifted to the right.
LeftIndent Element	Optional element. Specifies how many characters the data is shifted away from the left margin.
RightIndent Element	Optional element. Specifies how many characters the data is shifted away from the right margin.

Parent Elements

Expand table

Element	Description
CustomItem Element for CustomEntry for View	Defines what data is displayed by the control and how it is displayed.

Remarks

You cannot specify the FirstLineHanging and the FirstLineIndent elements in the same `Frame` element.

See Also

FirstLineHanging Element

FirstLineIndent Element

LeftIndent Element

RightIndent Element

CustomItem Element for CustomEntry for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

FirstLineHanging Element for Frame for CustomControl

Specifies how many characters the first line of data is shifted to the left. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element
- FirstLineHanging Element

Syntax

XML

```
<FirstLineHanging>NumberOfCharactersToShift</FirstLineHanging>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `FirstLineHanging` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Frame Element for CustomItem for CustomControl for View	Defines how the data is displayed, such as shifting the data to the left or right.

Text Value

Specify the number of characters that you want to shift the first line of the data.

Remarks

If this element is specified, you cannot specify the FirstLineIndent element.

See Also

FirstLineIndent Element for Frame for CustomControl for View

Frame Element for CustomItem for CustomControl for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

FirstLineIndent Element for Frame for CustomControl for View

Specifies how many characters the first line of data is shifted to the right. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element
- FirstLineIndent Element

Syntax

XML

```
<FirstLineIndent>NumberOfCharactersToShift</FirstLineIndent>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `FirstLineIndent` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Frame Element for CustomItem for CustomControl for View	Defines how the data is displayed, such as shifting the data to the left or right.

Text Value

Specify the number of characters that you want to shift the first line of the data.

Remarks

If this element is specified, you cannot specify the FirstLineHanging element.

See Also

FirstLineHanging Element for Frame for CustomControl for View

Frame Element for CustomItem for CustomControl for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

LeftIndent Element for Frame for CustomControl for View

Specifies how many characters the data is shifted away from the left margin. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element
- LeftIndent Element

Syntax

XML

```
<LeftIndent>CharactersToShift</LeftIndent>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `LeftIndent` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Frame Element for CustomItem for CustomControl for View	Defines how the data is displayed, such as shifting the data to the left or right.

Text Value

Specify the number of characters that you want to shift the data to the left.

Remarks

See Also

Frame Element for CustomItem for CustomControl for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

RightIndent Element for Frame for CustomControl for View

Specifies how many characters the data is shifted away from the right margin. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element
- RightIndent Element

Syntax

XML

```
<RightIndent>CharactersToShift</RightIndent>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `RightIndent` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Frame Element for CustomItem for CustomControl for View	Defines how the data is displayed, such as shifting the data to the left or right.

Text Value

Specify the number of characters that you want to shift the data to the right.

Remarks

See Also

Frame Element for CustomItem for CustomControl for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

NewLine Element for CustomItem for CustomControl for View

Adds a blank line to the display of the control. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- NewLine Element

Syntax

XML

```
<NewLine/>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `NewLine` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
CustomItem Element for CustomEntry for View	Defines a control for the custom control view.

Remarks

See Also

CustomItem Element for CustomEntry for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Text Element for CustomItem for CustomView for View

Specifies text that is added to the data that is displayed by the control, such as a label, brackets to enclose the data, and spaces to indent the data. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Text Element

Syntax

XML

```
<Text>TextToDisplay</Text>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `Text` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
CustomItem Element for CustomEntry for View	Defines a control for the custom control view.

Text Value

Specify the text of a control for data that you want to display.

Remarks

See Also

CustomItem Element for CustomEntry for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

EntrySelectedBy Element for CustomEntry for CustomControl for View

Defines the .NET types that use this custom entry or the condition that must exist for this entry to be used.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element

Syntax

XML

```
<EntrySelectedBy>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>NameofSelectionSet</SelectionSetName>
  <SelectionCondition>...</SelectionCondition>
</EntrySelectedBy>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `EntrySelectedBy` element.

Attributes

None.

Child Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for CustomEntry	Optional element. Defines the condition that must exist for this definition to be used.
SelectionSetName Element for EntrySelectedBy for CustomEntry	Optional element. Specifies a set of .NET types that use this definition of the control view.
TypeName Element for EntrySelectedBy for CustomEntry	Optional element. Specifies a .NET type that uses this definition of the control view.

Parent Elements

Expand table

Element	Description
CustomEntry Element for CustomEntries for View	Defines the controls used by specific .NET objects.

Remarks

You must specify at least one type, selection set, or selection condition for an entry. There is no maximum limit to the number of child elements that you can use.

Selection conditions are used to define a condition that must exist for the entry to be used, such as when an object has a specific property or when a specific property value or script evaluates to `true`. For more information about selection conditions, see Defining Conditions for when a View Entry or Item is Used.

For more information about the components of a custom control view, see Custom Control View.

See Also

SelectionCondition Element for EntrySelectedBy for CustomEntry

SelectionSetName Element for EntrySelectedBy for CustomEntry

TypeName Element for EntrySelectedBy for CustomEntry

CustomEntry Element for CustomEntries for View

Custom Control View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionCondition Element for EntrySelectedBy for CustomControl

Defines a condition that must exist for a control definition to be used. This element is used when defining a custom control view.

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- EntrySelectedBy Element
- SelectionCondition Element

Syntax

XML

```xml
<SelectionCondition>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>NameofSelectionSet</SelectionSetName>
  <PropertyName>.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate</ScriptBlock>
</SelectionCondition>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionCondition` element.

Attributes

None.

Child Elements

Expand table

Element	Description
PropertyName Element for SelectionCondition for CustomControl for View	Optional element. Specifies a .NET property that triggers the condition.
ScriptBlock Element for SelectionCondition for CustomControl for View	Optional element. Specifies the script that triggers the condition.
SelectionSetName Element for SelectionCondition for Custom Control for View	Optional element. Specifies the set of .NET types that triggers the condition.
TypeName Element for SelectionCondition for CustomControl for View	Optional element. Specifies a .NET type that triggers the condition.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for CustomEntry for CustomControl for View	Defines the .NET types that use this control definition or the condition that must exist for this definition to be used.

Remarks

When you are defining a selection condition, the following requirements apply:

- The selection condition must specify a least one property name or a script block, but cannot specify both.
- The selection condition can specify any number of .NET types or selection sets, but cannot specify both.

For more information about how to use selection conditions, see Defining Conditions for when Data is Displayed.

See Also

PropertyName Element for SelectionCondition for CustomControl for View

ScriptBlock Element for SelectionCondition for CustomControl for View

SelectionSetName Element for SelectionCondition for Custom Control for View

TypeName Element for SelectionCondition for CustomControl for View

EntrySelectedBy Element for CustomEntry for CustomControl for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

PropertyName Element for SelectionCondition for EntrySelectedBy for EnumerableExpansion

Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the definition is used.

Schema

- Configuration Element
- DefaultSettings Element
- EnumerableExpansions Element
- EnumerableExpansion Element
- EntrySelectedBy Element
- SelectionCondition Element
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for EnumerableExpansion	Defines the condition that must exist to expand the collection objects of this definition.

Text Value

Specify the .NET property name.

Remarks

The selection condition must specify at least one property name or a script to evaluate, but cannot specify both. For more information about how to use selection conditions, see Defining Conditions for when Data is Displayed.

See Also

Defining Conditions for When Data is Displayed

ScriptBlock Element for SelectionCondition for EntrySelectedBy for EnumerableExpansion

SelectionCondition Element for EntrySelectedBy for EnumerableExpansion

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for SelectionCondition for EntrySelectedBy for EnumerableExpansion

Specifies the script that triggers the condition.

Schema

- Configuration Element
- DefaultSettings Element
- EnumerableExpansions Element
- EnumerableExpansion Element
- EntrySelectedBy Element
- SelectionCondition Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for EnumerableExpansion	Defines the condition that must exist to expand the collection objects of this definition.

Text Value

Specify the script that is evaluated.

Remarks

The selection condition must specify at least one script or property name to evaluate, but cannot specify both. For more information about how to use selection conditions, see Defining Conditions for when Data is Displayed.

See Also

Defining Conditions for When Data Is Displayed

PropertyName Element for SelectionCondition for EntrySelectedBy for EnumerableExpansion

SelectionCondition Element for EntrySelectedBy for EnumerableExpansion

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSetName Element for SelectionCondition for CustomControl for View

Specifies the set of .NET types that trigger the condition. When any of the types in this set are present, the condition is met and the object is displayed using this control. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element

Syntax

XML

```
<SelectionSetName>NameofSelectionSet</SelectionSetName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionSetName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for CustomControl for View	Defines a condition that must exist for the control definition to be used.

Text Value

Specify the name of the selection set.

Remarks

Selection sets are common groups of .NET objects that can be used by any view that the formatting file defines. For more information about creating and referencing selection sets, see Defining Sets of Objects.

The selection condition can specify a selection set or .NET type, but cannot specify both. For more information about how to use selection conditions, see Defining Conditions for when Data is Displayed.

See Also

SelectionCondition Element for EntrySelectedBy for CustomControl for View

Defining Conditions for When Data Is Displayed

Defining Selection Sets

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for SelectionCondition for CustomControl for View

Specifies a .NET type that triggers the condition. This element is used when defining a custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- SelectionCondition Element
- TypeName Element

Syntax

XML

```
<TypeName>Nameof.NetType</TypeName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `TypeName` Element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for CustomControl for View	Defines a condition that must exist for the control definition to be used.

Text Value

Specify the fully qualified name of the .NET type, such as `System.IO.DirectoryInfo`.

Remarks

See Also

SelectionCondition Element for EntrySelectedBy for CustomControl for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSetName Element for EntrySelectedBy for CustomControl for View

Specifies a set of .NET objects for the list entry. There is no limit to the number of selection sets that can be specified for an entry.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionSetName Element

Syntax

XML

```
<SelectionSetName>NameofSelectionSet</SelectionSetName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionSetName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for CustomEntry for View	Defines the .NET types that use this custom entry or the condition that must exist for this entry to be used.

Text Value

Specify the name of the selection set.

Remarks

Each custom control entry must have at least one type name, selection set, or selection condition defined.

Selection sets are typically used when you want to define a group of objects that are used in multiple views. For example, you might want to create a table view and a list view for the same set of objects. For more information about defining selection sets, see Defining Selection Sets.

For more information about the components of a custom control view, see Creating Custom Controls.

See Also

EntrySelectedBy Element for CustomEntry for View

Custom Control View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for EntrySelectedBy for CustomEntry for View

Specifies a .NET type that uses this definition of the custom control view. There is no limit to the number of types that can be specified for a definition.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- TypeName Element

Syntax

XML

```
<TypeName>Nameof.NetType</TypeName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `TypeName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for CustomEntry for View	Defines the .NET types that use this custom control view definition or the condition that must exist for this definition to be used.

Text Value

Specify the fully qualified name of the .NET type, such as `System.IO.DirectoryInfo`.

Remarks

Each custom control view definition must have at least one type name, selection set, or selection condition defined.

For more information about the components of a custom control view, see Creating Custom Controls.

See Also

Creating Custom Controls

EntrySelectedBy Element for CustomEntry for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

GroupBy Element for View

Defines how a new group of objects is displayed. This element is used when defining a table, list, wide, or custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element

Syntax

XML

```
<GroupBy>
  <PropertyName>.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate</ScriptBlock>
  <Label>TextToDisplay</Label>
  <CustomControl>...</CustomControl>
  <CustomControlName>NameOfControl</CustomControlName>
</GroupBy>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

None.

Child Elements

Expand table

Element	Description
CustomControl Element for GroupBy	Optional element.

Element	Description
	Defines the custom control that display new groups.
CustomControlName Element for GroupBy	Optional element. Specifies the name of a control that is used to display the new group.
Label Element for GroupBy	Optional element. Specifies a label that is displayed when a new group is encountered.
PropertyName Element for GroupBy	Optional element. Specifies the .NET property the starts a new group whenever its value changes.
ScriptBlock Element for GroupBy	Optional element. Specifies the script that starts a new group whenever its value changes.

Parent Elements

Expand table

Element	Description
View Element	Defines a view that displays one or more .NET objects.

Remarks

When defining how a new group of objects is displayed, you must specify the property or script that will start the new group; however, you cannot specify both.

See Also

CustomControlName Element for GroupBy

Label Element for GroupBy

PropertyName Element for GroupBy

ScriptBlock Element for GroupBy

View Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomControl Element for GroupBy

Defines the custom control that displays the new group.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element

Syntax

XML

```
<CustomControl>
  <CustomEntries>...</CustomEntries>
<CustomControl>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `CustomControl` element. You can specify any number of child elements and list them in any order.

Attributes

None.

Child Elements

Expand table

Element	Description
CustomEntries Element for CustomControl for GroupBy	Required element. Provides the definitions for the control.

Parent Elements

Expand table

Element	Description
GroupBy Element for View	Defines how Windows PowerShell displays a new group of objects.

Remarks

See Also

CustomEntries Element for CustomControl for GroupBy

GroupBy Element for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomEntries Element for CustomControl for GroupBy

Provides the definitions for the control. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element

Syntax

XML

```
<CustomEntries>
  <CustomEntry>...</CustomEntry>
</CustomEntries>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements of the `CustomEntries` element. There is no maximum limit to the number of child elements that can be specified.

Attributes

None.

Child Elements

Expand table

Element	Description
CustomEntry Element for CustomControl for GroupBy	Required element. Provides a definition of the control.

Parent Elements

Expand table

Element	Description
CustomControl Element for GroupBy	Defines the custom control that displays the new group.

Remarks

In most cases, a control has only one definition, which is specified in a single `CustomEntry` element. However, it is possible to provide multiple definitions if you want to use the same control to display different groups. In those cases, you can define a `CustomEntry` element for a group.

See Also

CustomEntry Element for CustomEntries for Controls for View

CustomControl Element for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomEntry Element for CustomControl for GroupBy

Provides a definition of the control. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element

Syntax

XML

```
<CustomEntry>
  <EntrySelectedBy>...</EntrySelectedBy>
  <CustomItem>...</CustomItem>
</CustomEntry>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent elements of the `CustomEntry` element.

Attributes

None.

Child Elements

Expand table

Element	Description
EntrySelectedBy Element for CustomEntry for GroupBy	Optional element. Defines the .NET types that use this control definition or the condition that must exist for this definition to be used.
CustomItem Element for CustomEntry for GroupBy	Required element. Defines how the control displays the data.

Parent Elements

Expand table

Element	Description
CustomEntries Element for CustomControl for GroupBy	Provides the definitions for the control.

Remarks

See Also

EntrySelectedBy Element for CustomEntry for GroupBy

CustomItem Element for CustomEntry for GroupBy

CustomEntries Element for CustomControl for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomItem Element for CustomEntry for GroupBy

Defines what data is displayed by the custom control view and how it is displayed. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomItem Element

Syntax

XML

```
<CustomItem>
  <ExpressionBinding>...</ExpressionBinding>
  <Frame>...</Frame>
  <NewLine/>
  <Text>TextToDisplay</Text>
</CustomItem>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `CustomItem` element.

Attributes

None.

Child Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for GroupBy	Optional element. Defines the data that is displayed by the control.
Frame Element for CustomItem for GroupBy	Optional element. Defines what data is displayed by the custom control view and how it is displayed.
NewLine Element for CustomItem for GroupBy	Optional element. Adds a blank line to the display of the control.
Text Element for CustomItem for GroupBy	Optional element. Specifies additional text to the data displayed by the control.

Parent Elements

Expand table

Element	Description
CustomEntry Element for CustomControl for GroupBy	Provides a definition of the custom control view.

Remarks

See Also

CustomEntry Element for CustomControl for GroupBy

ExpressionBinding Element for CustomItem for GroupBy

Frame Element for CustomItem for GroupBy

NewLine Element for CustomItem for GroupBy

Text Element for CustomItem for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ExpressionBinding Element for CustomItem for GroupBy

Defines the data that is displayed by the control. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element

Syntax

XML

```
<ExpressionBinding>
  <CustomControl>...</CustomControl>
  <CustomControlName>NameofCommonCustomControl</CustomControlName>
  <EnumerateCollection/>
  <ItemSelectionCondition>...</ItemSelectionCondition>
  <PropertyName>Nameof.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate></ScriptBlock>
</ExpressionBinding>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ExpressionBinding` element.

Attributes

None.

Child Elements

Expand table

Element	Description
`CustomControl Element`	Optional element. Defines a control that is used by this control.
CustomControlName Element for ExpressionBinding for GroupBy	Optional element. Specifies the name of a common control or a view control.
EnumerateCollection Element for ExpressionBinding for GroupByEnumerateCollection Element for ExpressionBinding for GroupBy	Optional element. Specified that the elements of collections are displayed.
ItemSelectionCondition Element for ExpressionBinding for GroupBy	Optional element. Defines the condition that must exist for this control to be used.
PropertyName Element for ExpressionBinding for GroupBy	Optional element. Specifies the .NET property whose value is displayed by the control.
ScriptBlock Element for ExpressionBinding for GroupBy	Optional element. Specifies the script whose value is displayed by the control.

Parent Elements

Expand table

Element	Description
CustomItem Element for CustomEntry for GroupBy	Defines what data is displayed by the custom control view and how it is displayed.

See Also

CustomControlName Element for ExpressionBinding for GroupBy

EnumerateCollection Element for ExpressionBinding for GroupBy

ItemSelectionCondition Element for ExpressionBinding for GroupBy

PropertyName Element for ExpressionBinding for GroupBy

ScriptBlock Element for ExpressionBinding for GroupBy

CustomItem Element for CustomEntry for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomControlName Element for ExpressionBinding for GroupBy

Specifies the name of a common control or a view control. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- CustomControlName Element

Syntax

XML

```
<CustomControlName>NameofCustomControl</CustomControlName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `CustomControlName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for GroupBy	Defines the data that is displayed by the control.

Text Value

Specify the name of the control.

Remarks

You can create common controls that can be used by all the views of a formatting file, and you can create view controls that can be used by a specific view. The following elements specify the names of these controls:

- Name Element for Control for Controls for Configuration
- Name Element for Control for Controls for View

See Also

Name Element for Control for Controls for Configuration

Name Element for Control for Controls for View

ExpressionBinding Element for CustomItem for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

EnumerateCollection Element for ExpressionBinding for GroupBy

Specifies that the elements of collections are displayed. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- EnumerateCollection Element

Syntax

XML

```
<EnumerateCollection/>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `EnumerateCollection` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for GroupBy	Defines the data that is displayed by the control.

Remarks

See Also

ExpressionBinding Element for CustomItem for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ItemSelectionCondition Element for ExpressionBinding for GroupBy

Defines the condition that must exist for this control to be used. There is no limit to the number of selection conditions that can be specified for a control item. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- ItemSelectionCondition Element

Syntax

XML

```
<ItemSelectionCondition>
  <PropertyName>.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate</ScriptBlock>
</ItemSelectionCondition>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ItemSelectionCondition` element.

Attributes

None.

Child Elements

Expand table

Element	Description
PropertyName Element for ItemSelectionCondition for GroupBy	Optional element. Specifies the .NET property that triggers the condition.
ScriptBlock Element for ItemSelectionCondition for GroupBy	Optional element. Specifies the script that triggers the condition.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for GroupBy	Defines the data that is displayed by the control.

Remarks

You can specify one property name or a script for this condition but cannot specify both.

See Also

Writing a PowerShell Formatting File

ExpressionBinding Element for CustomItem for GroupBy

Last updated on 05/20/2025

PropertyName Element for ItemSelectionCondition for GroupBy

Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the control is used. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- ItemSelectionCondition Element
- PropertyName Element

Syntax

XML

```xml
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ItemSelectionCondition Element for ExpressionBinding for GroupBy	Defines the condition that must exist for this control to be used.

Text Value

Specify the name of the .NET property that triggers the condition.

Remarks

If this element is used, you cannot specify the ScriptBlock element when defining the selection condition.

See Also

ScriptBlock Element for ItemSelectionCondition for GroupBy

ItemSelectionCondition Element for ExpressionBinding for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for ItemSelectionCondition for GroupBy

Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the control is used. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- ItemSelectionCondition Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ItemSelectionCondition Element for ExpressionBinding for GroupBy	Defines the condition that must exist for this control to be used.

Text Value

Specify the script that is evaluated.

Remarks

If this element is used, you cannot specify the PropertyName element when defining the selection condition.

See Also

ItemSelectionCondition Element for ExpressionBinding for GroupBy

PropertyName Element for ItemSelectionCondition for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

PropertyName Element for ExpressionBinding for GroupBy

Specifies the .NET property whose value is displayed by the control. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for GroupBy	Defines the data that is displayed by the control.

Text Value

Specify the name of the .NET property whose value is displayed by the control.

Remarks

See Also

ExpressionBinding Element for CustomItem for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for ExpressionBinding for GroupBy

Specifies the script whose value is displayed by the control. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- ExpressionBinding Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ExpressionBinding Element for CustomItem for GroupBy	Defines the data that is displayed by the control.

Text Value

Specify the script whose value is displayed by the control.

Remarks

See Also

ExpressionBinding Element for CustomItem for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Frame Element for CustomItem for GroupBy

Defines how the data is displayed, such as shifting the data to the left or right. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element

Syntax

XML

```
<Frame>
  <LeftIndent>NumberOfCharactersToShift</LeftIndent>
  <RightIndent>NumberOfCharactersToShift</RightIndent>
  <FirstLineHanging>NumberOfCharactersToShift</FirstLineHanging>
  <FirstLineIndent>NumberOfCharactersToShift</FirstLineIndent>
  <CustomItem>...</CustomItem>
</Frame>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `Frame` element.

Attributes

None.

Child Elements

Expand table

Element	Description
`CustomItem Element`	Required Element
FirstLineHanging Element for Frame for GroupBy	Optional element. Specifies how many characters the first line of data is shifted to the left.
FirstLineIndent Element for Frame for GroupBy	Optional element. Specifies how many characters the first line of data is shifted to the right.
LeftIndent Element for Frame for GroupBy	Optional element. Specifies how many characters the data is shifted away from the left margin.
RightIndent Element for Frame for GroupByRightIndent Element	Optional element. Specifies how many characters the data is shifted away from the right margin.

Parent Elements

Expand table

Element	Description
CustomItem Element for CustomEntry for GroupBy	Defines what data is displayed by the control and how it is displayed.

Remarks

You cannot specify the FirstLineHanging and the FirstLineIndent elements in the same `Frame` element.

See Also

FirstLineHanging Element for Frame for GroupBy

FirstLineIndent Element for Frame for GroupBy

LeftIndent Element for Frame for GroupBy

RightIndent Element for Frame for GroupBy

CustomItem Element for CustomEntry for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

FirstLineHanging Element for Frame for GroupBy

Specifies how many characters the first line of data is shifted to the left. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element
- FirstLineHanging Element

Syntax

XML

```
<FirstLineHanging>NumberOfCharactersToShift</FirstLineHanging>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `FirstLineHanging` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Frame Element for CustomItem for GroupBy	Defines how the data is displayed, such as shifting the data to the left or right.

Text Value

Specify the number of characters that you want to shift the first line of the data.

Remarks

If this element is specified, you cannot specify the FirstLineIndent element.

See Also

FirstLineIndent Element for Frame for GroupBy

Frame Element for CustomItem for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

FirstLineIndent Element for Frame for GroupBy

Specifies how many characters the first line of data is shifted to the right. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element for View
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element
- FirstLineIndent Element

Syntax

XML

```
<FirstLineIndent>NumberOfCharactersToShift</FirstLineIndent>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `FirstLineIndent` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Frame Element for CustomItem for GroupBy	Defines how the data is displayed, such as shifting the data to the left or right.

Text Value

Specify the number of characters that you want to shift the first line of the data.

Remarks

If this element is specified, you cannot specify the FirstLineHanging element.

See Also

FirstLineHanging Element for Frame for GroupBy

Frame Element for CustomItem for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

LeftIndent Element for Frame

Specifies how many characters the data is shifted away from the left margin. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element
- LeftIndent Element

Syntax

XML

```
<LeftIndent>CharactersToShift</LeftIndent>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `LeftIndent` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Frame Element for CustomItem for GroupBy	Defines how the data is displayed, such as shifting the data to the left or right.

Text Value

Specify the number of characters that you want to shift the data to the left.

Remarks

See Also

Frame Element for CustomItem for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

RightIndent Element for Frame for GroupBy

Specifies how many characters the data is shifted away from the right margin. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Frame Element
- RightIndent Element

Syntax

XML

```
<RightIndent>CharactersToShift</RightIndent>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `RightIndent` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
Frame Element for CustomItem for GroupBy	Defines how the data is displayed, such as shifting the data to the left or right.

Text Value

Specify the number of characters that you want to shift the data to the right.

Remarks

See Also

Frame Element for CustomItem for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

NewLine Element for CustomItem for GroupBy

Adds a blank line to the display of the control. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- NewLine Element

Syntax

XML

```
<NewLine/>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `NewLine` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
CustomItem Element for CustomEntry for GroupBy	Defines a control for the custom control view.

Remarks

See Also

CustomItem Element for CustomEntry for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Text Element for CustomItem for GroupBy

Specifies text that is added to the data that is displayed by the control, such as a label, brackets to enclose the data, and spaces to indent the data. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- CustomItem Element
- Text Element

Syntax

XML

```
<Text>TextToDisplay</Text>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `Text` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
CustomItem Element for CustomEntry for GroupBy	Defines a control for the custom control view.

Text Value

Specify the text of a control for data that you want to display.

Remarks

See Also

CustomItem Element for CustomEntry for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

EntrySelectedBy Element for CustomEntry for GroupBy

Defines the .NET types that use this control definition or the condition that must exist for this definition to be used. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element for View
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element

Syntax

XML

```xml
<EntrySelectedBy>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>NameofSelectionSet</SelectionSetName>
  <SelectionCondition>...</SelectionCondition>
</EntrySelectedBy>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `EntrySelectedBy` element. You must specify at least one type, selection set, or selection condition for a definition. There is no maximum limit to the number of child elements that you can use.

Attributes

None.

Child Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for GroupBy	Optional element. Defines the condition that must exist for this definition to be used.
SelectionSetName Element for EntrySelectedBy for GroupBy	Optional element. Specifies a set of .NET types that use this definition of the control.
TypeName Element for EntrySelectedBy for GroupBy	Optional element. Specifies a .NET type that uses this definition of the control.

Parent Elements

Expand table

Element	Description
CustomEntry Element for CustomControl for GroupBy	Provides a definition of the control.

Remarks

Selection conditions are used to define a condition that must exist for the definition to be used, such as when an object has a specific property or when a specific property value or script evaluates to `true`. For more information about selection conditions, see Defining Conditions for when a View Entry or Item is Used.

See Also

SelectionCondition Element for EntrySelectedBy for GroupBy

SelectionSetName Element for EntrySelectedBy for GroupBy

TypeName Element for EntrySelectedBy for GroupBy

CustomEntry Element for CustomEntries for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionCondition Element for EntrySelectedBy for GroupBy

Defines a condition that must exist for a control definition to be used. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element for View
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionCondition Element

Syntax

XML

```
<SelectionCondition>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>NameofSelectionSet</SelectionSetName>
  <PropertyName>.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate</ScriptBlock>
</SelectionCondition>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionCondition` element.

Attributes

None.

Child Elements

Expand table

Element	Description
PropertyName Element for SelectionCondition for GroupBy	Optional element. Specifies a .NET property that triggers the condition.
ScriptBlock Element for SelectionCondition for GroupBy	Optional element. Specifies the script that triggers the condition.
SelectionSetName Element for SelectionCondition for GroupBy	Optional element. Specifies the set of .NET types that triggers the condition.
TypeName Element for SelectionCondition for GroupBy	Optional element. Specifies a .NET type that triggers the condition.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for CustomEntry for GroupBy	Defines the .NET types that use this control definition or the condition that must exist for this definition to be used.

Remarks

When you are defining a selection condition, the following requirements apply:

- The selection condition must specify a least one property name or a script block, but cannot specify both.
- The selection condition can specify any number of .NET types or selection sets, but cannot specify

  both. For more information about how to use selection conditions, see Defining Conditions for when Data is Displayed.

See Also

PropertyName Element for SelectionCondition for CustomControl for View

ScriptBlock Element for SelectionCondition for CustomControl for View

SelectionSetName Element for SelectionCondition for Custom Control for View

TypeName Element for SelectionCondition for GroupBy

EntrySelectedBy Element for CustomEntry for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

PropertyName Element for SelectionCondition for GroupBy

Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the definition is used. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for GroupBy	Defines a condition that must exist for the control definition to be used.

Text Value

Specify the .NET property name.

Remarks

The selection condition must specify a least one property name or a script, but cannot specify both. For more information about how selection conditions can be used, see Defining Conditions for Displaying Data.

See Also

SelectionCondition Element for EntrySelectedBy for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for SelectionCondition for EntrySelectedBy for GroupBy

Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the definition is used. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for GroupBy	Defines a condition that must exist for the control definition to be used.

Text Value

Specify the script that is evaluated.

Remarks

The selection condition must specify a least one script or property name to evaluate, but cannot specify both. For more information about how selection conditions can be used, see Defining Conditions for Displaying Data.

See Also

SelectionCondition Element for EntrySelectedBy for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSetName Element for SelectionCondition for GroupBy

Specifies the set of .NET types that trigger the condition. When any of the types in this set are present, the condition is met, and the object is displayed by using this control. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- SelectionSetName Element

Syntax

XML

```xml
<SelectionSetName>NameofSelectionSet</SelectionSetName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionSetName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for GroupBy	Defines a condition that must exist for the control definition to be used.

Text Value

Specify the name of the selection set.

Remarks

Selection sets are common groups of .NET objects that can be used by any view that the formatting file defines. For more information about creating and referencing selection sets, see Defining Selection Sets.

When this element is specified, you cannot specify the TypeName element. For more information about defining selection conditions, see Defining Conditions for Displaying Data.

See Also

TypeName Element for SelectionCondition for GroupBy

SelectionCondition Element for EntrySelectedBy for GroupBy

Defining Conditions for When Data Is Displayed

Defining Selection Sets

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for SelectionCondition for GroupBy

Specifies a .NET type that triggers the condition. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- TypeName Element

Syntax

XML

```xml
<TypeName>Nameof.NetType</TypeName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `TypeName` Element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for GroupBy	Defines a condition that must exist for the control definition to be used.

Text Value

Specify the fully qualified name of the .NET type, such as `System.IO.DirectoryInfo`.

Remarks

When this element is specified, you cannot specify the `SelectionSetName` element. For more information about defining selection conditions, see Defining Conditions for Displaying Data.

See Also

SelectionCondition Element for EntrySelectedBy for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSetName Element for EntrySelectedBy for GroupBy

Specifies a set of .NET objects for the list entry. There is no limit to the number of selection sets that can be specified for an entry. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- SelectionSetName Element

Syntax

XML

```
<SelectionSetName>NameofSelectionSet</SelectionSetName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionSetName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for CustomEntry for GroupBy	Defines the .NET types that use this custom entry or the condition that must exist for this entry to be used.

Text Value

Specify the name of the selection set.

Remarks

Each custom control definition must have at least one type name, selection set, or selection condition defined.

Selection sets are typically used when you want to define a group of objects that are used in multiple views. For example, you may want to create a table view and a list view for the same set of objects. For more information about defining selection sets, see Defining Selection Sets.

For more information about the components of a custom control view, see Creating Custom Controls.

See Also

EntrySelectedBy Element for CustomEntry for GroupBy

Creating Custom Controls

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for EntrySelectedBy for GroupBy

Specifies a .NET type that uses this definition of the custom control. This element is used when defining how a new group of objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControl Element
- CustomEntries Element
- CustomEntry Element
- EntrySelectedBy Element
- TypeName Element

Syntax

XML

```
<TypeName>Nameof.NetType</TypeName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `TypeName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for CustomEntry for GroupBy	Defines the .NET types that use this control definition or the condition that must exist for this definition to be used.

Text Value

Specify the fully qualified name of the .NET type, such as `System.IO.DirectoryInfo`.

Remarks

Each control definition must have at least one type name, selection set, or selection condition defined.

For more information about the components of a custom control view, see Creating Custom Controls.

See Also

Creating Custom Controls

EntrySelectedBy Element for CustomEntry for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

CustomControlName Element for GroupBy

Specifies the name of a custom control that is used to display the new group. This element is used when defining a table, list, wide or custom control view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- CustomControlName Element

Syntax

XML

```xml
<CustomControlName>ControlName</CustomControlName>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent elements of the `CustomControlName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
GroupBy Element for View	Defines how Windows PowerShell displays a new group of objects.

Text Value

Specify the name of the custom control that is used to display a new group.

Remarks

You can create common controls that can be used by all the views of a formatting file, and you can create view controls that can be used by a specific view. The following elements specify the names of these custom controls:

- Name Element for Control for Controls for Configuration
- Name Element for Control for Controls for View

See Also

GroupBy Element for View

Name Element for Control for Controls for Configuration

Name Element for Control for Controls for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Label Element for GroupBy

Specifies a label that is displayed when a new group is encountered.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- Label Element

Syntax

XML

```xml
<Label>DisplayedLabel</Label>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `Label` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
GroupBy Element for View	Defines how a new group of objects is displayed.

Text Value

Specify the text that is displayed whenever Windows PowerShell encounters a new property or script value.

Remarks

In addition to the text specified by this element, Windows PowerShell displays the new value that starts the group, and adds a blank line before and after the group.

Example

The following example shows the label for a new group. The displayed label would look similar to this: `Service Type: NewValueofProperty`

XML

```xml
<GroupBy>
  <Label>Service Type</Label>
  <PropertyName>ServiceType</PropertyName>
</GroupBy>
```

For an example of a complete formatting file that includes this element, see Wide View (GroupBy).

See Also

GroupBy Element for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

PropertyName Element for GroupBy

Specifies the .NET property that starts a new group whenever its value changes.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
GroupBy Element for View	Defines how a group of .NET objects is displayed.

Text Value

Specify the .NET property name.

Remarks

Windows PowerShell starts a new group whenever the value of this property changes.

When this element is specified, you cannot specify the ScriptBlock element to start a new group.

Example

The following example shows how to start a new group when the value of a property changes.

XML

```
<GroupBy>
  <Label>Service Type</Label>
  <PropertyName>ServiceType</PropertyName>
</GroupBy>
```

For an example of a complete formatting file that includes this element, see Wide View (GroupBy).

See Also

GroupBy Element for View

ScriptBlock Element for GroupBy

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for GroupBy

Specifies the script that starts a new group whenever its value changes.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- GroupBy Element
- ScriptBlock Element

Syntax

XML

```xml
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
GroupBy Element for View	Defines how a group of .NET objects is displayed.

Text Value

Specify the script that is evaluated.

Remarks

PowerShell starts a new group whenever the value of this script changes.

When this element is specified, you cannot specify the PropertyName element to start a new group.

See Also

PropertyName Element for GroupBy

GroupBy Element for View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ListControl Element

Defines a list format for the view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element

Syntax

XML

```
<ListControl>
  <ListEntries>...</ListEntries>
</ListControl>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `ListControl` element. This element must contain only a single child element.

Attributes

None.

Child Elements

Expand table

Element	Description
ListEntries Element	Required element. Provides the definitions of the list view.

Parent Elements

Expand table

Element	Description
View Element	Defines a view that is used to display the members of one or more objects.

Remarks

For more information about creating a list view, see Creating a List View.

Example

This example shows a list view for the System.ServiceProcess.ServiceController object.

```
<View>
  <Name>System.ServiceProcess.ServiceController</Name>
  <ViewSelectedBy>
    <TypeName>System.ServiceProcess.ServiceController</TypeName>
  </ViewSelectedBy>
  <ListControl>
    <ListEntries>
      <ListEntry>...</ListEntry>
    </ListEntries>
  </ListControl>
</View>
```

See Also

View Element

ListEntries Element

Creating a List View

Writing a Windows PowerShell Formatting and Types File

Last updated on 05/20/2025

ListEntries Element

Provides the definitions of the list view. The list view must specify one or more definitions.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element

Syntax

XML

```
<ListEntries>
  <ListEntry>...</ListEntry>
</ListEntries>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `ListEntries` element. At least one child element must be specified.

Attributes

None.

Child Elements

Expand table

Element	Description
ListEntry Element	Provides a definition of the list view.

Parent Elements

Expand table

Element	Description
ListControl Element	Defines a list format for the view.

Remarks

For more information about list views, see List View.

Example

This example shows the XML elements that define the list view for the System.ServiceProcess.ServiceController object.

XML

```
<View>
  <Name>System.ServiceProcess.ServiceController</Name>
  <ViewSelectedBy>
    <TypeName>System.ServiceProcess.ServiceController</TypeName>
  </ViewSelectedBy>
  <ListControl>
    <ListEntries>
      <ListEntry>
        <ListItems>...</ListItems>
      </ListEntry>
    </ListEntries>
  </ListControl>
</View>
```

See Also

ListControl Element

ListEntry Element

List View

Writing a Windows PowerShell Formatting and Types File

Last updated on 05/20/2025

ListEntry Element

Provides a definition of the list view.

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element

Syntax

XML

```
<ListEntry>
  <EntrySelectedBy>...</EntrySelectedBy>
  <ListItems>...</ListItems>
</ListEntry>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `ListEntry` element.

Attributes

None.

Child Elements

Expand table

Element	Description
EntrySelectedBy Element for ListEntry	Optional element. Defines the .NET objects that use this list view definition or the condition that must exist for this definition to be used.

Element	Description
ListItems Element	Required element. Defines the properties and scripts whose values are displayed by the list view.

Parent Elements

Expand table

Element	Description
ListEntries Element	Provides the definitions of the list view.

Remarks

A list view is a list format that displays property values or script values for each object. For more information about list views, see Creating a List View.

Example

This example shows the XML elements that define the list view for the System.ServiceProcess.ServiceController object.

XML

```
<View>
  <Name>System.ServiceProcess.ServiceController</Name>
  <ViewSelectedBy>
    <TypeName>System.ServiceProcess.ServiceController</TypeName>
  </ViewSelectedBy>
  <ListControl>
    <ListEntries>
      <ListEntry>
        <ListItems>...</ListItems>
      </ListEntry>
    </ListEntries>
  </ListControl>
</View>
```

See Also

Creating a List View

EntrySelectedBy Element for ListEntry

ListEntries Element

ListItems Element

Writing a Windows PowerShell Formatting and Types File

Last updated on 05/20/2025

EntrySelectedBy Element for ListEntry

Defines the .NET types that use this list view definition or the condition that must exist for this definition to be used. In most cases only one definition is needed for a list view. However, you can provide multiple definitions for the list view if you want to use the same list view to display different data for different objects.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element
- EntrySelectedBy Element

Syntax

XML

```xml
<EntrySelectedBy>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>NameofSelectionSet</SelectionSetName>
  <SelectionCondition>...</SelectionCondition>
</EntrySelectedBy>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `EntrySelectedBy` element.

Attributes

None.

Child Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for ListControl	Optional element. Defines the condition that must exist for this list view definition to be used.
SelectionSetName Element for EntrySelectedBy for ListControl	Optional element. Specifies a set of .NET types that use this list view definition.
TypeName Element for EntrySelectedBy for ListControl	Optional element. Specifies a .NET type that uses this list view definition.

Parent Elements

Expand table

Element	Description
ListEntry Element for ListControl	Defines how the rows of the list are displayed.

Remarks

You must specify at least one type, selection set, or selection condition for a list view definition. There is no maximum limit to the number of child elements that you can use.

Selection conditions are used to define a condition that must exist for the definition to be used, such as when an object has a specific property or that a specific property value or script evaluates to `true`. For more information about selection conditions, see Defining Conditions for when Data is displayed.

For more information about the components of a list view, see Creating a List View.

Example

The following example shows how to define the objects for a list view using their .NET type name.

XML

```
<ListEntry>
  <EntrySelectedBy>
    <TypeName>NameofDotNetType</TypeName>>
  </EntrySelectedBy>
</ListEntry>
```

See Also

ListEntry Element for ListControl

SelectionCondition Element for EntrySelectedBy for ListControl

SelectionSetName Element for EntrySelectedBy for ListControl

TypeName Element for EntrySelectedBy for ListControl

Creating a List View

Defining Conditions for when Data is Displayed

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionCondition Element for EntrySelectedBy for ListControl

Defines the condition that must exist to use this definition of the list view. There is no limit to the number of selection conditions that can be specified for a list definition.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element
- EntrySelectedBy Element
- SelectionCondition Element

Syntax

XML

```xml
<SelectionCondition>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>NameofSelectionSet</SelectionSetName>
  <PropertyName>.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate</ScriptBlock>
</SelectionCondition>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionCondition` element.

Attributes

None.

Child Elements

Expand table

Element	Description
PropertyName Element for SelectionCondition for EntrySelectedBy for ListEntry	Optional element. Specifies the .NET property that triggers the condition.
ScriptBlock Element for SelectionCondition for EntrySelectedBy for ListEntry	Optional element. Specifies the script that triggers the condition.
SelectionSetName Element for SelectionCondition for EntrySelectedBy for ListEntry	Optional element. Specifies the set of .NET types that trigger the condition.
TypeName Element for SelectionCondition for EntrySelectedBy for ListEntry	Optional element. Specifies a .NET type that triggers the condition.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for TableRowEntry	Defines the .NET types that use this table entry or the condition that must exist for this entry to be used.

Remarks

lWhen you are defining a selection condition, the following requirements apply:

- The selection condition must specify a least one property name or a script block, but cannot specify both.
- The selection condition can specify any number of .NET types or selection sets, but cannot specify both.

For more information about how to use selection conditions, see Defining Conditions for when Data is Displayed.

For more information about other components of a list view, see Creating a List View.

See Also

Creating a List View

Defining Conditions for When Data Is Displayed

ListEntry Element

SelectionSetName Element for EntrySelectedBy for ListEntry

TypeName Element for EntrySelectedBy for ListEntry

Writing a PowerShell Formatting File

Last updated on 05/20/2025

PropertyName Element for SelectionCondition for EntrySelectedBy for ListControl

Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the list entry is used.

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for ListEntry	Defines the condition that must exist for this list entry to be used.

Text Value

Specify the .NET property name.

Remarks

The selection condition must specify at least one property name or a script block, but cannot specify both. For more information about how to use selection conditions, see Defining Conditions for when a View Entry or Item is Used.

For more information about other components of a list view, see Creating List View.

See Also

Creating a List View

Defining Conditions for When Data is Displayed

ListEntry Element

ScriptBlock Element for SelectionCondition for EntrySelectedBy for ListEntry

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for SelectionCondition for EntrySelectedBy for ListControl

Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the list entry is used.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for ListEntry	Defines the condition that must exist for this list entry to be used.

Text Value

Specify the script that is evaluated.

Remarks

The selection condition must specify a least one script or property name to evaluate, but cannot specify both. (For more information about how selection conditions can be used, see Defining Conditions for when a View Entry or Item is Used.)

For more information about the other components of a list view, see List View.

See Also

ListEntry Element

PropertyName Element for SelectionCondition for EntrySelectedBy for ListEntry

SelectionCondition Element for EntrySelectedBy for ListEntry

List View

Defining Conditions for when a View Entry or Item is Used

Writing a Windows PowerShell Formatting and Types File

Last updated on 05/20/2025

SelectionSetName Element for SelectionCondition for EntrySelectedBy for ListEntry

Specifies the set of .NET types that trigger the condition. When any of the types in this set are present, the condition is met, and the object is displayed by using this definition of the list view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- SelectionSetName Element

Syntax

XML

```
<SelectionSetName>NameofSelectionSet</SelectionSetName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionSetName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for ListEntry	Defines the condition that must exist to use this definition of the list view.

Text Value

Specify the name of the selection set.

Remarks

The selection condition can specify a selection set or .NET type, but cannot specify both. For more information about how to use selection conditions, see Defining Conditions for when Data is Displayed.

Selection sets are common groups of .NET objects that can be used by any view that the formatting file defines. For more information about creating and referencing selection sets, see Defining Sets of Objects.

For more information about other components of a list view, see Creating a List View.

See Also

Creating a List View

Defining Conditions for When Data Is Displayed

SelectionCondition Element for EntrySelectedBy for ListEntry

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for SelectionCondition for EntrySelectedBy for ListControl

Specifies a .NET type that triggers the condition. When this type is present, the list entry is used.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- TypeName Element

Syntax

XML

```
<TypeName>Nameof.NetType</TypeName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `TypeName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for ListControl	Defines the condition that must exist for this list entry to be used.

Text Value

Specify the fully qualified name of the .NET type, such as `System.IO.DirectoryInfo`.

Remarks

The selection condition can specify any number of .NET types or selection sets, but cannot specify both. For more information about how to use selection conditions, see Defining Conditions for when Data is Displayed.

For more information about other the components of a list view, see Creating a List View.

See Also

Creating a List View

Defining Conditions for When Data Is Displayed

SelectionCondition Element for EntrySelectedBy for ListControl

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSetName Element for EntrySelectedBy for ListControl

Specifies a set of .NET objects for the list entry. There is no limit to the number of selection sets that can be specified for an entry.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element
- EntrySelectedBy Element
- SelectionSetName Element

Syntax

XML

```
<SelectionSetName>NameofSelectionSet</SelectionSetName>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `SelectionSetName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for ListEntry	Defines the .NET types that use this list entry or the condition that must exist for this entry to be used.

Text Value

Specify the name of the selection set.

Remarks

Each list entry must have at least one type name, selection set, or selection condition defined.

Selection sets are typically used when you want to define a group of objects that are used in multiple views. For example, you might want to create a table view and a list view for the same set of objects. For more information about defining selection sets, see Defining Sets of objects for a View.

For more information about the components of a list view, see Creating a List View.

Example

The following example shows how to specify a selection set for an entry of a list view.

XML

```
<ListEntry>
  <EntrySelectedBy>
    <SelectionSetName>NameofSelectionSet</SelectionSetName>
  </EntrySelectedBy>
  <ListItems>...</ListItems>
</ListEntry>
```

See Also

Creating a List View

EntrySelectedBy Element for ListEntry

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for EntrySelectedBy for ListControl

Specifies a .NET type that uses this entry of the list view. There is no limit to the number of types that can be specified for a list entry.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element
- EntrySelectedBy
- TypeName Element

Syntax

XML

```
<TypeName>Nameof.NetType</TypeName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `TypeName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for ListEntry	Defines the .NET types that use this list entry or the condition that must exist for this entry to be used.

Text Value

Specify the fully-qualified name of the .NET type, such as `System.IO.DirectoryInfo`.

Remarks

Each list entry must have at least one type name, selection set, or selection condition defined.

For more information about how this element is used in a list view, see List View.

Example

The following example shows how to specify a selection set for an entry of a list view.

XML

```
<ListEntry>
  <EntrySelectedBy>
    <TypeName>Nameof.NetType</TypeName>
  </EntrySelectedBy>
  <ListItems>...</ListItems>
</ListEntry>
```

See Also

Creating a List View

EntrySelectedBy Element for ListEntry

SelectionSetName Element for EntrySelectedBy for ListEntry

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ListItems Element

Defines the properties and scripts whose values are displayed in the rows of the list view.

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element
- ListItems Element

Syntax

XML

```
<ListItems>
  <ListItem>...</ListItem>
</ListItems>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `ListItems` element. There is no limit to the number of child elements that can be specified. The order of the child elements defines the order that values are displayed in the list view.

Attributes

None.

Child Elements

Expand table

Element	Description
ListItem Element for ListControl	Required element. Defines the property or script whose value is displayed by the list view.

Parent Elements

Expand table

Element	Description
ListEntry Element for ListControl	Provides a definition of the list view.

Remarks

For more information about this type of view, see Creating a List View.

Example

This example shows the XML elements that define three rows of the list view.

XML

```
<ListEntry>
    <ListItems>
      <ListItem>
        <Label>Property1: </Label>
        <PropertyName>.NetTypeProperty1</PropertyName>
      </ListItem>
      <ListItem>
        <PropertyName>.NetTypeProperty2</PropertyName>
      </ListItem>
      <ListItem>
        <ScriptBlock>$_.ProcessName + ":" $_.Id</ScriptBlock>
      </ListItem>
  </ListEntry>
```

See Also

ListEntry Element for ListControl

ListItem Element for ListControl

Creating a List View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

FormatString Element for ListItem for ListControl

Specifies a format pattern that defines how the property or script value is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element
- ListItems Element
- ListItem Element
- FormatString Element

Syntax

XML

```
<FormatString>PropertyPattern</FormatString>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `FormatString` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ListItem Element	Defines the property or script whose value is displayed in a row of the list view.

Text Value

Specify the pattern that is used to format the data. For example, you can use this pattern to format the value of any property that is of type System.TimeSpan: {0:MMM}{0:dd}{0:HH}:{0:mm}.

Remarks

Format strings can be used when creating table views, list views, wide views, or custom views. For more information about formatting a value displayed in a view, see Formatting Displayed Data.

For more information about using format strings in list views, see Creating List View.

Example

The following example shows how to define a formatting string for the value of the `StartTime` property.

XML

```
<ListItem>
  <PropertyName>StartTime</PropertyName>
  <FormatString>{0:MMM} {0:DD} {0:HH}:{0:MM}</FormatString>
</ListItem>
```

See Also

Creating a List View

ListItem Element

Writing a Windows PowerShell Formatting and Types File

Last updated on 05/20/2025

ItemSelectionCondition Element for ListItem for ListControl

Defines the condition that must exist for this list item to be used.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element
- ListItems Element
- ListItem Element
- ItemSelectionCondition Element

Syntax

XML

```
<ItemSelectionCondition>
  <PropertyName>.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate</ScriptBlock>
</ItemSelectionCondition>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ItemSelectionCondition` element.

Attributes

None.

Child Elements

Expand table

Element	Description
PropertyName Element for ItemSelectionCondition for ListControl	Optional element. Specifies the .NET property that triggers the condition.
ScriptBlock Element for ItemSelectionCondition for ListControl	Optional element. Specifies the script that triggers the condition.

Parent Elements

Expand table

Element	Description
ListItem Element for ListItems for ListControl	Defines the property or script whose value is displayed in a row of the list view.

Remarks

You can specify one property name or a script for this condition but cannot specify both.

See Also

ListItem Element for ListItems for ListControl

PropertyName Element for ItemSelectionCondition for ListControl

ScriptBlock Element for ItemSelectionCondition for ListControl

Writing a PowerShell Formatting File

Last updated on 05/20/2025

PropertyName Element for ItemSelectionCondition for ListControl

Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the view is used. This element is used when defining a list view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element
- ListItems Element
- ListItem Element
- ItemSelectionCondition Element
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent elements of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ItemSelectionCondition Element for ListItem for ListControl	

Text Value

Specify the name of the property whose value is displayed.

Remarks

If this element is used, you cannot specify the ScriptBlock element when defining the selection condition.

See Also

ScriptBlock Element for ItemSelectionCondition for ListIControl

ItemSelectionCondition Element for ListItem for ListControl

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for ItemSelectionCondition for ListControl

Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the list item is used. This element is used when defining a list view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element for ListControl
- ListEntry Element for ListEntries for ListControl
- ListItems Element for ListEntry for ListControl
- ListItem Element for ListItems for List Control
- ItemSelectionCondition Element for ListItem for ListControl
- ScriptBlock Element for ItemSelectionCondition for ListControl

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent elements of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ItemSelectionCondition Element for ListItem for ListControl	Defines the condition that must exist for this list item to be used.

Text Value

Specify the script that is evaluated.

Remarks

If this element is used, you cannot specify the `PropertyName` element when defining the selection condition.

See Also

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Label Element for ListItem for ListControl

Specifies the label that is displayed to the left of the property or script value in the row.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element
- ListItems Element
- ListItem Element
- Label Element

Syntax

XML

```
<Label>Label for displayed value</Label>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `Label` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ListItem Element for ListItems for ListControl	Defines the property or script whose value is displayed in a row of the list view.

Text Value

Specify the label to be display to the left of the property or script value.

Remarks

If a label is not specified, the name of the property or the script is displayed. For more information about using labels in a list view, see Creating a List View.

Example

The following example shows how to add a label to a row.

XML

```
<ListItem>
  <Label>Property1: </Label>
  <PropertyName>DotNetProperty1</PropertyName>
</ListItem>
```

See Also

Creating a List View

ListItem Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ListItem Element

Defines the property or script whose value is displayed in a row of the list view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element
- ListItems Element
- ListItem Element

Syntax

XML

```xml
<ListItem>
  <PropertyName>PropertyToDisplay</PropertyName>
  <ScriptBlock>ScriptToExecute</ScriptBlock>
  <Label>LabelToDisplay</Label>
  <FormatString>FormatPattern</FormatString>
  <ItemSelectionCondition>...</ItemSelectionCondition>
</ListItem>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `ListItem` element. Only one property or script can be specified.

Attributes

None

Child Elements

Expand table

Element	Description
FormatString Element for ListItem for ListControl	Optional element. Specifies a format string that defines how the property or script value is displayed.
ItemSelectionCondition Element for ListItem for ListControl	Optional element. Defines the condition that must exist for this list item to be used.
Label Element for ListItem for ListControl	Optional element Specifies the label that is displayed to the left of the property or script value in the row.
PropertyName Element for ListItem for ListControl	Optional element. Specifies the .NET property whose value is displayed in the row.
ScriptBlock Element for ListItem for ListControl	Optional element. Specifies the script whose value is displayed in the row.

Parent Elements

Expand table

Element	Description
ListItems Element for List Control	Defines the properties and scripts whose values are displayed in the list view.

Remarks

For more information about the components of a list view, see Creating a List View.

Example

This example shows the XML elements that define three rows of the list view. The first two rows display the value of a .NET property, and the last row displays a value generated by a script.

XML

```
<ListEntry>
    <ListItems>
      <ListItem>
        <Label>Property1: </Label>
        <PropertyName>DotNetProperty1</PropertyName>
      </ListItem>
      <ListItem>
        <PropertyName>DotNetProperty2</PropertyName>
      </ListItem>
      <ListItem>
        <ScriptBlock>$_.ProcessName + ":" $_.Id</ScriptBlock>
      </ListItem>
    </ListItems>
</ListEntry>
```

See Also

ListItems Element

FormatString Element for ListItem

Label Element for ListItem

PropertyName Element for ListItem

ScriptBlock Element for ListItem

Creating a List View

Writing a Windows PowerShell Formatting and Types File

Last updated on 05/20/2025

PropertyName Element for ListItem for ListControl

Specifies the .NET property whose value is displayed in the list.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element
- ListItems Element
- ListItem Element
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ListItem Element	Defines the property or script whose value is displayed in the row of the list view.

Text Value

Specify the name of the property whose value is displayed.

Remarks

When this element is specified, you cannot specify the ScriptBlock element.

In addition to displaying the property value, you can also specify a label for the value or a format string that can be used to change the display of the value. For more information about specifying data in a list view, see Creating a List View.

Example

The following example shows how to specify the label and property whose value is displayed.

XML

```
ListItem>
  <Label>NameOfProperty</Label>
  <PropertyName>.NetTypeProperty</PropertyName>
</ListItem>
```

See Also

ScriptBlock Element for ListItem for ListControl

Creating a List View

ListItem Element for ListControl(Format)

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for ListItem for ListControl

Specifies the script whose value is displayed in the row.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ListControl Element
- ListEntries Element
- ListEntry Element
- ListItems Element
- ListItem Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ListItem Element	Defines the property or script whose value is displayed in a row of the list view.

Text Value

Specify the script whose value is displayed in the row.

Remarks

When this element is specified, you cannot specify the PropertyName element.

For more information about specifying scripts in a list view, see List View.

Example

The following example shows how to specify the property whose value is displayed.

XML

```
<ListItem>
  <ScriptBlock>$_.ProcessName + ":" $_.Id</ScriptBlock>
</ListItem>
```

See Also

PropertyName Element for ListItem for ListControl

Creating a List View

ListItem Element for ListItems for ListControl

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Name Element for View

Specifies the name that is used to identify the view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- Name Element

Syntax

XML

```
<Name>ViewName</Name>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `Name` element. Only one `Name` element is allowed for each view.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
View Element	Defines a view that is used to display the members of one or more .NET objects.

Text Value

Specify a unique friendly name for the view. This name can include a reference to the type of the view (such as a table view or list view), which object or set of objects use the view, what command returns the objects, or a combination of these.

Remarks

For more information about the different types of views, see the following topics: Table View, List View, Wide View, and Custom View.

Example

The following example shows a `View` element that defines a table view for the System.ServiceProcess.ServiceController object. The name of the view is "service".

XML

```
<View>
  <Name>service</Name>
  <ViewSelectedBy>
    <TypeName>System.ServiceProcess.ServiceController</TypeName>
  </ViewSelectedBy>
  <TableControl>...</TableControl>
</View>
```

See Also

Creating a List View

Creating a Table View

Creating a Wide View

Creating Custom Controls

View Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

OutOfBand Element

In a pipeline, the first object emitted is chosen as the type to format the output of the pipeline. PowerShell attempts for format subsequent objects using the same view. If the object does not fit the view, it is not displayed. You can create OutOfBand views that can be used for format these other types.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- OutOfBand Element

Syntax

XML

```
<OutOfBand/>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `OutOfBand` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
View Element	Defines a view that displays one or more .NET objects.

Remarks

When the "shape" of formatting (view) has been determined by previous objects, you may want objects of different types to continue using that shape (table, list, or whatever) even if they specify their own views. Or sometimes you want your view to take over. When OutOfBand is true, the view applies regardless of previous objects that may have selected a different view.

See Also

View Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TableControl Element

Defines a table format for a view.

Schema

- ViewDefinitions Element
- View Element
- TableControl Element

Syntax

XML

```
<TableControl>
  <AutoSize/>
  <HideTableHeaders/>
  <TableHeaders>...</TableHeaders>
  <TableRowEntries>...</TableRowEntries>
</TableControl>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `TableControl` element. You must specify the rows of the table. All other child elements are optional.

Attributes

None.

Child Elements

Expand table

Element	Description
AutoSize Element for TableControl	Optional element.

Element	Description
	Specifies whether the column size and the number of columns are adjusted based on the size of the data.
HideTableHeaders Element for TableControl	Optional element. Indicates whether the header of the table is not displayed.
TableHeaders Element for TableControl	Required element. Defines the labels, the widths, and the alignment of the data for the columns of the table view.
TableRowEntries Element for TableControl	Optional element. Provides the definitions of the table view.

Parent Elements

Expand table

Element	Description
View Element	Defines a view that is used to display the members of one or more objects.

Remarks

For more information about the components of a table view, see Creating a Table View.

Example

This example shows a `TableControl` element that is used to display the properties of the System.ServiceProcess.ServiceController object.

XML

```
<View>
  <Name>service</Name>
  <ViewSelectedBy>
    <TypeName>System.ServiceProcess.ServiceController</TypeName>
  </ViewSelectedBy>
  <TableControl>
    <TableHeaders>...</TableHeaders>
    <TableRowEntries>...</TableRowEntries>
  </TableControl>
```

```
</View>
```

See Also

Creating a Table View

View Element

AutoSize Element for TableControl

HideTableHeaders Element

TableHeaders Element

TableRowEntries Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

AutoSize Element for TableControl

Specifies whether the column size and the number of columns are adjusted based on the size of the data.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- AutoSize Element

Syntax

XML

```
<AutoSize/>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `AutoSize` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
TableControl Element	Defines a table format for a view.

Remarks

For more information about the components of a table view, see Creating a Table View.

See Also

Creating a Table View

TableControl Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

HideTableHeaders Element

Specifies that the headers of the table are not displayed.

Schema

- ViewDefinitions Element
- View Element
- TableControl Element
- HideTableHeaders Element

Syntax

VB

```
<HideTableHeaders/>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `HideTableHeaders` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
TableControl Element	Defines a table format for a view.

Text Value

Specify `true` to hide the headers of the table.

Remarks

For more information about the components of a table view, see Creating a Table View.

See Also

Creating a Table View

TableControl Element

Last updated on 05/20/2025

TableHeaders Element

Defines the headers for the columns of a table.

Schema

- ViewDefinitions Element
- View Element
- TableControl Element
- TableHeaders Element for TableControl

Syntax

XML

```
<TableHeaders>
  <TableColumnHeader>...</TableColumnHeader>
</TableHeaders>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent elements of the `TableHeaders` element. There must be a child element for each property of the object that is to be displayed. The column header information is displayed in the order that the child elements are specified.

Attributes

None.

Child Elements

Expand table

Element	Description
TableColumnHeader Element	Optional element.

Element	Description
	Defines the label, the width, and the alignment of the data for a column of a table view.

Parent Elements

Expand table

Element	Description
TableControl Element	Defines a table format for a view.

Remarks

For more information about the components of a table view, see Creating a Table View.

Example

This example shows a `TableHeaders` element that defines two column headers.

XML

```
<TableHeaders>
  <TableColumnHeader>
    <Label>Column 1</Label>
    <Width>16</Width>
    <Alignment>Left</Alignment>
  </TableColumnHeader>
  <TableColumnHeader>
    <Label>Column 2</Label>
    <Width>10</Width>
    <Alignment>Centered</Alignment>
  </TableColumnHeader>
</TableHeaders>
```

See Also

Creating a Table View

TableColumnHeader Element

TableControl Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TableColumnHeader Element

Defines the label, the width of the column, and the alignment of the label for a column of the table.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableHeaders Element
- TableColumnHeader Element

Syntax

XML

```
<TableColumnHeader>
  <Label>DisplayedLabel</Label>
  <Width>NumberOfCharacters</Width>
  <Alignment>Left, Right, or Centered</Alignment>
</TableColumnHeader>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `TableColumnHeader` element.

Attributes

None.

Child Elements

Expand table

Element	Description
Label Element For TableColumnHeader for TableControl	Optional element. Defines the label that is displayed at the top of the column. If no label is specified, the name of the property whose value is displayed in the rows is used.
Width Element for TableColumnHeader for TableControl	Required element. Specifies the width (in characters) of the column.
Alignment Element for TableColumnHeader for TableControl	Optional element. Specifies how the label of the column is displayed. If no alignment is specified, the label is aligned on the left.

Parent Elements

Expand table

Element	Description
TableHeaders Element	Defines the columns of a table view.

Remarks

Specify a header for each column of the table. The columns are displayed in the order in which the `TableColumnHeader` elements are defined.

A table must have the same number of `TableColumnHeader` elements as `TableRowEntry` elements. The column header defines how the text at the top of the table is displayed. The row entries define what data is displayed in the rows of the table.

For more information about the components of a table view, see Table View.

Example

The following example shows two `TableColumnHeader` elements. The first element defines a column whose label is "Column 1", has a width of 16 characters, and whose label is aligned on the left. The second element defines a column whose label is "Column 2", has a width of 10 characters, and whose label is centered in the column.

XML

```
<TableHeaders>
  <TableColumnHeader>
    <Label>Column 1</Label>
    <Width>16</Width>
    <Alignment>Left</Alignment>
  </TableColumnHeader>
    <TableColumnHeader>
    <Label>Column 2</Label>
    <Width>10</Width>
    <Alignment>Centered</Alignment>
  </TableColumnHeader>
</TableHeaders>
```

See Also

Alignment Element for TableColumnHeader for TableControl

Creating a Table View

Label Element for TableColumnHeader for TableControl

TableHeaders Element for TableControl

Width for TableColumnHeader for TableControl Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Alignment Element for TableColumnHeader

Defines how the data in a column header is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableHeaders Element
- TableColumnHeader Element
- Alignment Element

Syntax

XML

```
<Alignment>AlignmentType</Alignment>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `Alignment` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
TableColumnHeader Element	Defines a label, the width, and the alignment of the data for a column of the table.

Text Value

Specify one of the following values. These values are not case-sensitive.

- Left - Aligns the data displayed in the column on the left This is the default if this element is not specified.
- Right - Aligns the data displayed in the column on the right.
- Center - Centers the data displayed in the column.

Remarks

For more information about the components of a table view, see Creating a Table View.

Example

This example shows a `TableColumnHeader` element whose data is aligned on the center.

XML

```
<TableColumnHeader>
  <Label>Column 1</Label>
  <Width>16</Width>
  <Alignment>Center</Alignment>
</TableColumnHeader>
```

See Also

Creating a Table View

TableColumnHeader Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Label Element for TableColumnHeader for TableControl

Defines the label that is displayed at the top of a column. This element is used when defining a table view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableHeaders Element
- TableColumnHeader Element
- Label Element

Syntax

XML

```
<Label>DisplayedLabel</Label>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `Label` element. Only one label is allowed for each column.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
TableColumnHeader Element for TableHeaders for TableControl	Defines a label, the width, and the alignment of the data for a column of the table.

Text Value

Specify the text that is displayed at the top of the column of the table. There are no restricted characters for the column label.

Remarks

If no label is specified, the name of the property whose value is displayed in the rows is used.

For more information about the components of a table view, see Creating a Table View.

Example

This example shows a `TableColumnHeader` element whose label is "Column 1".

XML

```
<TableColumnHeader>
  <Label>Column 1</Label>
  <Width>16</Width>
  <Alignment>Left</Alignment>
</TableColumnHeader>
```

See Also

Creating a Table View

TableColumnHeader Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Width Element for TableColumnHeader

Defines the width (in characters) of a column.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableHeaders Element
- TableColumnHeader
- Width Element

Syntax

XML

```
<Width>NumberOfCharacters</Width>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `Width` element used when defining column headers.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
TableColumnHeader Element for TableHeaders for TableControl	Defines a label, width, and alignment of the data for a column of the table.

Text Value

When at all possible, specify a width (in characters) that is greater than the length of the displayed property values.

Remarks

For more information about the components of a table view, see Creating a Table View.

Example

The following example shows a `TableColumnHeader` element whose width is 16 characters.

XML

```
<TableColumnHeader>
  <Label>Column 1</Label>
  <Width>16</Width>
  <Alignment>Left</Alignment>
</TableColumnHeader>
```

See Also

Creating a Table View

TableColumnHeader Element for TableHeader for TableControl

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TableRowEntries Element

Defines the rows of the table.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element

Syntax

XML

```
<TableRowEntries>
  <TableRowEntry>...</TableRowEntry>
</TableRowEntries>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `TableRowEntries` element.

Attributes

None.

Child Elements

Expand table

Element	Description
TableRowEntry Element for TableRowEntries for TableControl	Required element. Defines the data that is displayed in a row of the table.

Parent Elements

Expand table

Element	Description
TableControl Element	Defines a table format for a view.

Remarks

You must specify one or more `TableRowEntry` elements for the table view. There is no maximum limit to the number of `TableRowEntry` elements that can be added nor is their order significant.

For more information about the components of a table view, see Creating a Table View.

Example

The following example shows a `TableRowEntries` element that defines a row that displays the values of two properties of the System.Diagnostics.Process object.

XML

```
<TableRowEntries>
  <TableRowEntry>
    <EntrySelectedBy>
      <TypeName>System.Diagnostics.Process</TypeName>
    </EntrySelectedBy>
    <TableColumnItems>
      <TableColumnItem>
        <PropertyName> Property for first column</PropertyName>
      </TableColumnItem>
      <TableColumnItem>
        <PropertyName> Property for second column</PropertyName>
      </TableColumnItem>
    </TableColumnItems>
  </TableRowEntry>
</TableRowEntries>
```

See Also

Creating a Table View

TableControl Element

TableRowEntry Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TableColumnItems Element

Defines the properties or scripts whose values are displayed in a row.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element for TableControl
- TableRowEntry Element for TableRowEntries for TableControl
- TableColumnItems Element for TableControlEntry for TableControl

Syntax

XML

```
<TableColumnItems>
  <TableColumnItem>...</TableColumnItem>
</TableColumnItems>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `TableColumnItems` element.

Attributes

None.

Child Elements

Expand table

Element	Description
TableColumnItem Element for TableColumnItems for TableControl	Required element.

Element	Description
	Defines the property or script whose value is displayed in a column of the row.

Parent Elements

Expand table

Element	Description
TableRowEntry Element for TableRowEntries for TableControl	Defines the data that is displayed in a row of the table.

Remarks

A `TableColumnItem` element is required for each column of the row. The first entry is displayed in first column, the second entry in the second column, and so on.

For more information about the components of a table view, see Creating a Table View.

Example

The following example shows a `TableColumnItems` element that defines three properties of the System.Diagnostics.Process object.

XML

```
<TableColumnItems>
  <TableColumnItem>
    <PropertyName>Status</PropertyName>
  </TableColumnItem>
  <TableColumnItem>
    <PropertyName>Name</PropertyName>
  </TableColumnItem>
  <TableColumnItem>
    <PropertyName>DisplayName</PropertyName>
  </TableColumnItem>
</TableColumnItems>
```

See Also

Creating a Table View

TableColumnItem Element

TableRowEntry Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TableColumnItem Element

Defines the property or script whose value is displayed in the column of the row.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element
- TableRowEntry Element
- TableColumnItems Element
- TableColumnItem Element

Syntax

XML

```xml
<TableColumnItem>
  <Alignment>Left, Right, or Center</Alignment>
  <FormatString>FormatPattern</FormatString>
  <PropertyName>Nameof.NetProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate</ScriptBlock>
</TableColumnItem>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `TableColumnItem` element.

Attributes

None.

Child Elements

Expand table

Element	Description
Alignment Element for TableColumnItem for TableControl	Optional element. Defines how the data in a column of the row is displayed.
FormatString Element for TableColumnItem for TableControl	Specifies a format pattern that is used to format the data in the column of the row.
PropertyName Element for TableColumnItem for TableControl	Optional element. Specifies the name of the property whose value is displayed.
ScriptBlock Element for TableColumnItem for TableControl	Optional element. Specifies the script whose value is displayed in the column of a row.

Parent Elements

Expand table

Element	Description
TableColumnItems Element for TableControlEntry for TableControl	Defines the properties or scripts whose values are displayed in the row.

Remarks

You can specify a property of an object or a script in each column of the row. If no child elements are specified, the item is a placeholder, and no data is displayed.

For more information about the components of a table view, see Creating a Table View.

Example

This example shows a `TableColumnItem` element that displays the value of the `Status` property of the System.Diagnostics.Process object.

XML

```
<TableColumnItem>
   <Alignment>Centered</Alignment>
  <PropertyName>Status</PropertyName>
```

```
</TableColumnItem>
```

See Also

Creating a Table View

Alignment Element for TableColumnItem for TableControl

TableColumnItems Element

FormatString Element for TableColumnItem for TableControl

PropertyName Element for TableColumnItem for TableControl

ScriptBlock Element for TableColumnItem for TableControl

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Alignment Element for TableColumnItem

Defines how the data in a column of the row is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element
- TableRowEntry Element
- TableColumnItems Element
- TableColumnItem Element
- Alignment Element

Syntax

XML

```
<Alignment>AlignmentType</Alignment>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `Alignment` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
TableColumnItem Element	Defines a label, the width, and the alignment of the data for a column of the table.

Text Value

Specify one of the following values. (These values are not case-sensitive.)

- Left - Shifts the data displayed in the column to the left. (This is the default if this element is not specified.)
- Right - Shifts the data displayed in the column to the right.
- Center - Centers the data displayed in the column.

Remarks

For more information about the components of a table view, see Table View.

See Also

Table View

TableColumnItem Element

Last updated on 05/20/2025

FormatString Element for TableColumnItem for TableControl

Specifies a format pattern that defines how the property or script value of the table is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element
- TableRowEntry Element
- TableColumnItems Element
- TableColumnItem Element
- FormatString Element

Syntax

XML

```
<FormatString>FormatPattern</FormatString>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `FormatString` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
TableColumnItem Element	Defines the property or script whose value is displayed in the column of the row.

Text Value

Specify the pattern that is used to format the data. For example, this pattern can be used to format the value of any property that is of type System.TimeSpan: {0:MMM}{0:dd}{0:HH}:{0:mm}.

Remarks

Format strings can be used when creating table views, list views, wide views, or custom views. For more information about formatting a value displayed in a view, see Formatting Displayed Data.

For more information about the components of a table view, see Table View.

Example

The following example shows how to define a formatting string for the value of the `StartTime` property.

XML

```
<TableColumnItem>
  <PropertyName>StartTime</PropertyName>
  <FormatString>{0:MMM} {0:DD} {0:HH}:{0:MM}</FormatString>
</TableColumnItem>
```

See Also

Creating a Table View

Formatting Displayed Data

TableColumnItem Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

PropertyName Element for TableColumnItem for TableControl

Specifies the property whose value is displayed in the column of the row.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element
- TableRowEntry Element
- TableColumnItems Element
- TableColumnItem Element
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
TableColumnItem Element	Defines the property or script whose value is displayed in the column of the row.

Text Value

Specify the name of the property whose value is displayed.

Remarks

For more information about the components of a table view, see Creating a Table View.

Example

This example shows a `TableColumnItem` element that specifies the `Status` property of the System.Diagnostics.Process object.

XML

```
<TableColumnItem>
   <Alignment>Centered</Alignment>
  <PropertyName>Status</PropertyName>
</TableColumnItem>
```

See Also

Creating a Table View

TableColumnItem Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for TableColumnItem for TableControl

Specifies the script whose value is displayed in the column of the row.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element
- TableRowEntry Element
- TableColumnItems Element
- TableColumnItem Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
TableColumnItem Element	Defines the property or script whose value is displayed in the column of the row.

Text Value

Specify the script whose value is displayed.

Remarks

For more information about the components of a table view, see Creating a Table View.

See Also

Creating a Table View

TableColumnItem Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TableRowEntry Element

Defines the data that is displayed in a row of the table.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element for TableControl
- TableRowEntry Element for TableRowEntries

Syntax

XML

```
<TableRowEntry>
  <Wrap/>
  <EntrySelectedBy>...</EntrySelectedBy>
  <TableColumnItems>...</TableColumnItems>
</TableRowEntry>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent element of the `TableRowEntry` element.

Attributes

None.

Child Elements

Expand table

Element	Description
EntrySelectedBy Element for TableRowEntry for TableControl	Required element. Defines the objects whose property values are displayed in the row.
TableColumnItems Element for TableRowEntry for TableControl	Required element. Defines the properties or scripts whose values are displayed.
Wrap Element for TableRowEntry for TableControl	Optional element. Specifies that text that exceeds the column width is displayed on the next line.

Parent Elements

Expand table

Element	Description
TableRowEntries Element for TableControl	Defines the rows of the table.

Remarks

One `TableColumnItems` element and one `EntrySelectedBy` element must be specified.

For more information about the components of a table view, see Creating a Table View.

Example

The following example shows a `TableRowEntry` element that defines a row that displays the values of two properties of the System.Diagnostics.Process object.

XML

```
<TableRowEntry>
  <EntrySelectedBy>
    <TypeName>System.Diagnostics.Process</TypeName>
  </EntrySelectedBy>
  <TableColumnItems>
    <TableColumnItem>
      <PropertyName> Property for first column</PropertyName>
    </TableColumnItem>
```

```
    <TableColumnItem>
      <PropertyName> Property for second column</PropertyName>
    </TableColumnItem>
  </TableColumnItems>
</TableRowEntry>
```

See Also

Creating a Table View

EntrySelectedBy Element for TableRowEntry for TableControl

TableColumnItems Element for TableRowEntry for TableControl

TableRowEntries Element for TableControl

Wrap Element for TableRowEntry for TableControl

Writing a PowerShell Formatting File

Last updated on 05/20/2025

EntrySelectedBy Element for TableRowEntry

Defines the .NET types that use this definition of the table view or the condition that must exist for this definition to be used.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element
- TableRowEntry Element
- EntrySelectedBy Element

Syntax

XML

```
<EntrySelectedBy>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>NameofSelectionSet</SelectionSetName>
  <SelectionCondition>...</SelectionCondition>
</EntrySelectedBy>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `EntrySelectedBy` element.

Attributes

None.

Child Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for TableControl	Optional element. Defines the condition that must exist for this table view definition to be used.
SelectionSetName Element for EntrySelectedBy for TableControl	Optional element. Specifies a set of .NET types that use this table view definition.
TypeName Element for EntrySelectedBy for TableControl	Optional element. Specifies a .NET type that uses this table view definition.

Parent Elements

Expand table

Element	Description
TableRowEntry Element for TableControl	Defines the data that is displayed in a row of the table.

Remarks

You must specify at least one type, selection set, or selection condition for a table view definition. There is no maximum limit to the number of child elements that you can use.

Selection conditions are used to define a condition that must exist for the definition to be used, such as when an object has a specific property or that a specific property value or script evaluates to `true`. For more information about selection conditions, see Defining Conditions for when a View Entry or Item is Used.

For more information about the components of a table view, see Creating a Table View.

Example

The following example shows a `TableRowEntry` element that is used to display the properties of the System.Diagnostics.Process object.

XML

```xml
<TableRowEntry>
  <EntrySelectedBy>
    <TypeName>System.Diagnostics.Process</TypeName>
  </EntrySelectedBy>
  <TableColumnItems>
    <TableColumnItem>
      <PropertyName>PropertyForFirstColumn</PropertyName>
    </TableColumnItem>
    <TableColumnItem>
      <PropertyName>PropertyForSecondColumn</PropertyName>
    </TableColumnItem>
  </TableColumnItems>
</TableRowEntry>
```

See Also

Creating a Table View

SelectionCondition Element for EntrySelectedBy for TableControl

SelectionSetName Element for EntrySelectedBy for TableControl

TableRowEntry Element for TableControl

TypeName Element for EntrySelectedBy for TableControl

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionCondition Element for EntrySelectedBy for TableControl

Defines the condition that must exist to use for this definition of the table view. There is no limit to the number of selection conditions that can be specified for a table definition.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element
- TableRowEntry Element
- EntrySelectedBy Element
- SelectionCondition Element

Syntax

XML

```
<SelectionCondition>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>NameofSelectionSet</SelectionSetName>
  <PropertyName>.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate</ScriptBlock>
</SelectionCondition>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the SelectionCondition element.

Attributes

None.

Child Elements

Expand table

Element	Description
PropertyName Element for SelectionCondition for EntrySelectedBy for TableRowEntry	Optional element. Specifies the .NET property that triggers the condition.
ScriptBlock Element for SelectionCondition for EntrySelectedBy for TableRowEntry	Optional element. Specifies the script that triggers the condition.
SelectionSetName Element for SelectionCondition for EntrySelectedBy for TableRowEntry	Optional element. Specifies the set of .NET types that trigger the condition.
TypeName Element for SelectionCondition for EntrySelectedBy for TableRowEntry	Optional element. Specifies a .NET type that triggers the condition.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for TableRowEntry	Defines the .NET types that use this table entry or the condition that must exist for this entry to be used.

Remarks

Each list entry must have at least one type name, selection set, or selection condition defined.

When you are defining a selection condition, the following requirements apply:

- The selection condition must specify a least one property name or a script block, but cannot specify both.
- The selection condition can specify any number of .NET types or selection sets, but cannot specify both.

For more information about how to use selection conditions, see Defining Conditions for when a View Entry or Item is Used.

For more information about the components of a table view, see Creating a Table View.

See Also

Creating a Table View

Defining Conditions for When Data Is Displayed

EntrySelectedBy Element

PropertyName Element for SelectionCondition for EntrySelectedBy for TableRowEntry

ScriptBlock Element for SelectionCondition for EntrySelectedBy for TableRowEntry

SelectionSetName Element for SelectionCondition for EntrySelectedBy for TableRowEntry

TypeName Element for SelectionCondition for EntrySelectedBy for TableRowEntry

Writing a Windows PowerShell Formatting and Types File

Last updated on 05/20/2025

PropertyName Element for SelectionCondition for EntrySelectedBy for TableRowEntry

Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the table entry is used.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element
- TableRowEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for TableRowEntry	Defines the condition that must exist for this table entry to be used.

Text Value

Specify the .NET property name.

Remarks

The selection condition must specify at least one property name or a script block, but cannot specify both. For more information about how selection conditions can be used, see Defining Conditions for when a View Entry or Item is Used.

For more information about the components of a table view, see Creating a Table View.

See Also

Creating a Table View

Defining Conditions for When Data Is Displayed

ScriptBlock Element for SelectionCondition for EntrySelectedBy for TableRowEntry

SelectionCondition Element for EntrySelectedBy for TableRowEntry

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for SelectionCondition for EntrySelectedBy for TableControl

Specifies the script block that triggers the condition. When this script is evaluated to `true`, the condition is met, and the table entry is used.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element
- TableRowEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for TableRowEntry	Defines the condition that must exist for this table entry to be used.

Text Value

Specify the script that is evaluated.

Remarks

The selection condition must specify at least one script block or property name, but cannot specify both. For more information about how to use selection conditions, see Defining Conditions for when a View Entry or Item is Used.

For more information about the components of a table view, see Creating a Table View.

See Also

Creating a Table View

Defining Conditions for When Data Is Displayed

PropertyName Element for SelectionCondition for EntrySelectedBy for TableRowEntry

SelectionCondition Element for EntrySelectedBy for TableRowEntry

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSetName Element for SelectionCondition for EntrySelectedBy for TableControl

Specifies the set of .NET types that trigger the condition. When any of the types in this set are present, the condition is met, and the object is displayed by using this definition of the table view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element
- TableRowEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- SelectionSetName Element

Syntax

XML

```
<SelectionSetName>NameofSelectionSet</SelectionSetName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionSetName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for TableRowEntry	Defines the condition that must exist to use for this definition of the table view.

Text Value

Specify the name of the selection set.

Remarks

The selection condition can specify a selection set or .NET type, but cannot specify both. For more information about how to use selection conditions, see Defining Conditions for when Data is Displayed.

Selection sets are common groups of .NET objects that can be used by any view that the formatting file defines. For more information about creating and referencing selection sets, see Defining Sets of Objects.

For more information about other components of a wide view, see Creating a Table View.

See Also

Creating a Table View

Defining Conditions for When Data Is Displayed

TypeName Element for SelectionCondition for EntrySelectedBy for TableRowEntry

SelectionCondition Element for EntrySelectedBy for TableRowEntry

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for SelectionCondition for EntrySelectedBy for TableControl

Specifies a .NET type that triggers the condition. When this type is present, the condition is met, and the table row is used.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element
- TableRowEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- TypeName Element

Syntax

XML

```
<TypeName>Nameof.NetType</TypeName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `TypeName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for TableRowEntry	Defines the condition that must exist for this table row to be used.

Text Value

Specify the fully qualified name of the .NET type, such as `System.IO.DirectoryInfo`.

Remarks

The selection condition can specify any number of .NET types or selection sets, but cannot specify both. For more information about how to use selection conditions, see Defining Conditions for when a View Entry or Item is Used.

For more information about the components of a table view, see Creating a Table View.

See Also

Creating a Table View

Defining Conditions for When Data Is Displayed

SelectionCondition Element for EntrySelectedBy for TableRowEntry

SelectionSetName Element for SelectionCondition for EntrySelectedBy for TableRowEntry

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSetName Element for EntrySelectedBy for TableControl

Specifies a set of .NET types the use this entry of the table view. There is no limit to the number of selection sets that can be specified for an entry.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element
- TableRowEntry Element
- EntrySelectedBy Element
- SelectionSetName Element

Syntax

XML

```
<SelectionSetName>NameofSelectionSet</SelectionSetName>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element	Defines the .NET types that use this entry or the condition that must exist for this entry to be used.

Text Value

Specify the name of the selection set.

Remarks

Selection sets are typically used when you want to define a group of objects that are used in multiple views. For example, you might want to create a table view and a list view for the same set of objects. For more information about defining selection sets, see Defining Sets of objects for a View.

If you specify a selection set for an entry, you cannot specify a type name. For more information about how to specify a .NET type, see TypeName Element for EntrySelectedBy for TableRowEntry.

For more information about the components of a table view, see Creating a Table View.

See Also

EntrySelectedBy Element

Defining Sets of objects for a View

Creating a Table View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for EntrySelectedBy for TableControl

Specifies a .NET type that uses this entry of the table view. There is no limit to the number of types that can be specified for a table entry.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element
- TableRowEntry Element
- EntrySelectedBy Element
- TypeName Element

Syntax

XML

```
<TypeName>Nameof.NetType</TypeName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `TypeName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element	Defines the .NET types that use this entry or the condition that must exist for this entry to be used.

Text Value

Specify the name of the .NET type.

Remarks

Each list entry must have at least one type name, selection set, or selection condition defined.

For more information about the components of a table view, see Creating a Table View.

See Also

Creating a Table View

EntrySelectedBy Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Wrap Element for TableRowEntry

Specifies that text that exceeds the column width is displayed on the next line. By default, text that exceeds the column width is truncated.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- TableControl Element
- TableRowEntries Element for TableControl
- TableRowEntry Element for TableRowEntries for TableControl
- Wrap Element for TableRowEntry for TableControl

Syntax

XML

```
<Wrap/>
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements of the `Wrap` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
TableRowEntry Element for TableRowEntries for TableControl	Defines the data that is displayed in a row of the table.

Remarks

For more information about the components of a table view, see Creating a Table View.

See Also

Creating a Table View

TableRowEntry Element for TableRowEntries for TableControl

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ViewSelectedBy Element

Defines the .NET objects that are displayed by the view. Each view must specify at least one .NET object.

Schema

- ViewDefinitions Element
- View Element
- ViewSelectedBy Element

Syntax

XML

```xml
<ViewSelectedBy>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>SelectionSet</SelectionSetName>
</ViewSelectedBy>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `ViewSelectedBy` element. This element must contain at least one `TypeName` or `SelectionSetName` child element. There is no limit to the number of child elements that can be specified nor is their order significant.

Attributes

None.

Child Elements

Expand table

Element	Description
TypeName Element for ViewSelectedBy	Optional element.

Element	Description
	Specifies a .NET object that is displayed by the view.
SelectionSetName Element for ViewSelectedBy	Optional element. Specifies a set of .NET objects that are displayed by the view.

Parent Elements

Expand table

Element	Description
View Element	Defines a view that displays one or more .NET objects.

Remarks

For more information about how this element is used in different views, see Table View Components, List View Components, Wide View Components, and Custom Control Components.

The `SelectionSetName` element is used when the formatting file defines a set of objects that are displayed by multiple views. For more information about how selection sets are defined and referenced, see Defining Sets of Objects.

Example

The following example shows how to specify the System.ServiceProcess.ServiceController object for a list view. The same schema is used for table, wide, and custom views.

XML

```
<View>
  <Name>System.ServiceProcess.ServiceController</Name>
  <ViewSelectedBy>
    <TypeName>System.ServiceProcess.ServiceController</TypeName>
  </ViewSelectedBy>
  <ListControl>...</ListControl>
</View>
```

See Also

Creating a List View

Creating a Table View

Creating a Wide View

Creating Custom Controls

Defining Selection Sets

SelectionSetName Element for ViewSelectedBy

TypeName Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSetName Element for ViewSelectedBy

Specifies a set of .NET objects that are displayed by the view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ViewSelectedBy Element
- SelectionSetName Element

Syntax

XML

```
<SelectionSetName>Name of selection set<SelectionSetName>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `SelectionSetName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ViewSelectedBy Element	Defines the .NET objects that are displayed by the view.

Text Value

Specify the name of the selection set that is defined by the `Name` element for the selection set.

Remarks

You can use selection sets when you have a set of related objects that you want to reference by using a single name, such as a set of objects that are related through inheritance. For more information about defining and referencing selection sets, see Defining Sets of Objects.

Example

The following example shows how to specify a selection set for a list view. The same schema is used for table, wide, and custom views.

XML

```
<View>
  <Name>Name of View</Name>
  <ViewSelectedBy>
    <SelectionSetName>NameofSelectionSet</SelectionSetName>>
  </ViewSelectedBy>
  <ListControl>...</ListControl>
</View>
```

See Also

Defining Selection Sets

ViewSelectedBy Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for ViewSelectedBy

Specifies a .NET object that is displayed by the view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- ViewSelectedBy Element
- TypeName Element

Syntax

XML

```
<TypeName>FullyQualifiedTypeName</TypeName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent elements of the `TypeName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
ViewSelectedBy Element	Defines the .NET objects that are displayed by the view.

Text Value

Specify the fully qualified name of the .NET type, such as `System.IO.DirectoryInfo`.

Remarks

For more information about how this element is used in different views, see Creating a Table View, Creating a List View, Creating a Wide View, and Custom View Components.

Example

The following example shows how to specify the System.ServiceProcess.ServiceController object for a list view. The same schema is used for table, wide, and custom views.

XML

```
<View>
  <Name>System.ServiceProcess.ServiceController</Name>
  <ViewSelectedBy>
    <TypeName>System.ServiceProcess.ServiceController</TypeName>
  </ViewSelectedBy>
  <ListControl>...</ListControl>
</View>
```

See Also

Creating a List View

Creating a Table View

Creating a Wide View

Creating Custom Controls

ViewSelectedBy Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

WideControl Element

Defines a wide (single value) list format for the view. This view displays a single property value or script value for each object.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- WideControl Element

Syntax

XML

```
<WideControl>
  <AutoSize/>
  <ColumnNumber>PositiveInteger</ColumnNumber>
  <WideEntries>...</WideEntries>
</WideControl>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `WideControl` element. You cannot specify the `AutoSize` and `ColumnNumber` elements at the same time.

Attributes

None.

Child Elements

Expand table

Element	Description
AutoSize Element for WideControl	Optional element. Specifies whether the column size and the number of columns are adjusted based on the size of the data.
ColumnNumber Element for WideControl	Optional element. Specifies the number of columns displayed in the wide view.
WideEntries Element	Required element. Provides the definitions of the wide view.

Parent Elements

Expand table

Element	Description
View Element	Defines a view that is used to display one or more .NET objects.

Remarks

When defining a wide view, you can add the `AutoSize` element or the `ColumnNumber` but you cannot add both.

In most cases, only one definition is required for each wide view, but it is possible to have multiple definitions if you want to use the same view to display different .NET objects. In those cases, you can provide a separate definition for each object or set of objects.

For more information about the components of a wide view, see Wide View Components.

Example

The following example shows a `WideControl` element that is used to display a property of the System.Diagnostics.Process object.

XML

```
<View>
  <Name>process</Name>
  <ViewSelectedBy>
    <TypeName>System.Diagnostics.Process</TypeName>
```

```
  </ViewSelectedBy>
  <WideControl>
    <WideEntries>...</WideEntries>
  </WideControl>
</View>
```

For a complete example of a wide view, see Wide View (Basic).

See Also

Autosize Element for WideControl

ColumnNumber Element for WideControl

View Element

WideEntries Element

Wide View (Basic)

Creating a Wide View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

AutoSize Element for WideControl

Specifies whether the column size and the number of columns are adjusted based on the size of the data.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- WideControl Element
- Autosize Element

Syntax

XML

```
<AutoSize/>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `AutoSize` element.

Attributes

None.

Child Elements

None

Parent Elements

Expand table

Element	Description
WideControl Element	Defines a wide (single value) list format for the view.

Remarks

When defining a wide view, you can add the `AutoSize` element or the ColumnNumber element, but you cannot add both.

For more information about the components of a wide view, see Creating a Wide View.

For an example of a wide view, see Wide View (Basic).

See Also

ColumnNumber Element for WideControl

Creating a Wide View

WideControl Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ColumnNumber Element for WideControl

Specifies the number of columns displayed in the wide view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- WideControl Element
- ColumnNumber Element

Syntax

XML

```
<ColumnNumber>PositiveInteger</ColumnNumber>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ColumnNumber` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
WideControl Element	Defines a wide (single value) list format for the view.

Text Value

Specify a positive integer value.

Remarks

When defining a wide view, you can add the `AutoSize` element or the `ColumnNumber` element, but you cannot add both.

For more information about the components of a wide view, see Creating a Wide View.

For an example of a wide view, see Wide View (Basic).

See Also

Autosize Element for WideControl

Creating a Wide View

Wide View (Basic)

Writing a PowerShell Formatting File

Last updated on 05/20/2025

WideEntries Element

Provides the definitions of the wide view. The wide view must specify one or more definitions.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- WideControl Element
- WideEntries Element

Syntax

XML

```
<WideEntries>
  <WideEntry>...</WideEntry>
</WideEntries>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `WideEntries` element. At least one child element must be specified.

Attributes

None.

Child Elements

Expand table

Element	Description
WideEntry Element	Provides a definition of the wide view.

Parent Elements

Expand table

Element	Description
WideControl Element	Defines a wide (single value) list format for the view.

Remarks

A wide view is a list format that displays a single property value or script value for each object. For more information about the components of a wide view, see Wide View Components.

Example

The following example shows a `WideEntries` element that defines a single `WideEntry` element. The `WideEntry` element contains a single `WideItem` element that defines what property or script value is displayed in the view.

XML

```
<WideControl>
  <WideEntries>
    <WideEntry>
      <WideItem>...</WideItem>
    <WideEntry>
  </WideEntries>
</WideControl>
```

For a complete example of a wide view, see Wide View (Basic).

See Also

Creating a Wide View

WideControl Element

WideEntry Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

WideEntry Element

Provides a definition of the wide view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- WideControl Element
- WideEntries Element
- WideEntry Element

Syntax

XML

```
<WideEntry>
  <EntrySelectedBy>...</EntrySelectedBy>
  <WideItem>...</WideItem>
</WideEntry>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `WideEntry` element. You must specify a single `WideItem` child element.

Attributes

None.

Child Elements

Expand table

Element	Description
EntrySelectedBy Element for WideEntry	Optional element.

Element	Description
	Defines the .NET types that use this wide entry definition or the condition that must exist for this definition to be used.
WideItem Element	Required element. Defines the property or script whose value is displayed.

Parent Elements

Expand table

Element	Description
WideEntries Element	Provides the definitions of the wide view.

Remarks

A wide view is a list format that displays a single property value or script value for each object. Unlike other types of views, you can specify only one item element for each view definition. For more information about the other components of a wide view, see Creating a Wide View.

Example

The following example shows a `WideEntry` element that defines a single `WideItem` element. The `WideItem` element defines the property whose value is displayed in the view.

XML

```
<WideEntries>
  <WideEntry>
    <WideItem>
      <PropertyName>ProcessName</PropertyName>
    </WideItem>
  </WideEntry>
</WideEntries>
```

For a complete example of a wide view, see Wide View (Basic).

See Also

Creating a Wide View

SelectionCondition Element for WideEntry

SelectionSetName Element for WideEntry

TypeName Element for WideEntry

WideEntries Element

WideItem Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

EntrySelectedBy Element for WideEntry

Defines the .NET types that use this definition of the wide view or the condition that must exist for this definition to be used.

Schema

Configuration Element ViewDefinitions Element View Element WideControl Element WideEntries Element WideEntry Element EntrySelectedBy Element

Syntax

XML

```
<EntrySelectedBy>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>NameofSelectionSet</SelectionSetName>
  <SelectionCondition>...</SelectionCondition>
</EntrySelectedBy>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `EntrySelectedBy` element.

Attributes

None.

Child Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for WideEntry	Optional element. Defines the condition that must exist for this wide view definition to be used.

Element	Description
SelectionSetName Element for EntrySelectedBy for WideEntry	Optional element. Specifies a set of .NET types that use this wide view definition.
TypeName Element for EntrySelectedBy for WideEntry	Optional element. Specifies a .NET type that uses this wide view definition.

Parent Elements

Expand table

Element	Description
WideEntry Element	Provides a definition of the wide view.

Remarks

You must specify at least one type, selection set, or selection condition for a wide view definition. There is no maximum limit to the number of child elements that you can use.

Selection conditions are used to define a condition that must exist for the definition to be used, such as when an object has a specific property or that a specific property value or script value evaluates to `true`. For more information about selection conditions, see Defining Conditions for Displaying Data.

For more information about other components of a wide view, see Creating a Wide View.

See Also

WideEntry Element

SelectionCondition Element for EntrySelectedBy for WideEntry

SelectionSetName Element for EntrySelectedBy for WideEntry

TypeName Element for EntrySelectedBy for WideEntry

Creating a Wide View

Defining Conditions for Displaying Data

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionCondition Element for EntrySelectedBy for WideControl

Defines the condition that must exist for this definition to be used. There is no limit to the number of selection conditions that can be specified for a wide entry definition.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- WideControl Element
- WideEntries Element
- WideEntry Element
- EntrySelectedBy Element
- SelectionCondition Element

Syntax

XML

```xml
<SelectionCondition>
  <TypeName>Nameof.NetType</TypeName>
  <SelectionSetName>NameofSelectionSet</SelectionSetName>
  <PropertyName>.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToEvaluate</ScriptBlock>
</SelectionCondition>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionCondition` element. You must specify a single `PropertyName` or `ScriptBlock` element. The `SelectionSetName` and `TypeName` elements are optional. You can specify one of either element.

Attributes

None.

Child Elements

Expand table

Element	Description
PropertyName Element for SelectionCondition for EntrySelectedBy for WideEntry	Optional element. Specifies the .NET property that triggers the condition.
ScriptBlock Element for SelectionCondition for EntrySelectedBy for WideEntry	Optional element. Specifies the script block that triggers the condition.
SelectionSetName Element for SelectionCondition for EntrySelectedBy for WideEntry	Optional element. Specifies the set of .NET types that triggers the condition.
TypeName Element for SelectionCondition for EntrySelectedBy for WideEntry	Optional element. Specifies a .NET type that triggers the condition.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for WideEntry	Defines the .NET types that use this wide entry or the condition that must exist for this entry to be used.

Remarks

Each wide entry must have at least one type name, selection set, or selection condition defined.

When you are defining a selection condition, the following requirements apply:

- The selection condition must specify a least one property name or a script block, but cannot specify both.
- The selection condition can specify any number of .NET types or selection sets, but cannot specify both.

For more information about how to use selection conditions, see Defining Conditions for when a View Entry or Item is Used.

For more information about other components of a wide view, see Creating a Wide View.

See Also

Creating a Wide View

Defining Conditions for When Data Is Displayed

EntrySelectedBy Element for WideEntry

PropertyName Element for SelectionCondition for EntrySelectedBy for WideEntry

ScriptBlock Element for SelectionCondition for EntrySelectedBy for WideEntry

SelectionSetName Element for SelectionCondition for EntrySelectedBy for WideEntry

TypeName Element for SelectionCondition for EntrySelectedBy for WideEntry

Writing a PowerShell Formatting File

Last updated on 05/20/2025

PropertyName Element for SelectionCondition for EntrySelectedBy for WideEntry

Specifies the .NET property that triggers the condition. When this property is present or when it evaluates to `true`, the condition is met, and the definition is used.

- Configuration Element
- ViewDefinitions Element
- View Element
- WideControl Element
- WideEntries Element
- WideEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

C#

```

```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for WideEntry	Defines the condition that must exist for this definition to be used.

Text Value

Specify the .NET property name.

Remarks

The selection condition must specify at least one property name or a script to evaluate, but cannot specify both. For more information about how to use selection conditions, see Defining Conditions for when Data is Displayed.

For more information about other components of a wide view, see Wide View.

See Also

Creating a Wide View

Defining Conditions for When Data Is Displayed

ScriptBlock Element for SelectionCondition for EntrySelectedBy for WideEntry

SelectionCondition Element for EntrySelectedBy for WideEntry

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for SelectionCondition for EntrySelectedBy for WideControl

Specifies the script that triggers the condition. When this script is evaluated to `true`, the condition is met, and the wide entry definition is used.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- WideControl Element
- WideEntries Element
- WideEntry Element
- EntrySelectedBy Element
- SelectionCondition
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToEvaluate</ScriptBlock>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for WideEntry	Defines the condition that must exist for this definition to be used.

Text Value

Specify the script that is evaluated.

Remarks

The selection condition must specify at least one script or property name to evaluate, but cannot specify both. For more information about how to use selection conditions, see Defining Conditions for when Data is Displayed.

For more information about other components of a wide view, see Wide View.

See Also

Creating a Wide View

Defining Conditions for When Data Is Displayed

PropertyName Element for SelectionCondition for EntrySelectedBy for WideEntry

SelectionCondition Element for EntrySelectedBy for WideEntry

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSetName Element for SelectionCondition for EntrySelectedBy for WideEntry

Specifies the set of .NET types that trigger the condition. When any of the types in this set are present, the condition is met, and the object is displayed by using this definition of the wide view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- WideControl Element
- WideEntries Element
- WideEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- SelectionSetName Element

Syntax

XML

```
<SelectionSetName>NameofSelectionSet</SelectionSetName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `SelectionSetName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for WideEntry	Defines the condition that must exist for this definition to be used.

Text Value

Specify the name of the selection set.

Remarks

The selection condition can specify a selection set or .NET type, but cannot specify both. For more information about how to use selection conditions, see Defining Conditions for when Data is Displayed.

Selection sets are common groups of .NET objects that can be used by any view that the formatting file defines. For more information about creating and referencing selection sets, see Defining Sets of Objects.

For more information about other components of a wide view, see Creating a Wide View.

See Also

Creating a Wide View

Defining Conditions for When Data Is Displayed

Defining Selection Sets

SelectionCondition Element for EntrySelectedBy for WideEntry

TypeName Element for SelectionCondition for EntrySelectedBy for WideEntry

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for SelectionCondition for EntrySelectedBy for WideControl

Specifies a .NET type that triggers the condition. When this type is present, the definition is used.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- WideControl Element
- WideEntries Element
- WideEntry Element
- EntrySelectedBy Element
- SelectionCondition Element
- TypeName Element

Syntax

XML

```
<TypeName>Nameof.NetType</TypeName>
```

Attributes and Elements

The following sections describe attributes, child elements, and the parent element of the `TypeName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
SelectionCondition Element for EntrySelectedBy for WideEntry	Defines the condition that must exist for this wide entry to be used.

Text Value

Specify the fully qualified name of the .NET type, such as `System.IO.DirectoryInfo`.

Remarks

The selection condition can specify a .NET type or a selection set, but cannot specify both. For more information about how to use selection conditions, see Defining Conditions for when Data is Displayed.

For more information about other components of a wide view, see Creating a Wide View.

See Also

Creating a Wide View

Defining Conditions for When Data Is Displayed

SelectionCondition Element for EntrySelectedBy for WideEntry

SelectionSetName Element for SelectionCondition for EntrySelectedBy for WideEntry

Writing a PowerShell Formatting File

Last updated on 05/20/2025

TypeName Element for EntrySelectedBy for WideEntry

Specifies a .NET type for the definition. The definition is used whenever this object is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- WideControl Element
- WideEntries Element
- WideEntry Element
- EntrySelectedBy Element
- TypeName Element

Syntax

XML

```
<TypeName>Nameof.NetType</TypeName>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `TypeName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for WideEntry	Defines the .NET types that use this wide entry or the condition that must exist for this entry to be used.

Text Value

Specify the fully qualified name of the .NET type, such as `System.IO.DirectoryInfo`.

Remarks

Each wide entry must specify one or more .NET types, a selection set, or the selection condition that must exist for the definition to be used.

For more information about other components of a wide view, see Creating a Wide View.

See Also

Creating a Wide View

EntrySelectedBy Element for WideEntry

Writing a PowerShell Formatting File

Last updated on 05/20/2025

SelectionSetName Element for EntrySelectedBy for WideControl

Specifies a set of .NET objects for the definition. The definition is used whenever one of these objects is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- WideControl Element
- WideEntries Element
- WideEntry Element
- EntrySelectedBy Element
- SelectionSetName Element

Syntax

XML

```
<SelectionSetName>NameofSelectionSet</SelectionSetName>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `SelectionSetName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
EntrySelectedBy Element for WideEntry	Defines the .NET types that use this wide entry or the condition that must exist for this entry to be used.

Text Value

Specify the name of the selection set.

Remarks

Each definition must specify one type name, selection set, or selection condition.

Selection sets are typically used when you want to define a group of objects that are used in multiple views. For example, you might want to create a table view and a list view for the same set of objects. For more information about defining selection sets, see Defining Sets of Objects for a View.

For more information about other components of a wide view, see Creating a Wide View.

See Also

Creating a Wide View

Defining Selection Sets

EntrySelectedBy Element for WideEntry

Writing a PowerShell Formatting File

Last updated on 05/20/2025

WideItem Element for WideControl

Defines the property or script whose value is displayed.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- WideControl Element
- WideEntries Element
- WideEntry Element
- WideItem Element

Syntax

XML

```
<WideItem>
  <PropertyName>.NetTypeProperty</PropertyName>
  <ScriptBlock>ScriptToExecute</ScriptBlock>
  <FormatString>FormatPattern</FormatString>
</WideItem>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `WideItem` element. The `FormatString` element is optional. However, you must specify a `PropertyName` or `ScriptBlock` element, but you cannot specify both.

Attributes

None.

Child Elements

Expand table

Element	Description
FormatString Element for WideItem for WideControl	Optional element. Specifies a format pattern that defines how the property or script value is displayed in the view.
PropertyName Element for WideItem	Specifies the property of the object whose value is displayed in the wide view.
ScriptBlock Element for WideItem	Specifies the script whose value is displayed in the wide view.

Parent Elements

Expand table

Element	Description
WideEntry Element	Provides a definition of the wide view.

Remarks

For more information about the components of a wide view, see Wide View.

Example

The following example shows a `WideEntry` element that defines a single `WideItem` element. The `WideItem` element defines the property or script whose value is displayed in the view.

XML

```
<WideEntry>
  <WideItem>
    <PropertyName>ProcessName</PropertyName>
  </WideItem>
</WideEntry>
```

For a complete example of a wide view, see Wide View (Basic).

See Also

FormatString Element for WideItem for WideControl

PropertyName Element for WideItem

ScriptBlock Element for WideItem

WideEntry Element

Writing a PowerShell Formatting File

Last updated on 05/20/2025

FormatString Element for WideItem

Specifies a format pattern that defines how the property or script value is displayed in the view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- WideControl Element
- WideEntries Element
- WideEntry Element
- WideItem Element
- FormatString Element

Syntax

XML

```
<FormatString>PropertyPattern</FormatString>
```

Attributes and Elements

The following sections describe the attributes, child elements, and the parent element of the `FormatString` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
WideItem Element for WideControl	Defines the property or script whose value is displayed in a row of the list view.

Text Value

Specify the pattern that is used to format the data. For example, you can use this pattern to format the value of any property that is of type System.TimeSpan: {0:MMM}{0:dd}{0:HH}:{0:mm}.

Remarks

Format strings can be used when creating table views, list views, wide views, or custom views. For more information about formatting a value displayed in a view, see Formatting Displayed Data.

For more information about using format strings in wide views, see Creating a Wide View.

Example

The following example shows how to define a formatting string for the value of the `StartTime` property.

XML

```
<WideItem>
  <PropertyName>StartTime</PropertyName>
  <FormatString>{0:MMM} {0:DD} {0:HH}:{0:MM}</FormatString>
</WideItem>
```

See Also

Creating a Wide View

WideItem Element for WideControl

Writing a Windows PowerShell Formatting and Types File

Last updated on 05/20/2025

PropertyName Element for WideItem for WideControl

Specifies the property of the object whose value is displayed in the wide view.

- Configuration Element
- ViewDefinitions Element
- View Element
- WideControl Element
- WideEntries Element
- WideEntry Element
- WideItem Element
- PropertyName Element

Syntax

XML

```
<PropertyName>.NetTypeProperty</PropertyName>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `PropertyName` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
WideItem Element	Defines the property or script whose value is displayed in the wide view.

Text Value

Specify the name of the property whose value is displayed.

Remarks

For more information about the components of a wide view, see Creating a Wide View.

Example

This example shows a wide view that displays the value of the ProcessName property of the System.Diagnostics.Process object.

XML

```
View>
  <Name>process</Name>
  <ViewSelectedBy>
    <TypeName>System.Diagnostics.Process</TypeName>
  </ViewSelectedBy>
  <WideControl>
    <WideEntries>
      <WideEntry>
        <WideItem>
          <PropertyName>ProcessName</PropertyName>
        </WideItem>
      </WideEntry>
    </WideEntries>
  </WideControl>
</View>
```

See Also

WideItem Element

Creating a Wide View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

ScriptBlock Element for WideItem for WideControl

Specifies the script whose value is displayed in the wide view.

Schema

- Configuration Element
- ViewDefinitions Element
- View Element
- WideControl Element
- WideEntries Element
- WideEntry Element
- WideItem Element
- ScriptBlock Element

Syntax

XML

```
<ScriptBlock>ScriptToExecute</ScriptBlock>
```

Attributes and Elements

The following sections describe the attributes, child elements, and parent element of the `ScriptBlock` element.

Attributes

None.

Child Elements

None.

Parent Elements

Expand table

Element	Description
WideItem Element	Defines the property or script block whose value is displayed in the wide view.

Text Value

Specify the script whose value is displayed.

Remarks

For more information about the components of a wide view, see Creating a Wide View.

Example

This example shows a `WideItem` element that defines a script whose value is displayed in the view.

XML

```
<WideItem>
  <ScriptBlock>ScriptToExecute</ScriptBlock>
</WideItem>
```

See Also

WideItem Element

Creating a Wide View

Writing a PowerShell Formatting File

Last updated on 05/20/2025

Writing Help for PowerShell Scripts and Functions

PowerShell scripts and functions should be fully documented whenever they're shared with others. The `Get-Help` cmdlet displays the script and function help topics in the same format as it displays help for cmdlets, and all the `Get-Help` parameters work on script and function help topics.

PowerShell scripts can include a help topic about the script and help topics about each functions in the script. Functions that are shared independently of scripts can include their own help topics.

This document explains the format and correct placement of the help topics, and it suggests guidelines for the content.

Types of Script and Function Help

Comment-Based Help

The help topic that describes a script or function can be implemented as a set of comments within the script or function. When writing comment-based help for a script and for functions in a script, pay careful attention to the rules for placing the comment-based help. The placement determines whether the `Get-Help` cmdlet associates the help topic with the script or a function. For more information about writing comment-based help topics, see about_Comment_Based_Help.

XML-Based Command Help

The help topic that describes a script or function can be implemented in an XML file that uses the command help schema. To associate the script or function with the XML file, use the `.EXTERNALHELP` comment keyword followed by the path and name of the XML file.

When the `.EXTERNALHELP` comment keyword is present, it takes precedence over comment-based help, even when `Get-Help` can't find a help file that matches the value of the `.EXTERNALHELP` keyword.

Online Help

You can post your help topics on the internet and then direct `Get-Help` to open the topics. For more information about writing comment-based help topics, see Supporting Online Help.

There is no established method for writing conceptual ("About") topics for scripts and functions. However, you can post conceptual topics on the internet list the topics and their URLs in the Related Links section of a command help topic.

Content Considerations for Script and Function Help

- If you are writing a very brief help topic with only a few of the available command help sections, be sure to include clear descriptions of the script or function parameters. Also include one or two sample commands in the examples section, even if you decide to omit example descriptions.

- In all descriptions, refer to the command as a script or function. This information helps the user to understand and manage the command.

  For example, the following detailed description states that the New-Topic command is a script. This reminds users that they need to specify the path and full name when they run it.

  > "The New-Topic script creates a blank conceptual topic for each topic name in the input file..."

  The following detailed description states that `Disable-PSRemoting` is a function. This information is particularly useful to users when the session includes multiple commands with the same name, some of which might be hidden by a command with higher precedence.

  > The `Disable-PSRemoting` function disables all session configurations on the local computer...

- In a script help topic, explain how to use the script as a whole. If you are also writing help topics for functions in the script, mention the functions in your script help topic and include references to the function help topics in the Related Links section of the script help topic. Conversely, when a function is part of a script, explain in the function help topic the role that the function plays in the script and how it might be used

independently. Then list the script help topic in the Related Links section of the function help topic.

- When writing examples for a script help topic, be sure to include the path to the script file in the example command. This reminds users that they must specify the path explicitly, even when the script is in the current directory.
- In a function help topic, remind users that the function exists only in the current session and, to use it in other sessions, they need to add it, or add it a PowerShell profile.
- `Get-Help` displays the help topic for a script or function only when the script file and help topic files are saved in the correct locations. Therefore, it's not useful to include instructions for installing PowerShell, or saving or installing the script or function in a script or function help topic. Instead, include any installation instructions in the document that you use to distribute the script or function.

See Also

Writing Comment-Based Help Topics

Last updated on 05/20/2025

Writing Comment-Based Help Topics

You can write comment-based Help topics for functions and scripts using special Help comment keywords.

The `Get-Help` cmdlet displays comment-based Help in the same format in which it displays the cmdlet Help topics that are generated from XML files. Users can use all of the parameters of `Get-Help`, such as Detailed, Full, Example, and Online, to display function and script Help.

You can also write XML-based Help topics for scripts and functions and use the Help comment keywords to redirect users to the XML-based topics or other topics.

In This Section

- Syntax of Comment-Based Help - Describes the syntax of comment-based help.
- Comment-Based Help Keywords - Lists the keywords in comment-based help.
- Placing Comment-Based Help in Functions - Shows where to place comment-based help for a function.
- Placing Comment-Based Help in Scripts - Shows where to place comment-based help for a script.

Last updated on 05/20/2025

Syntax of Comment-Based Help

This section describes the syntax of comment-based help.

Syntax Diagram

The syntax for comment-based Help is as follows:

```
# .< help keyword>
# <help content>
```

-or -

```
<#
.< help keyword>
< help content>
#>
```

Syntax Description

Comment-based Help is written as a series of comments. You can type a comment symbol (`#`) before each line of comments, or you can use the `<#` and `#>` symbols to create a comment block. All the lines within the comment block are interpreted as comments.

Each section of comment-based Help is defined by a keyword and each keyword is preceded by a dot (`.`). The keywords can appear in any order. The keyword names aren't case-sensitive.

A comment block must contain at least one help keyword. Some of the keywords, such as `.EXAMPLE`, can appear many times in the same comment block. The Help content for each keyword begins on the line after the keyword and can span multiple lines.

All the lines in a comment-based Help topic must be contiguous. If a comment-based Help topic follows a comment that isn't part of the Help topic, there must be at least one blank line between the last non-Help comment line and the beginning of the comment-based Help.

For example, the following comment-based help topic contains the `.DESCRIPTION` keyword and its value, which is a description of a function or script.

PowerShell

```
<#
    .DESCRIPTION
    The Get-Function function displays the name and syntax of all functions in the
session.
#>
```

Last updated on 05/20/2025

Comment-Based Help Keywords

09/23/2025

This topic lists and describes the keywords in comment-based help.

Keywords in Comment-Based Help

The following are valid comment-based Help keywords. They're listed in the order in which they typically appear in a Help topic along with their intended use. These keywords can appear in any order in the comment-based Help, and they're not case-sensitive.

Note that the `.EXTERNALHELP` keyword takes precedence over all other comment-based help keywords. When `.EXTERNALHELP` is present, the Get-Help cmdlet doesn't display comment-based help, even when it can't find a help file that matches the value of the keyword.

`.SYNOPSIS`

A brief description of the function or script. This keyword can be used only once in each topic.

`.DESCRIPTION`

A detailed description of the function or script. This keyword can be used only once in each topic.

`.PARAMETER <Parameter-Name>`

The description of a parameter. You can include a `.PARAMETER` keyword for each parameter in the function or script.

The `.PARAMETER` keywords can appear in any order in the comment block, but the order in which the parameters appear in the `param` statement or function declaration determines the order in which the parameters appear in Help topic. To change the order of parameters in the Help topic, change the order of the parameters in the `param` statement or function declaration.

You can also specify a parameter description by placing a comment in the `param` statement immediately before the parameter variable name. If you use both a `param` statement comment and a `.PARAMETER` keyword, the description associated with the `.PARAMETER` keyword is used, and the `param` statement comment is ignored.

`.EXAMPLE`

A sample command that uses the function or script, optionally followed by sample output and a description. Repeat this keyword for each example.

`.INPUTS`

The .NET types of objects that can be piped to the function or script. You can also include a description of the input objects. Repeat this keyword for each input type.

`.OUTPUTS`

The .NET type of the objects that the cmdlet returns. You can also include a description of the returned objects. Repeat this keyword for each output type.

`.NOTES`

Additional information about the function or script.

`.LINK`

The name of a related topic. Repeat this keyword for each related topic. This content appears in the Related Links section of the Help topic.

The `.LINK` keyword content can also include a Uniform Resource Identifier (URI) to an online version of the same help topic. The online version opens when you use the **Online** parameter of `Get-Help`. The URI must begin with `http` or `https`.

`.COMPONENT`

The name of the technology or feature that the function or script uses, or to which it's related. The **Component** parameter of `Get-Help` uses this value to filter the search results returned by `Get-Help`.

`.ROLE`

The name of the user role for the help topic. The **Role** parameter of `Get-Help` uses this value to filter the search results returned by `Get-Help`.

.FUNCTIONALITY

The keywords that describe the intended use of the function. The **Functionality** parameter of `Get-Help` uses this value to filter the search results returned by `Get-Help`.

.FORWARDHELPTARGETNAME <Command-Name>

Redirects to the help topic for the specified command. You can redirect users to any help topic, including help content for a function, script, cmdlet, or provider.

```
PowerShell
# .FORWARDHELPTARGETNAME <Command-Name>
```

.FORWARDHELPCATEGORY

Specifies the help category of the item in `.FORWARDHELPTARGETNAME`. Valid values are `Alias`, `Cmdlet`, `HelpFile`, `Function`, `Provider`, `General`, `FAQ`, `Glossary`, `ScriptCommand`, `ExternalScript`, `Filter`, or `All`. Use this keyword to avoid conflicts when there are commands with the same name.

```
PowerShell
# .FORWARDHELPCATEGORY <Category>
```

.REMOTEHELPRUNSPACE <PSSession-variable>

Specifies a session that contains the help topic. Enter a variable that contains a **PSSession** object. This keyword is used by the [Export-PSSession][09] cmdlet to find the help content for the exported commands.

```
PowerShell
# .REMOTEHELPRUNSPACE <PSSession-variable>
```

.EXTERNALHELP

Specifies an XML-based help file for the script or function.

```
```

PowerShell

```
# .EXTERNALHELP <XML Help File>
```

The `.EXTERNALHELP` keyword is required when a function or script is documented in XML files. Without this keyword, `Get-Help` can't find the XML-based help file for the function or script.

The `.EXTERNALHELP` keyword takes precedence over other comment-based help keywords. If `.EXTERNALHELP` is present, `Get-Help` doesn't display comment-based help, even if it can't find a help topic that matches the value of the `.EXTERNALHELP` keyword.

If the function is exported by a module, set the value of the `.EXTERNALHELP` keyword to a filename without a path. `Get-Help` looks for the specified filename in a language-specific subdirectory of the module directory. There are no requirements for the name of the XML-based help file for a function. Beginning in PowerShell 5.0, functions that are exported by a module can be documented in a help file that's named for the module. You don't need to use `.EXTERNALHELP` comment keyword. For example, if the `Test-Function` function is exported by the `MyModule` module, you can name the help file `MyModule-help.xml`. The `Get-Help` cmdlet looks for help for the `Test-Function` function in the `MyModule-help.xml` file in the module directory.

If the function isn't included in a module, include a path to the XML-based help file. If the value includes a path and the path contains UI-culture-specific subdirectories, `Get-Help` searches the subdirectories recursively for an XML file with the name of the script or function in accordance with the language fallback standards established for Windows, just as it does in a module directory.

For more information about the cmdlet help XML-based help file format, see How to Write Cmdlet Help.

Placing Comment-Based Help in Functions

This topic explains where to place comment-based help for a function so that the `Get-Help` cmdlet associates the comment-based help topic with the correct function.

Where to Place Comment-Based Help for a Function

- At the beginning of the function body.
- At the end of the function body.
- Before the `Function` keyword. When the function is in a script or script module, there can't be more than one blank line between the last line of the comment-based help and the `Function` keyword. Otherwise, `Get-Help` associates the help with the script, not with the function.

Examples of Help Placement in a Function

The following examples show each of the three placement options for comment-based help for a function.

Help at the Beginning of a Function Body

The following example shows comment-based at the beginning of a function body.

PowerShell

```
function MyProcess
{
    <#
       .DESCRIPTION
       The MyProcess function gets the Windows PowerShell process.
    #>

    Get-Process powershell
}
```

Help at the End of a Function Body

The following example shows comment-based at the end of a function body.

PowerShell

```
function MyFunction
{
    Get-Process powershell

    <#
       .DESCRIPTION
       The MyProcess function gets the Windows PowerShell process.
    #>
}
```

Help Before the Function Keyword

The following examples shows comment-based on the line before the function keyword.

PowerShell

```
<#
    .DESCRIPTION
    The MyProcess function gets the Windows PowerShell process.
#>
function MyFunction { Get-Process powershell}
```

Last updated on 05/20/2025

Placing Comment-Based Help in Scripts

This topic explains where to place comment-based help for a script so that the `Get-Help` cmdlet associates the comment-based help topic with scripts and not with any functions that might be in the script.

Where to Place Comment-Based Help for a Script

- At the beginning of the script file.

  Script Help can be preceded in the script only by comments and blank lines.

- At the end of the script file.

  If the first item in the script body (after the Help) is a function declaration, there must be at least two blank lines between the end of the script Help and the function declaration. Otherwise, the Help is interpreted as being Help for the function, not Help for the script.

Examples of Help Placement in a Script

The following examples show each of the placement options for comment-based help for a script.

Help at the Beginning of a Script

The following example shows comment-based at the beginning of a script.

PowerShell

```
<#
.DESCRIPTION
This script performs a series of network connection tests.
#>

param [string]$ComputerName
...
```

Help at the End of a Script

The following example shows comment-based at the end of a script.

PowerShell

```powershell
...
function Ping { Test-Connection -ComputerName $ComputerName }

<#
.DESCRIPTION
This script performs a series of network connection tests.
#>
```

Last updated on 05/20/2025

Autogenerated Elements of Comment-Based Help

The `Get-Help` cmdlet automatically generates several elements of a comment-based topic. These autogenerated elements make comment-based help look very much like the help that's generated from XML files.

Autogenerated Elements

The `Get-Help` cmdlet automatically generates the following elements of a help topic. You can't edit these elements directly, but you can change the results by changing the source of the element.

Name

The Name section of a function Help topic is taken from the function name in the function definition. The Name of a script Help topic is taken from the script filename. To change the name or its capitalization, change the function definition or the script filename.

Syntax

The Syntax section of the Help topic is generated from the parameter list in the `param` statement of the function or script. To add detail to the Help topic syntax, such as the .NET type of a parameter, add the detail to the parameter list. If you don't specify a parameter type, the **Object** type is inserted as the default value.

Parameter List

The Parameters section of the Help topic is generated from the parameter list in the function or script and from the descriptions that you add using the `.PARAMETER` keyword or comments in the parameter list.

Parameters appear in the Parameters section in the same order that they appear in the parameter list. The spelling and capitalization of parameter names is also taken from the parameter list; it isn't affected by the parameter name specified by the `.PARAMETER` keyword.

Common Parameters

The common parameters are added to the syntax and parameter list of the Help topic, even if they have no effect. For more information about the common parameters, see about_CommonParameters.

Parameter Attribute Table

`Get-Help` generates the table of parameter attributes that appears when you use the **Full** or **Parameter** parameter of `Get-Help`. The value of the **Required**, **Position**, and **Default** value attributes is taken from the function or script syntax.

Remarks

The Remarks section of the Help topic is automatically generated from the function or script name. You can't change or affect its content.

Last updated on 05/20/2025

Examples of Comment-based Help

This topic includes examples that demonstrate how to use comment-based help for scripts and functions.

Example 1: Comment-based Help for a Function

The following sample function includes comment-based Help.

PowerShell

```
function Add-Extension
{
    param ([string]$Name,[string]$Extension = "txt")
    $Name = $Name + "." + $Extension
    $Name

    <#
        .SYNOPSIS
        Adds a file name extension to a supplied name.

        .DESCRIPTION
        Adds a file name extension to a supplied name.
        Takes any strings for the file name or extension.

        .PARAMETER Name
        Specifies the file name.

        .PARAMETER Extension
        Specifies the extension. "Txt" is the default.

        .INPUTS
        None. You can't pipe objects to Add-Extension.

        .OUTPUTS
        System.String. Add-Extension returns a string with the extension or file
name.

        .EXAMPLE
        PS> Add-Extension -Name "File"
        File.txt

        .EXAMPLE
        PS> Add-Extension -Name "File" -Extension "doc"
        File.doc

        .EXAMPLE
        PS> Add-Extension "File" "doc"
```

```
        File.doc

        .LINK
        Online version: http://www.fabrikam.com/add-extension.html

        .LINK
        Set-Item
    #>
}
```

The following output shows the results of a `Get-Help` command that displays the help for the `Add-Extension` function.

PowerShell

```
PS> Get-Help Add-Extension -Full
```

Output

```
NAME
    Add-Extension

SYNOPSIS
    Adds a file name extension to a supplied name.

SYNTAX
    Add-Extension [[-Name] <String>] [[-Extension] <String>] [<CommonParameters>]

DESCRIPTION
    Adds a file name extension to a supplied name. Takes any strings for the file
name or extension.

PARAMETERS
    -Name
        Specifies the file name.

        Required?                    false
        Position?                    0
        Default value
        Accept pipeline input?       false
        Accept wildcard characters?

    -Extension
        Specifies the extension. "Txt" is the default.

        Required?                    false
        Position?                    1
        Default value
        Accept pipeline input?       false
        Accept wildcard characters?
```

```
    <CommonParameters>
        This cmdlet supports the common parameters: -Verbose, -Debug,
        -ErrorAction, -ErrorVariable, -WarningAction, -WarningVariable,
        -OutBuffer and -OutVariable. For more information, type
        "Get-Help about_CommonParameters".

INPUTS
    None. You can't pipe objects to Add-Extension.

OUTPUTS
    System.String. Add-Extension returns a string with the extension or file name.

    -------------------------- EXAMPLE 1 --------------------------

    PS> Add-Extension -Name "File"
    File.txt

    -------------------------- EXAMPLE 2 --------------------------

    PS> Add-Extension -Name "File" -Extension "doc"
    File.doc

    -------------------------- EXAMPLE 3 --------------------------

    PS> Add-Extension "File" "doc"
    File.doc

RELATED LINKS
    Online version: http://www.fabrikam.com/add-extension.html
    Set-Item
```

Example 2: Comment-based Help for a Script

The following sample function includes comment-based Help.

Notice the blank lines between the closing `#>` and the `param` statement. In a script that doesn't have a `param` statement, there must be at least two blank lines between the final comment in the Help topic and the first function declaration. Without these blank lines, `Get-Help` associates the Help topic with the function, instead of the script.

PowerShell

```powershell
<#
  .SYNOPSIS
  Performs monthly data updates.

  .DESCRIPTION
  The Update-Month.ps1 script updates the registry with new data generated
  during the past month and generates a report.
```

```
  .PARAMETER InputPath
  Specifies the path to the CSV-based input file.

  .PARAMETER OutputPath
  Specifies the name and path for the CSV-based output file. By default,
  MonthlyUpdates.ps1 generates a name from the date and time it runs, and
  saves the output in the local directory.

  .INPUTS
  None. You can't pipe objects to Update-Month.ps1.

  .OUTPUTS
  None. Update-Month.ps1 doesn't generate any output.

  .EXAMPLE
  PS> .\Update-Month.ps1

  .EXAMPLE
  PS> .\Update-Month.ps1 -InputPath C:\Data\January.csv

  .EXAMPLE
  PS> .\Update-Month.ps1 -InputPath C:\Data\January.csv -OutputPath
C:\Reports\2009\January.csv
#>

param ([string]$InputPath, [string]$OutputPath)

function Get-Data { }
```

The following command gets the script Help. Because the script isn't in a directory that's listed in the PATH environment variable, the `Get-Help` command that gets the script Help must specify the script path.

PowerShell

```
PS> Get-Help C:\ps-test\update-month.ps1 -Full
```

Output

```
NAME
    C:\ps-test\Update-Month.ps1

SYNOPSIS
    Performs monthly data updates.

SYNTAX
    C:\ps-test\Update-Month.ps1 [-InputPath] <String> [[-OutputPath]
    <String>] [<CommonParameters>]

DESCRIPTION
    The Update-Month.ps1 script updates the registry with new data
```

```
    generated during the past month and generates a report.

PARAMETERS
    -InputPath
        Specifies the path to the CSV-based input file.

        Required?                    true
        Position?                    0
        Default value
        Accept pipeline input?       false
        Accept wildcard characters?

    -OutputPath
        Specifies the name and path for the CSV-based output file. By
        default, MonthlyUpdates.ps1 generates a name from the date
        and time it runs, and saves the output in the local directory.

        Required?                    false
        Position?                    1
        Default value
        Accept pipeline input?       false
        Accept wildcard characters?

    <CommonParameters>
        This cmdlet supports the common parameters: -Verbose, -Debug,
        -ErrorAction, -ErrorVariable, -WarningAction, -WarningVariable,
        -OutBuffer and -OutVariable. For more information, type,
        "Get-Help about_CommonParameters".

INPUTS
        None. You can't pipe objects to Update-Month.ps1.

OUTPUTS
        None. Update-Month.ps1 doesn't generate any output.

-------------------------- EXAMPLE 1 -------------------------

PS> .\Update-Month.ps1

-------------------------- EXAMPLE 2 -------------------------

PS> .\Update-Month.ps1 -InputPath C:\Data\January.csv

-------------------------- EXAMPLE 3 -------------------------

PS> .\Update-Month.ps1 -InputPath C:\Data\January.csv -OutputPath
C:\Reports\2009\January.csv

RELATED LINKS
```

Example 3: Parameter Descriptions in a `param` Statement

This example shows how to insert parameter descriptions in the `param` statement of a function or script. This format is most useful when the parameter descriptions are brief.

PowerShell

```
function Add-Extension
{
    param
    (
        [string]
        # Specifies the file name.
        $Name,

        [string]
        # Specifies the file name extension. "Txt" is the default.
        $Extension = "txt"
    )
    $Name = $Name + "." + $Extension
    $Name

    <#
        .SYNOPSIS
        Adds a file name extension to a supplied name.
...
    #>
}
```

The results are the same as the results for Example 1. `Get-Help` interprets the parameter descriptions as though they were accompanied by the `.PARAMETER` keyword.

Example 4: Redirecting to an XML File

You can write XML-based Help topics for functions and scripts. Although comment-based Help is easier to implement, XML-based Help is required if you want more precise control over Help content or if you are translating Help topics into multiple languages.The following example shows the first few lines of the `Update-Month.ps1` script. The script uses the `.EXTERNALHELP` keyword to specify the path to an XML-based Help topic for the script.

PowerShell

```
# .EXTERNALHELP C:\MyScripts\Update-Month-Help.xml

    param ([string]$InputPath, [string]$OutputPath)

    function Get-Data { }
```

The following example shows the use of the `.EXTERNALHELP` keyword in a function.

PowerShell

```
function Add-Extension
{
    param ([string]$Name, [string]$Extension = "txt")
    $Name = $Name + "." + $Extension
    $Name

    # .EXTERNALHELP C:\ps-test\Add-Extension.xml
}
```

Example 5: Redirecting to a Different Help Topic

The following code is an excerpt from the beginning of the built-in `help` function in PowerShell, which displays one screen of Help text at a time. Because the Help topic for the Get-Help cmdlet describes the Help function, the Help function uses the `.FORWARDHELPTARGETNAME` and `.FORWARDHELPCATEGORY` keywords to redirect the user to the Get-Help cmdlet Help topic.

PowerShell

```
function help
{
    <#
      .FORWARDHELPTARGETNAME Get-Help
      .FORWARDHELPCATEGORY Cmdlet
    #>
    [CmdletBinding(DefaultParameterSetName='AllUsersView')]
    param(
            [Parameter(Position=0, ValueFromPipelineByPropertyName=$true)]
            [System.String]
            ${Name},
    ...
}
```

The following command uses this feature. When a user types a `Get-Help` command for the `help` function, `Get-Help` displays the Help topic for the `Get-Help` cmdlet.

PowerShell

```
PS> Get-Help help
```

Output

```
NAME
    Get-Help
```

```
SYNOPSIS
    Displays information about Windows PowerShell cmdlets and concepts.
...
```

Last updated on 05/20/2025

Writing Help for PowerShell Cmdlets

PowerShell cmdlets can be useful, but unless your Help topics clearly explain what the cmdlet does and how to use it, the cmdlet may not get used or, even worse, it might frustrate users. The XML-based cmdlet Help file format enhances consistency, but great help requires much more.

If you have never written cmdlet Help, review the following guidelines. The XML schema required to author the cmdlet Help topic is described in the following section. Start with Creating the Cmdlet Help File. That topic includes a description of the top-level XML nodes.

Writing Guidelines for Cmdlet Help

Write well

Nothing replaces a well-written topic. If you aren't a professional writer, find a writer or editor to help you. Another alternative is to copy your Help text into Microsoft Word and use the grammar and spelling checks to improve your work.

Write simply

Use simple words and phrases. Avoid jargon. Consider that many readers are equipped only with a foreign-language dictionary and your Help topic.

Write consistently

Help for related cmdlets should be similar (for example, `Get-Content` and `Set-Content`). Use the standard descriptions for standard parameters, like **Force** and **InputObject**. (Copy them from Help for the core cmdlets.) Use standard terms. For example, use "parameter", not "argument", and use "cmdlet" not "command" or "command-let."

Start the synopsis with a verb

The synopsis field informs the user what the cmdlet does, not what it's or how it works. Verbs create a task-based statement that informs users if this cmdlet meets their requirements. Use simple verbs like "get", "create", and "change." Avoid "set", which can be vague and fancy words like "modify".

Focus on objects

Most "get" cmdlets display something, but their primary function is to get an object. In your Help, focus on the object, so that users understand that the default display is one of many, and that they can use the methods and properties of the object that you retrieved for them in different ways.

Write detailed descriptions

Briefly list everything that the cmdlet can do in the detailed description. If the main function is to change one property, but the cmdlet can change all properties, list this in the detailed description.

Use conventional syntax

Use the standard Backus-Naur format which is common for Windows and Unix command-line Help.

Use Microsoft .NET types for parameter values

The placeholders for parameter values (in the syntax and parameter descriptions) show the .NET Framework types of the objects that the parameter will accept. The PowerShell team developed this convention to help teach users about the .NET Framework.

Write complete parameter descriptions

Parameter descriptions must inform users of two things: what the parameter does (its effect) and what they must type for the parameter values.

Write practical examples

The examples should show how to use all of the parameters, but the most important thing is to show how to use the cmdlet in real-world tasks. Start with a simple example and write increasingly complex examples. In the final example, show how to use the cmdlet in a pipeline.

Use the Notes field

Use the Notes field to explain concepts that users need to understand the cmdlet. You can also use notes to help users avoid common errors. Avoid URLs as they change. Instead, provide users terms to search for.

Test your Help

Test the Help just like you test your code. Have friends and colleagues read your Help content and provide feedback. You can also solicit feedback from newsgroups.

See Also

- How to Create the Cmdlet Help File
- How to Add the Cmdlet Name and Synopsis to a Cmdlet Help Topic
- How to Add the Detailed Description to a Cmdlet Help Topic
- How to Add Syntax to a Cmdlet Help Topic
- How to Add Parameters to a Cmdlet Help Topic
- How to add Input Types to a Cmdlet Help Topic
- How to Add Return Values to a Cmdlet Help Topic
- How to Add Notes to a Cmdlet Help Topic
- How to Add Examples to a Cmdlet Help Topic
- How to Add Related Links to a Cmdlet Help Topic
- Windows PowerShell SDK

Last updated on 05/20/2025

How to create the cmdlet help file

> ⓘ Note
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

This section describes how to create a valid XML file that contains content for Windows PowerShell cmdlet Help topics. This section discusses how to name the Help file, how to add the appropriate XML headers, and how to add nodes that will contain the different sections of the cmdlet Help content.

> ⓘ Note
>
> For a complete view of a Help file, open one of the `dll-Help.xml` files located in the Windows PowerShell installation directory. For example, the `Microsoft.PowerShell.Commands.Management.dll-Help.xml` file contains content for several of the PowerShell cmdlets.

How to create a cmdlet help file

1. Create a text file and save it using UTF8 encoding. The filename must have the following format so that Windows PowerShell can detect it as a cmdlet Help file.

   `<PSSnapInAssemblyName>.dll-Help.xml`

2. Add the following XML headers to the text file. Be aware that the file will be validated against the Microsoft Assistance Markup Language (MAML) schema. Currently, PowerShell doesn't provide any tools to validate the file.

   `<?xml version="1.0" encoding="utf-8" ?> <helpItems xmlns="http://msh" schema="maml">`

3. Add a **Command** node to the cmdlet Help file for each cmdlet in the assembly. Each node within the **Command** node relates to the different sections of the cmdlet Help topic.

The following table lists the XML element for each node, followed by a descriptions of each node.

Expand table

Node	Description
`<details>`	Adds content for the NAME and SYNOPSIS sections of the cmdlet Help topic. For more information, see How to Add the Cmdlet Name and Synopsis.
`<maml:description>`	Adds content for the DESCRIPTION section of the cmdlet Help topic. For more information, see How to Add the Detailed Description to a Cmdlet Help Topic.
`<command:syntax>`	Adds content for the SYNTAX section of the cmdlet Help topic. For more information, see How to Add Syntax to a Cmdlet Help Topic.
`<command:parameters>`	Adds content for the PARAMETERS section of the cmdlet Help topic. For more information, see How to Add Parameters to a Cmdlet Help Topic.
`<command:inputTypes>`	Adds content for the INPUTS section of the cmdlet Help topic. For more information, see How to Add Input Types to a Cmdlet Help Topic.
`<command:returnValues>`	Adds content for the OUTPUTS section of the cmdlet Help topic. For more information, see How to Add Return Values to a Cmdlet Help Topic.
`<maml:alertset>`	Adds content for the NOTES section of the cmdlet Help topic. For more information, see How to add Notes to a Cmdlet Help Topic.
`<command:examples>`	Adds content for the EXAMPLES section of the cmdlet Help topic. For more information, see How to Add Examples to a Cmdlet Help Topic.
`<maml:relatedLinks>`	Adds content for the RELATED LINKS section of the cmdlet Help topic. For more information, see How to Add Related Links to a Cmdlet Help Topic.

Example

Here is an example of a **Command** node that includes the nodes for the various sections of the cmdlet Help topic.

XML

```
<command:command
  xmlns:maml="http://schemas.microsoft.com/maml/2004/10"
  xmlns:command="http://schemas.microsoft.com/maml/dev/command/2004/10"
  xmlns:dev="http://schemas.microsoft.com/maml/dev/2004/10">
  <command:details>
```

```
    <!--Add name and synopsis here-->
  </command:details>
  <maml:description>
    <!--Add detailed description here-->
  </maml:description>
  <command:syntax>
    <!--Add syntax information here-->
  </command:syntax>
  <command:parameters>
    <!--Add parameter information here-->
  </command:parameters>
  <command:inputTypes>
    <!--Add input type information here-->
  </command:inputTypes>
  <command:returnValues>
    <!--Add return value information here-->
  </command:returnValues>
  <maml:alertSet>
    <!--Add Note information here-->
  </maml:alertSet>
  <command:examples>
    <!--Add cmdlet examples here-->
  </command:examples>
  <maml:relatedLinks>
    <!--Add links to related content here-->
  </maml:relatedLinks>
</command:command>
```

See also

- How to Add the Cmdlet Name and Synopsis
- How to Add the Detailed Description to a Cmdlet Help Topic
- How to Add Syntax to a Cmdlet Help Topic
- How to Add Parameters to a Cmdlet Help Topic
- How to Add Input Types to a Cmdlet Help Topic
- How to Add Return Values to a Cmdlet Help Topic
- How to add Notes to a Cmdlet Help Topic
- How to Add Examples to a Cmdlet Help Topic
- How to Add Related Links to a Cmdlet Help Topic
- Windows PowerShell SDK

Last updated on 05/20/2025

How to add the cmdlet name and synopsis to a cmdlet help topic

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

This section describes how to add content that's displayed in the **NAME** and **SYNOPSIS** sections of the cmdlet help. In the Help file, this content is added to the Command node for each cmdlet.

> **Note**
>
> For a complete view of a Help file, open one of the `dll-Help.xml` files located in the PowerShell installation directory. For example, the `Microsoft.PowerShell.Commands.Management.dll-Help.xml` file contains content for several of the PowerShell cmdlets.

To add the cmdlet name and a synopsis

The cmdlet Help can display two descriptions for the cmdlet. The first description is a short description that's referred to as the synopsis. The second description is a more detailed description that's discussed in Adding the Detailed Description to a Cmdlet Help Topic. Both these descriptions should be written as a single paragraph.

The `<command:details>` node contains the cmdlet name and the synopsis. The cmdlet name is enclosed in a `<command:name>` element, and the synopsis is enclosed in a `<maml:description>` element. The `<command:verb>` and `<command:noun>` elements contain the cmdlet verb and noun, respectively. For example, the following XML shows the `<command:details>` node for the `Get-ChildItem` cmdlet.

XML

```
<command:details>
  <command:name>Get-ChildItem</command:name>
  <maml:description>
    <maml:para>Gets the items and child items in one or more specified locations.
</maml:para>
  </maml:description>
  <command:verb>Get</command:verb>
  <command:noun>ChildItem</command:noun>
</command:details>
```

Writing tips

- In the synopsis don't repeat the cmdlet name. Informing the user that the `Get-Server` cmdlet gets a server is brief, but not informative. Instead, use synonyms and add details to the description.

  Example: "Gets an object that represents a local or remote computer."

- Use simple verbs like "get", "create", and "change" in the synopsis. Avoid using "set" because it is vague, and fancy words such as "modify."

  Example: "Gets information about the Authenticode signature in a file."

- Write in active voice. For example, "Use the TimeSpan object..." is much clearer than "the TimeSpan object can be used to..."

- Avoid the verb "display" when describing cmdlets that get objects. Although Windows PowerShell displays cmdlet data, it's important to introduce users to the concept that the cmdlet returns .NET Framework objects whose data may not be displayed. If you emphasize the display, the user might not realize that the cmdlet may have returned many other useful properties and methods that aren't displayed.

See also

Windows PowerShell SDK

Last updated on 07/07/2026

How to Add a Cmdlet Description

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

This section describes how to add content that's displayed in the **DESCRIPTION** section of the cmdlet Help. In the Help file, this content is added to the **Command** node for each cmdlet.

> **Note**
>
> For a complete view of a Help file, open one of the `dll-Help.xml` files located in the PowerShell installation directory. For example, the `Microsoft.PowerShell.Commands.Management.dll-Help.xml` file contains content for several of the PowerShell cmdlets.

To Add a Description

The `<maml:description>` node contains the detailed description of the cmdlet. This element contains one or more `<maml:para>` elements. The description is enclosed in a `<maml:para>` element. The following example shows the structure of the XML:

XML

```xml
<maml:description>
  <maml:para>...</maml:para>
</maml:description>
```

Writing tips

- Begin by explaining the basic features of the cmdlet in more detail. In many cases, you can explain the terms used in the cmdlet name and illustrate unfamiliar concepts with an example. For example, if the cmdlet appends data to a file, explain that it adds data to the end of an existing file.

- To find all of the features of the cmdlet, review the parameter list. Describe the primary function of the cmdlet, and then include other functions and features. For example, if the main function of the cmdlet is to change one property, but the cmdlet can change all of the properties, say so in the detailed description. If the cmdlet parameters let the users solicit information in different ways, explain it.

- Include information on ways that users can use the cmdlet, in addition to the obvious uses. For example, you can use the object that the `Get-Host` cmdlet retrieves to change the color of text in the Windows PowerShell command window.

  Example: "The `Get-Acl` cmdlet gets objects that represent the security descriptor of a file or resource. The security descriptor contains the access control lists (ACLs) of the resource. The ACL specifies the permissions that users and user groups have to access the resource."

- The detailed description should describe the cmdlet, but it shouldn't describe concepts that the cmdlet uses. Place concept definitions in Additional Notes.

See Also

Windows PowerShell SDK

Last updated on 07/07/2026

How to add syntax to a cmdlet help topic

> ⓘ Note
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

Before you start to code the XML for the syntax diagram in the cmdlet Help file, read this section to get a clear picture of the kind of data you need to provide, such as the parameter attributes, and how that data is displayed in the syntax diagram..

Parameter attributes

- Required
  - If true, the parameter must appear in all commands that use the parameter set.
  - If false, the parameter is optional in all commands that use the parameter set.
- Position
  - If named, the parameter name is required.
  - If positional, the parameter name is optional. When it's omitted, the parameter value must be in the specified position in the command. For example, if the value is position="1", the parameter value must be the first or only unnamed parameter value in the command.
- Pipeline Input
  - If true (ByValue), you can pipe input to the parameter. The input is associated with ("bound to") the parameter even if the property name and the object type don't match the expected type. The PowerShell parameter binding components try to convert the input to the correct type and fail the command only when the type can't be converted. Only one parameter in a parameter set can be associated by value.
  - If true (ByPropertyName), you can pipe input to the parameter. However, the input is associated with the parameter only when the parameter name matches the name of a property of the input object. For example, if the parameter name is `Path`, objects piped to the cmdlet are associated with that parameter only when the object has a property named path.

    - If true (ByValue, ByPropertyName), you can pipe input to the parameter either by property name or by value. Only one parameter in a parameter set can be associated by value.
    - If false, you can't pipe input to this parameter.
- Globbing
    - If true, the text that the user types for the parameter value can include wildcard characters.
    - If false, the text that the user types for the parameter value can't include wildcard characters.

Parameter value attributes

- Required
    - If true, the specified value must be used whenever using the parameter in a command.
    - If false, the parameter value is optional. Typically, a value is optional only when it's one of several valid values for a parameter, such as in an enumerated type.

The **Required** attribute of a parameter value is different from the **Required** attribute of a parameter.

The required attribute of a parameter indicates whether the parameter (and its value) must be included when invoking the cmdlet. In contrast, the required attribute of a parameter value is used only when the parameter is included in the command. It indicates whether that particular value must be used with the parameter.

Typically, parameter values that are placeholders are required and parameter values that are literal aren't required, because they're one of several values that might be used with the parameter.

Gathering syntax information

1. Start with the cmdlet name.

    ```
    SYNTAX
        Get-Tech
    ```

2. List all the parameters of the cmdlet. Type a hyphen (`-`) before each parameter name. Separate the parameters into parameter sets (some cmdlets may have only one parameter

set). In this example the Get-Tech cmdlet has two parameter sets.

```
SYNTAX
    Get-Tech -Name -Type
    Get-Tech -Id -List -Type
```

Start each parameter set with the cmdlet name.

List the default parameter set first. The default parameter is specified by the cmdlet class.

For each parameter set, list its unique parameter first, unless there are positional parameters that must appear first. In the previous example, the Name and Id parameters are unique parameters for the two parameter sets (each parameter set must have one parameter that's unique to that parameter set). This makes it easier for users to identify what parameter they need to supply for the parameter set.

List the parameters in the order that they should appear in the command. If the order doesn't matter, list related parameters together, or list the most frequently used parameters first.

Be sure to list the WhatIf and Confirm parameters if the cmdlet supports ShouldProcess.

Don't list the common parameters (such as Verbose, Debug, and ErrorAction) in your syntax diagram. The `Get-Help` cmdlet adds that information for you when it displays the Help topic.

3. Add the parameter values. In PowerShell, parameter values are represented by their .NET type. However, the type name can be abbreviated, such as "string" for System.String.

```
SYNTAX
    Get-Tech -Name string -Type Basic Advanced
    Get-Tech -Id int -List -Type Basic Advanced
```

Abbreviate types as long as their meaning is clear, such as **string** for **System.String** and **int** for **System.Int32**.

List all values of enumerations, such as the `-Type` parameter in the previous example, which can be set to **basic** or **advanced**.

`[switch]` parameters, such as `-List` in the previous example, don't have values.

4. Add angle brackets to parameters values that are placeholder, as compared to parameter values that are literals.

```
SYNTAX
    Get-Tech -Name <string> -Type Basic Advanced
    Get-Tech -Id <int> -List -Type Basic Advanced
```

5. Enclose optional parameters and their vales in square brackets.

```
SYNTAX
    Get-Tech -Name <string> [-Type Basic Advanced]
    Get-Tech -Id <int> [-List] [-Type Basic Advanced]
```

6. Enclose optional parameters names (for positional parameters) in square brackets. The name for parameters that are positional, such as the Name parameter in the following example, don't have to be included in the command.

```
SYNTAX
    Get-Tech [-Name] <string> [-Type Basic Advanced]
    Get-Tech -Id <int> [-List] [-Type Basic Advanced]
```

7. If a parameter value can contain multiple values, such as a list of names in the Name parameter, add a pair of square brackets directly following the parameter value.

```
SYNTAX
    Get-Tech [-Name] <string[]> [-Type Basic Advanced]
    Get-Tech -Id <int[]> [-List] [-Type Basic Advanced]
```

8. If the user can choose from parameters or parameter values, such as the Type parameter, enclose the choices in curly brackets and separate them with the exclusive OR symbol(;).

```
SYNTAX
    Get-Tech [-Name] <string[]> [-Type {Basic | Advanced}]
```

```
    Get-Tech -Id <int[]> [-List] [-Type {Basic | Advanced}]
```

9. If the parameter value must use specific formatting, such as quotation marks or parentheses, show the format in the syntax.

```

SYNTAX
    Get-Tech [-Name] <"string[]"> [-Type {Basic | Advanced}]
    Get-Tech -Id <int[]> [-List] [-Type {Basic | Advanced}]
```

Coding the syntax diagram XML

The syntax node of the XML begins immediately after the description node, which ends with the `</maml:description>` tag. For information about gathering the data used in the syntax diagram, see Gathering Syntax Information.

Adding a syntax node

The syntax diagram displayed in the cmdlet Help topic is generated from the data in the syntax node of the XML. The syntax node is enclosed in a pair of `<command:syntax>` tags. With each parameter set of the cmdlet enclosed in a pair of `<command:syntaxitem>` tags. There is no limit to the number of `<command:syntaxitem>` tags that you can add.

The following example shows a syntax node that has syntax item nodes for two parameter sets.

XML

```
<command:syntax>
  <command:syntaxItem>
    ...
    <!--Parameter Set 1 (default parameter set) parameters go here-->
    ...
  </command:syntaxItem>
  <command:syntaxItem>
    ...
    <!--Parameter Set 2 parameters go here-->
    ...
  </command:syntaxItem>
</command:syntax>
```

Adding the cmdlet name to the parameter set data

Each parameter set of the cmdlet is specified in a syntax item node. Each syntax item node begins with a pair of `<maml:name>` tags that include the name of the cmdlet.

The following example includes a syntax node that has syntax item nodes for two parameter sets.

XML

```
<command:syntax>
  <command:syntaxItem>
    <maml:name>Cmdlet-Name</maml:name>
  </command:syntaxItem>
  <command:syntaxItem>
    <maml:name>Cmdlet-Name</maml:name>
  </command:syntaxItem>
</command:syntax>
```

Adding parameters

Each parameter added to the syntax item node is specified within a pair of `<command:parameter>` tags. You need a pair of `<command:parameter>` tags for each parameter included in the parameter set, with the exception of the common parameters that are provided by PowerShell.

The attributes of the opening `<command:parameter>` tag determine how the parameter appears in the syntax diagram. For information on parameter attributes, see Parameter Attributes.

> **Note**
>
> The `<command:parameter>` tag supports a child element `<maml:description>` whose content is never displayed. The parameter descriptions are specified in the parameter node of the XML. To avoid inconsistencies between the information in the syntax item nodes and the parameter nodes, omit the `<maml:description>` or leave it empty.

The following example includes a syntax item node for a parameter set with two parameters.

XML

```
<command:syntaxItem>
  <maml:name>Cmdlet-Name</maml:name>
  <command:parameter required="true" globbing="true"
    pipelineInput="true (ByValue)" position="1">
    <maml:name>ParameterName1</maml:name>
    <command:parameterValue required="true">
      string[]
    </command:parameterValue>
  </command:parameter>
```

```
  <command:parameter required="true" globbing="true"
    pipelineInput="true (ByPropertyName)">
    <maml:name>ParameterName2</maml:name>
    <command:parameterValue required="true">
      int32[]
    </command:parameterValue>
  </command:parameter>
</command:syntaxItem>
```

Last updated on 07/07/2026

How to add parameter information

> ⓘ Note
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

This section describes how to add content that's displayed in the **PARAMETERS** section of the cmdlet Help topic. The **PARAMETERS** section of the Help topic lists each of the parameters of the cmdlet and provides a detailed description of each parameter.

The content of the **PARAMETERS** section should be consistent with the content of the **SYNTAX** section of the Help topic. It's the responsibility of the Help author to make sure that both the **Syntax** and **Parameters** node contain similar XML elements.

> ⓘ Note
>
> For a complete view of a Help file, open one of the `dll-Help.xml` files located in the PowerShell installation directory. For example, the `Microsoft.PowerShell.Commands.Management.dll-Help.xml` file contains content for several of the PowerShell cmdlets.

To add parameters

1. Open the cmdlet Help file and locate the **Command** node for the cmdlet you are documenting. If you are adding a new cmdlet you will need to create a new **Command** node. Your Help file will contain a **Command** node for each cmdlet that you are providing Help content for. Here is an example of a blank **Command** node.

   XML

   ```
   <command:command>
   </command:command>
   ```

2. Within the **Command** node, locate the **Description** node and add a **Parameters** node as shown below. Only one **Parameters** node is allowed, and it should immediately follow the **Syntax** node.

```
XML
<command:command>
  <command:details></command:details>
  <maml:description></maml:description>
  <command:syntax></command:syntax>
  <command:parameters>
  </command:parameters>
</command:command>
```

3. Within the Parameters node, add a **Parameter** node for each parameter of the cmdlet as shown below.

   In this example, a **Parameter** node is added for three parameters.

```
XML
<command:parameters>
  <command:parameter></command:parameter>
  <command:parameter></command:parameter>
  <command:parameter></command:parameter>
</command:parameters>
```

   Because these are the same XML tags that are used in the **Syntax** node, and because the parameters specified here must match the parameters specified by the **Syntax** node, you can copy the **Parameter** nodes from the **Syntax** node and paste them into the **Parameters** node. However, be sure to copy only one instance of a **Parameter** node, even if the parameter is specified in multiple parameter sets in the syntax.

4. For each Parameter node, set the attribute values that define the characteristics of each parameter. These attributes include the following: required, **globbing**, **pipelineinput**, and **position**.

```
XML
<command:parameters>
  <command:parameter required="true" globbing="true"
           pipelineInput="false" position="named">
  </command:parameter>
  <command:parameter required="false" globbing="false"
           pipelineInput="false" position="named">
  </command:parameter>
```

```
  <command:parameter required="false" globbing="false"
           pipelineInput="false" position="named" ></command:parameter>
</command:parameters>
```

5. For each **Parameter** node, add the name of the parameter. Here is an example of the parameter name added to the **Parameter** node.

XML

```
<command:parameters>
  <command:parameter required="true" globbing="true"
           pipelineInput="false" position="named">
    <maml:name> Add parameter name...  </maml:name>
  </command:parameter>
</command:parameters>
```

6. For each **Parameter** node, add the description of the parameter. Here is an example of the parameter description added to the **Parameter** node.

XML

```
<command:parameters>
  <command:parameter required="true" globbing="true"
           pipelineInput="false" position="named">
    <maml:name> Add parameter name...  </maml:name>
    <maml:description>
      <maml:para> Add parameter description... </maml:para>
    </maml:description>
  </command:parameter>
</command:parameters>
```

7. For each **Parameter** node, add the .NET type of the parameter. The parameter type is displayed along with the parameter name.

   Here is an example of the parameter .NET type added to the **Parameter** node.

XML

```
<command:parameters>
  <command:parameter required="true" globbing="true"
           pipelineInput="false" position="named">
    <maml:name> Add parameter name...  </maml:name>
    <maml:description>
      <maml:para> Add parameter description... </maml:para>
    </maml:description>
    <dev:type> Add .NET Framework type... </dev:type>
  </command:parameter>
</command:parameters>
```

8. For each **Parameter** node, add the default value of the parameter. The following sentence is added to the parameter description when the content is displayed: **DefaultValue** is the default.

   Here is an example of the parameter default value is added to the **Parameter** node.

   XML

   ```
   <command:parameters>
     <command:parameter required="true" globbing="true"
              pipelineInput="false" position="named">
       <maml:name> Add parameter name...  </maml:name>
       <maml:description>
         <maml:para> Add parameter description... </maml:para>
       </maml:description>
       <dev:type> Add .NET Framework type... </dev:type>
       <dev:defaultvalue> Add default value...</dev:defaultvalue>
     </command:parameter>
   </command:parameters>
   ```

9. For each Parameter that has multiple values, add a **possibleValues** node.

   Here is an example of the of a **possibleValues** node that defines two possible values for the parameter

   XML

   ```
   <dev:possibleValues>
     <dev:possibleValue>
       <dev:value>Unknown</dev:value>
       <maml:description>
         <maml:para></maml:para>
       </maml:description>
     </dev:possibleValue>
     <dev:possibleValue>
       <dev:value>String</dev:value>
       <maml:description>
         <maml:para></maml:para>
       </maml:description>
     </dev:possibleValue>
   </dev:possibleValues>
   ```

Here are some things to remember when adding parameters.

- The attributes of the parameter aren't displayed in all views of the cmdlet Help topic. However, they're displayed in a table following the parameter description when the user

asks for the **Full** (`Get-Help <cmdletname> -Full`) or **Parameter** (`Get-Help <cmdletname> -Parameter`) view of the topic.

- The parameter description is one of the most important parts of a cmdlet Help topic. The description should be brief, as well as thorough. Also, remember that if the parameter description becomes too long, such as when two parameters interact with each other, you can add more content in the **NOTES** section of the cmdlet Help topic.

  The parameter description provides two types of information.

- What the cmdlet does when the parameter is used.
- What a legal value is for the parameter.
- Because the parameter values are expressed as .NET objects, users need more information about these values than they would in a traditional command-line Help. Tell the user what type of data the parameter is designed to accept, and include examples.

The default value of the parameter is the value that's used if the parameter isn't specified on the command line. Note that the default value is optional, and isn't needed for some parameters, such as required parameters. However, you should specify a default value for most optional parameters.

The default value helps the user to understand the effect of not using the parameter. Describe the default value very specifically, such as the "Current directory" or the "PowerShell installation directory (`$PSHOME`)" for an optional path. You can also write a sentence that describes the default, such as the following sentence used for the **PassThru** parameter: "If PassThru isn't specified, the cmdlet doesn't pass objects down the pipeline." Also, because the value is displayed opposite the field name **Default value**, you don't need to include the term "default value" in the entry.

The default value of the parameter isn't displayed in all views of the cmdlet Help topic. However, it's displayed in a table (along with the parameter attributes) following the parameter description when the user asks for the **Full** (`Get-Help <cmdletname> -Full`) or **Parameter** (`Get-Help <cmdletname> -Parameter`) view of the topic.

The following XML shows a pair of `<dev:defaultValue>` tags added to the `<command:parameter>` node. Notice that the default value follows immediately after the closing `</command:parameterValue>` tag (when the parameter value is specified) or the closing `</maml:description>` tag of the parameter description. name.

XML

```
<command:parameters>
  <command:parameter required="true" globbing="true"
           pipelineInput="false" position="named">
    <maml:name> Parameter name </maml:name>
    <maml:description>
      <maml:para> Parameter Description </maml:para>
    </maml:description>
    <command:parameterValue required="true">
      Value
    </command:parameterValue>
    <dev:defaultValue> Default parameter value </dev:defaultValue>
  </command:parameter>
</command:parameters>
```

Add Values for Enumerated Types

If the parameter has multiple values or values of an enumerated type, you can use an optional `<dev:possibleValues>` node. This node allows you to specify a name and description for multiple values.

Be aware that the descriptions of the enumerated values don't appear in any of the default Help views displayed by the `Get-Help` cmdlet, but other Help viewers may display this content in their views.

The following XML shows a `<dev:possibleValues>` node with two values specified.

XML

```
<command:parameters>
  <command:parameter required="true" globbing="true"
           pipelineInput="false" position="named">
    <maml:name> Parameter name </maml:name>
    <maml:description>
      <maml:para> Parameter Description </maml:para>
    </maml:description>
    <command:parameterValue required="true">
      Value
    </command:parameterValue>
    <dev:defaultValue> Default parameter value </dev:defaultValue>
    <dev:possibleValues>
      <dev:possibleValue>
        <dev:value> Value 1 </dev:value>
        <maml:description>
          <maml:para> Description 1 </maml:para>
        </maml:description>
      <dev:possibleValue>
      <dev:possibleValue>
        <dev:value> Value 2 </dev:value>
```

```
        <maml:description>
          <maml:para> Description 2 </maml:para>
        </maml:description>
      <dev:possibleValue>
    </dev:possibleValues>
  </command:parameter>
</command:parameters>
```

Last updated on 05/20/2025

How to add input types to a cmdlet help topic

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

This section describes how to add an **INPUTS** section to a PowerShell cmdlet Help topic. The **INPUTS** section lists the .NET classes of objects that the cmdlet accepts as input from the pipeline, either by value or by property name.

There is no limit to the number of classes that you can add to an **INPUTS** section. The input types are enclosed in a `<command:inputTypes>` node, with each class enclosed in a `<command:inputType>` element.

The schema includes two `<maml:description>` elements in each `<command:inputType>` element. However, the `Get-Help` cmdlet displays only the content of the `<command:inputType>/<maml:description>` element.

Beginning in PowerShell 3.0, the `Get-Help` cmdlet displays the content of the `<maml:uri>` element. This element lets you direct users to topics that describe the .NET class.

The following XML shows the `<maml:inputTypes>` node.

XML

```
<command:inputTypes>
  <command:inputType>
    <dev:type>
      <maml:name> Class name </maml:name>
      <maml:uri>  URI of a topic that describes the class </maml:uri>
      <maml:description/>
    </dev:type>
    <maml:description>
      <maml:para> Brief description </maml:para>
    </maml:description>
```

```
  </command:inputType>
</command:inputTypes>
```

The following XML shows an example of using the `<maml:inputTypes>` node to document an input type.

XML

```
<command:inputTypes>
  <command:inputType>
    <dev:type>
      <maml:name>System.DateTime</maml:name>
      <maml:uri>https://learn.microsoft.com/dotnet/api/system.datetime</maml:uri>
      <maml:description/>
    </dev:type>
    <maml:description>
      <maml:para> You can pipe a date to the Set-Date cmdlet. <maml:para>
    <maml:description>
  </command:inputType>
</command:inputTypes>
```

Last updated on 05/20/2025

How to add return values to a cmdlet help topic

This section describes how to add an OUTPUTS section to a PowerShell cmdlet Help topic. The **OUTPUTS** section lists the .NET classes of objects that the cmdlet returns or passes down the pipeline.

There is no limit to the number of classes that you can add to the **OUTPUTS** section. The return types of a cmdlet are enclosed in a `<command:returnValues>` node, with each class enclosed in a `<command:returnValue>` element.

If a cmdlet doesn't generate any output, use this section to indicate that there is no output. For example, in place of the class name, write **None** and provide a brief explanation. If the cmdlet generates output conditionally, use this node to explain the conditions and describe the conditional output.

The schema includes two `<maml:description>` elements in each `<command:returnValue>` element. However, the `Get-Help` cmdlet displays only the content of the `<command:returnValue>/<maml:description>` element.

Beginning in PowerShell 3.0, the `Get-Help` cmdlet displays the content of the `<maml:uri>` element. This element lets you direct users to topics that describe the .NET class.

The following XML shows the `<maml:returnValues>` node.

XML

```
<command:returnValues>
  <command:returnValue>
    <dev:type>
      <maml:name> Class Name </maml:name>
      <maml:uri>  URI of a topic that describes the class </maml:uri>
      <maml:description/>
    </dev:type>
    <maml:description>
       <maml:para> Brief description <maml:para>

</maml:description>
  </command: returnValue>
</command: returnValues>
```

The following XML shows an example of using the `<maml:returnValues>` node to document an output type.

XML

```
<command:returnValues>
  <command:returnValue>
    <dev:type>
      <maml:name> System.DateTime </maml:name>
      <maml:uri>  https://learn.microsoft.com/dotnet/api/system.datetime
</maml:uri>
      <maml:description/>
    </dev:type>
    <maml:description>
      <maml:para> Get-Date returns a DateTime object. <maml:para>
    </maml:description>
  </command: returnValue>
</command: returnValues>
```

Last updated on 05/20/2025

How to add notes to a cmdlet help topic

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

This section describes how to add a **NOTES** section to a PowerShell cmdlet Help topic. The **NOTES** section is used to explain details that don't fit easily into the other structured sections, such as a more detailed explanation of a parameter. This content could include comments on how the cmdlet works with a specific provider, some unique, yet important, uses of the cmdlet, or ways to avoid possible error conditions.

The **NOTES** section is defined using a single `<maml:alertset>` node. There are no limits to the number of notes that you can add to a Notes section. For each note, add a pair of `<maml:alert>` tags to the `<maml:alertset>` node. The content of each note is added within a set of `<maml:para>` tags. Use blank `<maml:para>` tags for spacing.

XML

```xml
<maml:alertSet>
  <maml:title>Optional title for Note</maml:title>
  <maml:alert>
    <maml:para>Note 1</maml:para>
    <maml:para>Note a</maml:para>
  </maml:alert>
  <maml:alert>
    <maml:para>Note 2</maml:para>
  </maml:alert>
</maml:alertSet>
```

Last updated on 05/20/2025

How to add examples to a cmdlet help topic

> ⓘ Note
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

Things to know about examples in cmdlet help

- List all of the parameter names in the command, even when the parameter names are optional. This helps the user to interpret the command easily.
- Avoid aliases and partial parameter names, even though they work in PowerShell.
- In the example description, explain the rational for the construction of the command. Explain why you chose particular parameters and values, and how you use variables.
- If the command uses expressions, explain them in detail.
- If the command uses properties and methods of objects, especially properties that don't appear in the default display, use the example as an opportunity tell the user about the object.

Help Views that Display Examples

Examples appear only in the Detailed and Full views of cmdlet Help.

Adding an examples node

The following XML shows how to add an **Examples** node that contains a single **Example** node. Add additional example nodes for each examples you want to include in the topic.

XML

```
<command:examples>
  <command:example>
  </command:example>
</command:examples>
```

Adding an example title

The following XML shows how to add a **title** for the example. The **title** is used to set the example apart from other examples. PowerShell uses a standard header that includes a sequential example number.

XML

```
<command:examples>
  <command:example>
    <maml:title>----------  EXAMPLE 1  ----------</maml:title>
  </command:example>
</command:examples>
```

Adding preceding characters

The following XML shows how to add characters, such as the Windows PowerShell prompt, that are displayed immediately before the example command (without any intervening spaces). PowerShell uses the Windows PowerShell prompt: `C:\PS>`.

XML

```
<command:examples>
  <command:example>
    <maml:title>----------  EXAMPLE 1  ----------</maml:title>
    <maml:introduction>
      <maml:para>C:\PS></maml:para>
    </maml:introduction>
</command:example>
</command:examples>
```

Adding the command

The following XML shows how to add the actual command of the example. When adding the command, type the entire name (do not use alias) of cmdlets and parameters. Also, use lowercase characters whenever possible.

XML

```
<command:examples>
  <command:example>
    <maml:title>----------  EXAMPLE 1  ----------</maml:title>
    <maml:introduction>
      <maml:para>C:\PS></maml:para>
    </maml:introduction>
    <dev:code> command </dev:code>
</command:example>
</command:examples>
```

Adding a Description

The following XML shows how to add a description for the example. PowerShell uses a single set of `<maml:para>` tags for the description, even though multiple `<maml:para>` tags can be used.

XML

```
<command:examples>
  <command:example>
    <maml:title>----------  EXAMPLE 1  ----------</maml:title>
    <maml:introduction>
      <maml:para>C:\PS></maml:para>
    </maml:introduction>
    <dev:code> command </dev:code>
    <dev:remarks>
      <maml:para> command description </maml:para>
    </dev:remarks>
</command:example>
</command:examples>
```

Adding example output

The following XML shows how to add the output of the command. The command results information is optional, but in some cases it's helpful to demonstrate the effect of using specific parameters. PowerShell uses two sets of blank `<maml:para>` tags to separate the command output from the command.

XML

```
<command:examples>
  <command:example>
    <maml:title>----------  EXAMPLE 1  ----------</maml:title>
    <maml:introduction>
      <maml:para>C:\PS></maml:para>
    </maml:introduction>
```

```
    <dev:code> command </dev:code>
    <dev:remarks>
      <maml:para> command description </maml:para>
      <maml:para></maml:para>
      <maml:para></maml:para>
      <maml:para> command output </maml:para>
</dev:remarks>
</command:example>
</command:examples>
```

Last updated on 05/20/2025

How to add related links to a cmdlet help topic

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

This section describes how to add references to other content that's related to a PowerShell cmdlet Help topic. Because these references appear in a command window, they don't link directly to the referenced content.

In the cmdlet Help topics that are included in PowerShell, these links reference other cmdlets, conceptual content (`about_`), and other documents and Help files that aren't related to PowerShell.

The following XML shows how to add a **RelatedLinks** node that contains two references to related topics.

XML

```
<maml:relatedLinks>
  <maml:navigationLink>
    <maml:linkText>Topic-name</maml:linkText>
  </maml:navigationLink>
  <maml:navigationLink>
    <maml:linkText>Topic-name</maml:linkText>
  </maml:navigationLink>
</ maml:relatedLinks >
```

Last updated on 05/20/2025

Writing Help for PowerShell Modules

09/23/2025

PowerShell modules can include Help topics about the module and about the module members, such as cmdlets, providers, functions and scripts. The `Get-Help` cmdlet displays the module Help topics in the same format as it displays Help for other PowerShell items, and users use standard `Get-Help` commands to get the Help topics.

This document explains the format and correct placement of module Help topics, and it suggests guidelines for module Help content.

Types of Module Help

A module can include the following types of Help.

- XML-based help
  - **Cmdlet Help**. The Help topics that describe cmdlets in a module are XML files that use the command help schema
  - **Provider Help**. The Help topics that describe providers in a module are XML files that use the provider help schema.
  - **Function Help**. The Help topics that describe functions in a module can be XML files that use the command help schema or comment-based Help topics within the function, or the script or script module
  - **Script Help**. The Help topics that describe scripts in a module can be XML files that use the command help schema or comment-based Help topics in the script or script module.
  - The `$PSHOME\Schemas\PSMaml` folder contains the schema files that define the XML format.

- Conceptual ("About") help text files

  You can use a conceptual ("about") Help topic to describe the module and its members and to explain how the members can be used together to perform tasks. By default, PowerShell includes over 100 of these conceptual About Help topics.

  - Conceptual Help topics are text files with UTF-8 with BOM encoding.

  - The filename must use the `about_<name>.help.txt` format, such as `about_MyModule.help.txt`.

  - For best display results, you should limit the length of each line to 80 characters.

- You can use the following sample template as a starting point for writing conceptual Help topics.

  > ⓘ **Note**
  >
  > The `TOPIC` section header must start in the first column of the first line of the file. The section content on the second line should match the filename, without the `.help.txt` suffix. You must indent the content exactly 5 spaces. The third line must be blank. The `SHORT DESCRIPTION` section header must start in the first column of the fourth line. You must indent the content on the fifth line exactly 5 spaces. These requirements are necessary for the `Get-Help` cmdlet to recognize the content correctly.

  ```
  TOPIC
      about_<subject or module name>

  SHORT DESCRIPTION
      A short, one-line description of the topic contents.

  LONG DESCRIPTION
      A detailed, full description of the subject or purpose of the module.

  EXAMPLES
      Examples of how to use the module or how the subject feature works in
      practice.

  KEYWORDS
      Terms or titles on which you might expect your users to search for the
      information in this topic.

  SEE ALSO
      Text-only references for further reading. Hyperlinks can't work in the
      PowerShell console.
  ```

  Except for the first two sections, the structure of conceptual Help topics is arbitrary. The remaining section titles can be whatever is appropriate for your content. By convention, you should use the same capitalization, indentation, and blank line separations.

Placement of Module Help

The `Get-Help` cmdlet looks for module Help topic files in language-specific subdirectories of the module directory.

For example, the following directory structure diagram shows the location of the Help topics for the SampleModule module.

```
<ModulePath>
    \SampleModule
        \<en-US>
            \about_SampleModule.help.txt
            \SampleModule.dll-help.xml
            \SampleNestedModule.dll-help.xml
        \<fr-FR>
            \about_SampleModule.help.txt
            \SampleModule.dll-help.xml
            \SampleNestedModule.dll-help.xml
```

> **Note**
>
> In the example, the `<ModulePath>` placeholder represents one of the paths in the `PSModulePath` environment variable, such as `$HOME\Documents\Modules`, `$PSHOME\Modules`, or a custom path that the user specifies.

Getting Module Help

When a user imports a module into a session, the Help topics for that module are imported into the session along with the module. You can list the Help topic files in the value of the FileList key in the module manifest, but Help topics aren't affected by the `Export-ModuleMember` cmdlet.

You can provide module Help topics in different languages. The `Get-Help` cmdlet automatically displays module Help topics in the language that's specified for the current user in the Regional and Language Options item in Control Panel. In Windows Vista and later versions of Windows, `Get-Help` searches for the Help topics in language-specific subdirectories of the module directory in accordance with the language fallback standards established for Windows.

Beginning in PowerShell 3.0, running a `Get-Help` command for a cmdlet or function triggers automatic importing of the module. The `Get-Help` cmdlet immediately displays the contents of the help topics in the module.

If the module doesn't contain help topics and there are no help topics for the commands in the module on the user's computer, `Get-Help` displays auto-generated help. The auto-generated help includes the command syntax, parameters, and input and output types, but doesn't

include any descriptions. The auto-generated help includes text that directs the user to try to use the `Update-Help` cmdlet to download help for the command from the internet or a file share. It also recommends using the **Online** parameter of the `Get-Help` cmdlet to get the online version of the help topic.

Supporting Updatable Help

Users of PowerShell 3.0 and later versions of PowerShell can download and install updated help files for a module from the internet or from a local file share. The `Update-Help` and `Save-Help` cmdlets hide the management details from the user. Users run the `Update-Help` cmdlet and then use the `Get-Help` cmdlet to read the newest help files for the module at the PowerShell command prompt. Users don't need to restart Windows or PowerShell.

Users behind firewalls and those without internet access can use Updatable Help, as well. Administrators with internet access use the `Save-Help` cmdlet to download and install the newest help files to a file share. Then, users use the **Path** parameter of the `Update-Help` cmdlet to get the newest help files from the file share.

Module authors can include help files in the module and use Updatable Help to update the help files, or omit help files from the module and use Updatable Help both to install and to update them.

For more information about Updatable Help, see Supporting Updatable Help.

Supporting Online Help

Users who can't or don't install updated help files on their computers often rely on the online version of module help topics. The **Online** parameter of the `Get-Help` cmdlet opens the online version of a cmdlet or advanced function help topic for the user in their default internet browser.

The `Get-Help` cmdlet uses the value of the **HelpUri** property of the cmdlet or function to find the online version of the help topic.

Beginning in PowerShell 3.0, you can help users find the online version of cmdlet and function help topics by defining the HelpUri attribute on the cmdlet class or the **HelpUri** property of the **CmdletBinding** attribute. The value of the attribute is the value of the **HelpUri** property of the cmdlet or function.

For more information, see Supporting Online Help.

See Also

- Writing a PowerShell Module
- Supporting Updatable Help
- Supporting Online Help

Naming Help files

09/23/2025

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

This topic explains how to name an XML-based help file so that the Get-Help cmdlet can find it. The name requirements differ for each command type.

Cmdlet Help files

The help file for a C# cmdlet must be named for the assembly in which the cmdlet is defined. Use the following filename format:

```
<AssemblyName>.dll-help.xml
```

The assembly name format is required even when the assembly is a nested module.

For example, the Get-WinEvent cmdlet is defined in the Microsoft.PowerShell.Diagnostics.dll assembly. The `Get-Help` cmdlet looks for a help topic for the `Get-WinEvent` cmdlet only in the `Microsoft.PowerShell.Diagnostics.dll-help.xml` file in the module directory.

Provider Help files

The help file for a PowerShell provider must be named for the assembly in which the provider is defined. Use the following filename format:

`<AssemblyName>.dll-help.xml`

The assembly name format is required even when the assembly is a nested module.

For example, the Certificate provider is defined in the `Microsoft.PowerShell.Security.dll` assembly. The `Get-Help` cmdlet looks for a help topic for the Certificate provider only in the `Microsoft.PowerShell.Security.dll-help.xml` file in the module directory.

Function Help files

Functions can be documented using comment-based help or documented in an XML help file. When the function is documented in an XML file, the function must have an `.EXTERNALHELP` comment keyword that associates the function with the XML file. Otherwise, the `Get-Help` cmdlet can't find the help file.

There are no technical requirements for the name of a function help file. However, a best practice is to name the help file for the script module in which the function is defined. For example, the following function is defined in the `MyModule.psm1` file.

PowerShell

```
#.EXTERNALHELP MyModule.psm1-help.xml
function Test-Function { ... }
```

Beginning in PowerShell 5.0, functions that are exported by a module can be documented in a help file that is named for the module. You don't need to use `.EXTERNALHELP` comment keyword. For example, if the `Test-Function` function is exported by the `MyModule` module, you can name the help file `MyModule-help.xml`. The `Get-Help` cmdlet looks for help for the `Test-Function` function in the `MyModule-help.xml` file in the module directory.

CIM Command Help files

The help file for a CIM command must be named for the CDXML file in which the CIM command is defined. Use the following filename format:

`<FileName>.cdxml-help.xml`

CIM commands are defined in CDXML files that can be included in modules as nested modules. When the CIM command is imported into the session as a function, PowerShell adds an `.EXTERNALHELP` comment keyword to the function definition that associates the function with an XML help file that is named for the CDXML file in which the CIM command is defined.

Script Workflow Help files

Script workflows that are included in modules can be documented in XML-based help files. There are no technical requirements for the name of the help file. However, a best practice is to name the help file for the script module in which the script workflow is defined. For example:

`<ScriptModule>.psm1-help.xml`

Unlike other scripted commands, script workflows don't require an `.EXTERNALHELP` comment keyword to associate them with a help file. Instead, PowerShell searches the UI-Culture-specific subdirectories of the module directory for XML-based help files and looks for help for the script workflow in all the files. `.EXTERNALHELP` comment keyword are ignored.

Because the `.EXTERNALHELP` comment keyword is ignored, the `Get-Help` cmdlet can find help for script workflows only when they're included in modules.

Supporting Updatable Help

The *Windows PowerShell Updatable Help System*, introduced in Windows PowerShell 3.0, is designed to ensure that users always have the newest help topics at the command prompt on their local computer. Along with Windows PowerShell online help, Updatable Help provides a complete help solution for users. This section describes the Updatable Help System and explains how module authors can support Updatable Help for their modules.

This section includes the following topics.

- Updatable Help Overview
- Updatable Help Authoring: Step-by-Step
- How Updatable Help Works
- How to Create a HelpInfo XML File
- How to Prepare Updatable Help CAB Files
- How to Update Help Files
- How to Test Updatable Help

See Also

- Supporting Online Help
- Updatable Help Status Table

Last updated on 05/20/2025

Updatable Help Overview

This document provides a basic introduction to the design and operation of the PowerShell Updatable Help feature. It's designed for module authors and others who deliver Windows PowerShell help topics to users.

Introduction

PowerShell help topics are an integral part of the PowerShell experience. Like PowerShell modules, help topics are continually updated and improved by the authors and by the contributions of the community of PowerShell users.

The *Updatable Help* feature, introduced in Windows PowerShell 3.0, ensures that users have the newest versions of help topics at the command prompt, even for built-in PowerShell commands, without downloading new modules or running Windows Update. Updatable Help makes updating simple by providing cmdlets that download the newest versions of help topics from the internet and install them in the correct subdirectories on the user's local computer. Even users who are behind firewalls can use the new cmdlets to get updated help from an internal file share.

Updatable Help is fully supported by all Windows PowerShell modules in Windows 8 and Windows Server 2012, and its features are available to all Windows PowerShell module authors. Updatable Help supports only XML-based help files. It doesn't support comment-based help.

Updatable Help includes the following features.

- The Update-Help cmdlet, which determines whether users have the newest help files for a module and, if not, downloads the newest help files from the internet, unpacks them, and installs them in the correct module subdirectories on the user's computer. Users can use the Get-Help cmdlet to view the newly-installed help topics immediately. They don't need to restart PowerShell.

- The Save-Help cmdlet, which downloads the newest help files from the internet and saves them in a file system directory. Users can use the `Update-Help` cmdlet to get help files from the file system directory, and unpack and install them in the module subdirectories on the user's computer. The `Save-Help` cmdlet is designed for users who have limited or no internet access and for enterprises who prefer to limit internet access.

- **Help for a Module**. Help files for a module are managed and delivered as a unit, so users can get all of the help files for the modules they use. Updatable help is supported only for modules, not for Windows PowerShell snap-ins.

- **Version support**. Updatable Help uses standard four-position (N1.N2.N3.N4) version numbers. Updatable Help downloads help files when the version number of the help files on the user's computer (or in the `Save-Help` directory) is lower than the version number of the help files at the internet location.

- **Multi-language support**. Updatable Help supports module help files in multiple UI cultures. Updatable Help filenames include standard language codes, such as "en-US" and "ja-JP", and the `Update-Help` and `Save-Help` cmdlets place the help files into language-specific subdirectories of the module directory.

- **Auto-generated help**. The Get-Help cmdlet displays basic help for commands that don't have help files. The auto-generated help includes the command syntax and aliases, and instructions for using online help and Updatable Help.

- **Enhanced Online help**. Easy access to online help no longer requires help files. The **Online** parameter of the `Get-Help` cmdlet now gets the URL of an online help topic from the value of the **HelpUri** property of any command, if it can't find the online help URL in a help file. You can populate the **HelpUri** property by adding a **HelpUri** attribute to the code of cmdlets, functions, and CIM commands, or using the `.LINK` comment-based help keyword in workflows and scripts.

  To make our help files updatable, the Windows PowerShell modules in Windows don't come with help files. Users can use Updatable Help to install help files and update them. Authors of other modules can include help files in modules or omit them. Support for Updatable Help is optional, but recommended.

Last updated on 05/20/2025

Updatable Help Authoring: Step-by-Step

This article documents the steps required to publish Updatable Help.

Authoring Updatable Help: Step-by-Step

Updatable Help is designed for end-users, but it also provides significant benefits to module authors and help writers, including the ability to add content, fix errors, deliver in multiple UI cultures, and respond to user comments and requests, long after the module has shipped. This topic explains how you package and upload help files so that users can download and install them using the Update-Help and Save-Help cmdlets.

The following steps provide an overview of the process of supporting Updatable Help.

Step 1: Find an internet site for your help files

The first step in creating updatable help is to find an internet location for your module's help files. Actually, you can use two different locations. You can keep your module's help information file (HelpInfo XML - described below) at one internet location and the help content files (CAB and ZIP) at another internet location. All help content files for a module must be in the same location. You can place help content files for different modules in the same location.

Step 2: Add a HelpInfoURI key to your module manifest

Add a **HelpInfoURI** key to your module manifest. The value of the key is the Uniform Resource Identifier (URI) of the location of the HelpInfo XML information file for your module. For security, the address must begin with `http:` or `https:`. The URI should specify an internet location for the HelpInfo XML file. Don't include the HelpInfo XML filename.

For example:

PowerShell

```
@{
    RootModule = TestModule.psm1
    ModuleVersion = '2.0'
    HelpInfoURI = 'https://go.microsoft.com/fwlink/?LinkID=0123'
}
```

> **Note**
>
> The **HelpInfoURI** must end with a forward slash (`/`) character or redirect to a location that ends with a forward slash (`/`).

Step 3: Create a HelpInfo XML file

The HelpInfo XML information file contains the URI of the internet location of your help files and the version numbers of the newest help files for your module in each supported UI culture. Every PowerShell module has one HelpInfo XML file. When you update your help files, you must update the HelpInfo XML file. For more information, see How to Create a HelpInfo XML File.

Step 4: Create CAB and ZIP files

PowerShell on Windows expects the help content files a module to be stored in a CAB file. PowerShell on Linux or macOS expects the help content files a module to be stored in a ZIP file. If your module runs across multiple platforms you must create both formats.

Use a tool, such as `MakeCab.exe`, to create a CAB file that contains the help files for your module. Create a separate CAB file for the help files in each supported UI culture. For more information, see How to Prepare Updatable Help CAB Files.

You can use the Compress-Archive cmdlet to create a ZIP file.

Step 5: Upload your files

To publish new or updated help files, upload the help content files to the internet location specified by the **HelpContentUri** element in the HelpInfo XML file. Then, upload the HelpInfo XML file to the internet location specified by the value of the **HelpInfoUri** key in the module manifest.

Using PlatyPS to create help content

PlatyPS is a PowerShell module designed to help you create Help content for your modules. You author the help content in Markdown files. **PlatyPS** can create Markdown templates for your cmdlet, convert the Markdown files to the XML help format (MAML), create HelpInfo XML files, and package the MAML help content into CAB and ZIP files.

For more information, see Create XML-based help using PlatyPS.

Last updated on 05/20/2025

How Updatable Help Works

This topic explains how Updatable Help processes the HelpInfo XML file and CAB files for each module, and installs updated help for users.

The Update-Help Process

The following list describes the actions of the Update-Help cmdlet when a user runs a command to update the help files for a module in a particular UI culture.

1. `Update-Help` gets the remote HelpInfo XML file from the location specified by the value of the **HelpInfoURI** key in the module manifest and validates the file against the schema. (To view the schema, see HelpInfo XML Schema.) Then `Update-Help` looks for a local HelpInfo XML file for the module in the module directory on the user's computer.

2. `Update-Help` compares the version number of the help files for the specified UI culture in the remote and local HelpInfo XML files for the module. If the version number on the remote file is greater than version number on the local file, or if there is no local HelpInfo XML file for the module, `Update-Help` prepares to download new help files.

3. `Update-Help` selects the CAB file for the module from the location specified by the **HelpContentUri** element in the remote HelpInfo XML file. It uses the module name, module GUID, and UI culture to identify the CAB file.

4. `Update-Help` downloads the CAB file, unpacks it, validates the help content files, and saves the help content files in the language-specific subdirectory of the module directory on the user's computer.

5. `Update-Help` creates a local HelpInfo XML file by copying the remote HelpInfo XML file. It edits the local HelpInfo XML file so that it includes elements only for the CAB file that it installed. Then it saves the local HelpInfo XML file in the module directory and concludes the update.

The Save-Help Process

The following list describes the actions of the Save-Help and Update-Help cmdlets when a user runs commands to update the help files in a file share, and then use those files to update the help files on the user's computer.

The `Save-Help` cmdlet performs the following actions in response to a command to save the help files for a module in a file share that's specified by the **DestinationPath** parameter.

1. `Save-Help` gets the remote HelpInfo XML file from the location specified by the value of the **HelpInfoURI** key in the module manifest and validates the file against the schema. (To view the schema, see HelpInfo XML Schema.) Then `Save-Help` looks for a local HelpInfo XML file in the directory that's specified by the **DestinationPath** parameter in the `Save-Help` command.

2. `Save-Help` compares the version number of the help files for the specified UI culture in the remote and local HelpInfo XML files for the module. If the version number on the remote file is greater than version number on the local file, or if there is no local HelpInfo XML file for the module in the **DestinationPath** directory, `Save-Help` prepares to download new help files.

3. `Save-Help` selects the CAB file for the module from the location specified by the **HelpContentUri** element in the remote HelpInfo XML file. It uses the module name, module GUID, and UI culture to identify the CAB file.

4. `Save-Help` downloads the CAB file and saves it in the **DestinationPath** directory. (It does not create any language-specific subdirectories.)

5. `Save-Help` creates a local HelpInfo XML file by copying the remote HelpInfo XML file. It edits the local HelpInfo XML file so that it includes elements only for the CAB file that it saved. Then it saves the local HelpInfo XML file in the **DestinationPath** directory and concludes the update.

   The `Update-Help` cmdlet performs the following actions in response to a command to update the help files on a user's computer from the files in a file share that's specified by the **SourcePath** parameter.

6. `Update-Help` gets the remote HelpInfo XML file from the **SourcePath** directory. Then it looks for a local HelpInfo XML file in the module directory on the user's computer.

7. `Update-Help` compares the version number of the help files for the specified UI culture in the remote and local HelpInfo XML files for the module. If the version number on the remote file is greater than version number on the local file, or if there is no local HelpInfo XML file, `Update-Help` prepares to install new help files.

8. `Update-Help` selects the CAB file for the module from **SourcePath** directory. It uses the module name, module GUID, and UI culture to identify the CAB file.

9. `Update-Help` unpacks the CAB file, validates the help content files, and saves the help content files in the language-specific subdirectory of the module directory on the user's computer.

10. `Update-Help` creates a local HelpInfo XML file by copying the remote HelpInfo XML file. It edits the local HelpInfo XML file so that it includes elements only for the CAB file that it installed. Then it saves the local HelpInfo XML file in the module directory and concludes the update.

Last updated on 02/19/2026

How to create a HelpInfo XML file

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

This topics in this section explains how to create and populate a help information file, commonly known as a "HelpInfo XML file," for the PowerShell Updatable Help feature.

HelpInfo XML file overview

The HelpInfo XML file is the primary source of information about Updatable Help for the module. It includes the location of the help files for the modules, the supported UI cultures, and the version numbers that Updatable Help uses to determine whether the user has the newest help files.

Each module has just one HelpInfo XML file, even if the module includes multiple help files for multiple UI cultures. The module author creates the HelpInfo XML file and places it in the internet location that's specified by the **HelpInfoUri** key in the module manifest. When the module help files are updated and uploaded, the module author updates the HelpInfo XML file and replaces the original HelpInfo XML file with the new version.

It's critical that the HelpInfo XML file is carefully maintained. If you upload new files, but forget to increment the version numbers, Updatable Help will not download the new files to users' computers. if you add help files for a new UI culture, but don't update the HelpInfo XML file or place it in the correct location, Updatable Help will not download the new files.

In this section

This section includes the following topics.

- HelpInfo XML Schema
- HelpInfo XML Sample File
- How to Name a HelpInfo XML File

- How to Set HelpInfo XML Version Numbers

See also

Supporting Updatable Help

Last updated on 05/20/2025

HelpInfo XML Schema

This topic contains the XML schema for Updatable Help Information files, commonly known as "HelpInfo XML files."

HelpInfo XML Schema

HelpInfo XML files are based on the following XML schema.

XML

```
<?xml version="1.0" encoding="utf-8"?>
<schema elementFormDefault="qualified"
targetNamespace="http://schemas.microsoft.com/powershell/help/2010/05"
xmlns="http://www.w3.org/2001/XMLSchema">
  <element name="HelpInfo">
    <complexType>
      <sequence>
        <element name="HelpContentURI" type="anyURI" minOccurs="1" maxOccurs="1" />
        <element name="SupportedUICultures" minOccurs="1" maxOccurs="1">
          <complexType>
            <sequence>
              <element name="UICulture" minOccurs="1" maxOccurs="unbounded">
                <complexType>
                  <sequence>
                    <element name="UICultureName" type="language" minOccurs="1"
maxOccurs="1" />
                    <element name="UICultureVersion" type="string" minOccurs="1"
maxOccurs="1" />
                  </sequence>
                </complexType>
              </element>
            </sequence>
          </complexType>
        </element>
      </sequence>
    </complexType>
  </element>
</schema>
```

HelpInfo XML Elements

The HelpInfo XML file includes the following elements.

- **HelpContentURI** - Contains the URI of the location of the help CAB files for the module. The URI must begin with "http" or "https". The URI should specify an internet location, but must not include the CAB filename. The **HelpContentURI** value can be the same or different from the **HelpInfoURI** value.
- **SupportedUICultures** - Represents the module help files in all UI cultures. Contains **UICulture** elements, each of which represents a set of help files for the module in a specified UI culture.
- **UICulture** - Represents a set of help files for the module in a specified UI culture. Add a **UICulture** element for each UI culture in which the help files are written.
- **UICultureName** - Contains the language code for the UI culture in which the help files are written.
- **UICultureVersion** - Contains a 4-part version number in "N1.N2.N3.N4" format that represents the version of the help CAB file in the UI culture. Increment this version number whenever you upload new help CAB files in the UI culture that's specified by **UICultureName**. For more information about this value, see Version Class.

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

Last updated on 05/20/2025

HelpInfo XML Sample File

This topic displays a sample of a well-formed Updatable Help Information file, commonly known as "HelpInfo XML file." In this sample file, the UI culture elements are arranged in alphabetical order by UI culture name. Alphabetical ordering is a best practice, but it's not required.

HelpInfo XML Sample File

XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<HelpInfo xmlns="http://schemas.microsoft.com/powershell/help/2010/05">
   <HelpContentURI>https://go.microsoft.com/fwlink/?LinkID=141553</HelpContentURI>
   <SupportedUICultures>
    <UICulture>
      <UICultureName>de-DE</UICultureName>
      <UICultureVersion>2.15.0.10</UICultureVersion>
    </UICulture>
    <UICulture>
      <UICultureName>en-US</UICultureName>
      <UICultureVersion>3.2.0.7</UICultureVersion>
    </UICulture>
    <UICulture>
      <UICultureName>it-IT</UICultureName>
      <UICultureVersion>1.1.0.5</UICultureVersion>
    </UICulture>
    <UICulture>
      <UICultureName>ja-JP</UICultureName>
      <UICultureVersion>3.2.0.4</UICultureVersion>
    </UICulture>
   </SupportedUICultures>
</HelpInfo>
```

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

Last updated on 05/20/2025

How to name a HelpInfo XML file

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

This topic explains the required name format for the Updatable Help Information files, commonly known as HelpInfo XML files. A HelpInfo XML file must have a name with the following format.

```
<ModuleName>_<ModuleGUID>_HelpInfo.xml
```

The elements of the name are as follows.

- `<ModuleName>` - The value of the **Name** property of the **ModuleInfo** object that the Get-Module cmdlet returns.
- `<ModuleGUID>` - The value of the **GUID** key in the module manifest.

For example, if the module name is "TestModule" and the module GUID is 9cabb9ad-f2ac-4914-a46b-bfc1bebf07f9, the name of the HelpInfo XML file for the module would be:

```
TestModule_9cabb9ad-f2ac-4914-a46b-bfc1bebf07f9_HelpInfo.xml
```

Last updated on 05/20/2025

How to set HelpInfo XML version numbers

> ⓘ Note
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

The version numbers in a HelpInfo XML file are critical to the operation of Updatable Help. The Update-Help and Save-Help cmdlets download new help files only when the version number for a UI culture in the remote HelpInfo XML file is greater than the version number for that UI culture in the local HelpInfo XML, or there is no local HelpInfo XML file.

The HelpInfo XML file uses the 4-part version number that's defined in the **System.Version** class of the Microsoft .NET Framework. The format is `N1.N2.N3.N4`. Module authors can use any version numbering scheme that's permitted by the **System.Version** class. Updatable Help requires only that the version number for a UI culture increase when a new version of the CAB file for that UI culture is uploaded to the location that's specified by the **HelpContentURI** element in the HelpInfo XML file.

The following example shows the elements of the HelpInfo XML file for the German (de-DE) UI culture when the version is 2.15.0.10.

XML

```xml
<UICulture>
  <UICultureName>de-DE</UICultureName>
  <UICultureVersion>2.15.0.10</UICultureVersion>
</UICulture>
```

The version number for a UI culture reflects the version of the CAB file for that UI culture. The version number applies to the entire CAB file. You can't set different version numbers for different files in the CAB file. The version number for each UI culture is evaluated independently and need not be related to the version numbers for other UI cultures that the module supports.

Last updated on 05/20/2025

How to prepare Updatable Help CAB files

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

This topic explains the contents and use of cabinet files in Windows PowerShell Updatable Help.

This section includes the following topics.

- How to Create and Upload CAB Files
- How to Name an Updatable Help CAB File
- File Types Permitted in an Updatable Help CAB File

See also

Supporting Updatable Help

Last updated on 05/20/2025

How to create and upload CAB files

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

This topic explains how to create Updatable Help CAB files and upload them to the location where the Updatable Help cmdlets can find them.

How to create and upload updatable help CAB files

You can use the Updatable Help feature to deliver new or updated help files for a module in multiple languages and cultures. An Updatable Help package for a module consists of one HelpInfo XML file and one or more cabinet (`.CAB`) files. Each CAB file contains help files for the module in one UI culture. Use the following procedure to create CAB files for Updatable Help.

1. Organize the help files for the module by UI culture. Each Updatable Help CAB file contains the help files for one module in one UI culture. You can deliver multiple help CAB files for the module, each for a different UI culture.

2. Verify that help files include only the file types permitted for Updatable Help and validate them against a help file schema. If the `Update-Help` cmdlet encounters a file that's invalid or is not a permitted type, it doesn't install the invalid file and stops installing files from the CAB. For a list of permitted file types, see File Types Permitted in an Updatable Help CAB File.

3. Include all the help files for the module in the UI culture, not only files that are new or have changed. If the CAB file is incomplete, users who download help files for the first time or do not download every update, won't have all the help files.

4. Use a utility that creates cabinet files, such as `MakeCab.exe`. PowerShell doesn't include cmdlets that create CAB files.

5. Name the CAB files. For more information, see How to Name an Updatable Help CAB File.

6. Upload the CAB files for the module to the location that's specified by the **HelpContentUri** element in the HelpInfo XML file for the module. Then upload the HelpInfo XML file to the location that's specified by the **HelpInfoUri** key of the module manifest. The **HelpContentUri** and **HelpInfoUri** can point to the same location.

> **⊗ Caution**
>
> The value of the **HelpInfoUri** key and the **HelpContentUri** element must begin with `http` or `https`. The value must a URL path pointing to the location (folder) containing the updateable help. The URL must end with `/`. The URL must not include a filename.

Last updated on 05/20/2025

How to name an Updatable Help CAB file

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

An updatable cabinet (`.CAB`) file must have a name with the following format.

```
<ModuleName>_<ModuleGUID>_<UICulture>_HelpContent.cab
```

The elements of the name are as follows.

- `<ModuleName>` -The value of the **Name** property of the **ModuleInfo** object that the Get-Module cmdlet returns.
- `<ModuleGUID>` - The value of the **GUID** key in the module manifest.
- `<UICulture>` - The UI culture of the help files in the CAB file. This value must match the value of one of the **UICulture** elements in the HelpInfo XML file for the module.

For example, if the module name is "TestModule," the module GUID is 9cabb9ad-f2ac-4914-a46b-bfc1bebf07f9, and the UI culture is `en-US`, the name of the CAB file would be:

```
TestModule_9cabb9ad-f2ac-4914-a46b-bfc1bebf07f9_en-US_HelpContent.cab
```

Last updated on 05/20/2025

File Types Permitted in an Updatable Help CAB File

Uncompressed CAB file content is limited to 1 GB by default. To bypass this limit, users have to use the **Force** parameter of the Update-Help and Save-Help cmdlets.

To assure the security of help files that are downloaded from the internet, an Updatable Help CAB file can include only the file types listed below. The Update-Help cmdlet validates all files against the help topic schemas. If the `Update-Help` cmdlet encounters a file that's invalid or is not a permitted type, it doesn't install the invalid file and stops installing files from the CAB on the user's computer.

- XML-based help topics for cmdlets.
- XML-based help topics for scripts and functions.
- XML-based help topics for PowerShell providers.
- Text-based help topics, such as About topics.

The Update-Help verifies the CAB contents when it unpacks the CAB. If `Update-Help` finds non-compliant file types in an Updatable Help CAB file, it generates a terminating error and stops the operation. It doesn't install any help files from the CAB, even those of compliant file types.

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

Last updated on 05/20/2025

How to update help files

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

There are many reasons to update help files, such as correcting errors, clarifying a concept, answering a frequently asked question, adding new files, or adding new and better examples.

To update a help file:

1. Change the files.
2. Translate the files into other UI cultures.
3. Collect all help files (new, changed, and unchanged) for the module in each UI culture.
4. Validate the files against the XML schema.
5. Rebuild the CAB files for each UI culture.
6. In the HelpInfo XML file, increment the version numbers of the CAB file for each UI culture.
7. Upload the new CAB files to the location that's specified by the value of the **HelpContentUri** element in the HelpInfo XML file. Replace the older CAB files with the new CAB files.
8. Upload the updated HelpInfo XML file to the location that's specified by the **HelpInfoUri** key in the module manifest. Replace the older HelpInfo XML file with the new file.

Last updated on 05/20/2025

How to test Updatable Help

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

This topic describes approaches to testing Updatable Help for a module.

Using verbose to detect errors

After uploading the HelpInfo XML file and CAB files for your module, test the files by running an Update-Help command with the **Verbose** parameter. The **Verbose** parameter directs `Update-Help` to report the critical steps in its actions, from reading the **HelpInfoUri** key in the module manifest to validating the file types in the unpacked CAB file and placing the files in the language-specific module directory.

When all verbose messages are resolved, run an `Update-Help` command with the **Debug** parameter. This parameter should detect any remaining problems with the Updatable Help files.

Last updated on 05/20/2025

Supporting Online Help

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

Beginning in PowerShell 3.0, there are two ways to support the `Get-Help` Online feature for PowerShell commands. This topic explains how to implement this feature for different command types.

About Online Help

Online help has always been a vital part of PowerShell. Although the `Get-Help` cmdlet displays help topics at the command prompt, many users prefer the experience of reading online, including color-coding, hyperlinks, and sharing ideas in Community Content and wiki-based documents. Most importantly, before the advent of Updatable Help, online help provided the most up-to-date version of the help files.

With the advent of Updatable Help in PowerShell 3.0, online help still plays a vital role. In addition to the flexible user experience, online help provides help to users who don't or can't use Updatable Help to download help topics.

How Get-Help -Online Works

To help users find the online help topics for commands, the `Get-Help` command has an Online parameter that opens the online version of help topic for a command in the user's default internet browser.

For example, the following command opens the online help topic for the `Invoke-Command` cmdlet.

PowerShell

```
Get-Help Invoke-Command -Online
```

To implement `Get-Help -Online`, the `Get-Help` cmdlet looks for a Uniform Resource Identifier (URI) for the online version help topic in the following locations.

- The first link in the **Related Links** section of the help topic for the command. The help topic must be installed on the user's computer. This feature was introduced in PowerShell 2.0.

- The **HelpUri** property of any command. The **HelpUri** property is accessible even when the help topic for the command isn't installed on the user's computer. This feature was introduced in PowerShell 3.0.

  `Get-Help` looks for a URI in the first entry in the **Related Links** section before getting the **HelpUri** property value. If the property value is incorrect or has changed, you can override it by entering a different value in the first related link. However, the first related link works only when the help topics are installed on the user's computer.

Adding a URI to the first related link of a command help topic

You can support `Get-Help -Online` for any command by adding a valid URI to the first entry in the **Related Links** section of the XML-based help topic for the command. This option is valid only in XML-based help topics and works only when the help topic is installed on the user's computer. When the help topic is installed and the URI is populated, this value takes precedence over the **HelpUri** property of the command.

To support this feature, the URI must appear in the `maml:uri` element under the first `maml:relatedLinks/maml:navigationLink` element in the `maml:relatedLinks` element.

The following XML shows the correct placement of the URI. The `Online version:` text in the `maml:linkText` element is a best practice, but it's not required.

XML

```
<maml:relatedLinks>
    <maml:navigationLink>
        <maml:linkText>Online version:</maml:linkText>
        <maml:uri>https://go.microsoft.com/fwlink/?LinkID=113279</maml:uri>
    </maml:navigationLink>
    <maml:navigationLink>
        <maml:linkText>about_History</maml:linkText>
        <maml:uri/>
```

```
    </maml:navigationLink>
</maml:relatedLinks>
```

Adding the HelpUri property to a command

This section shows how to add the **HelpUri** property to commands of different types.

Adding a HelpUri Property to a Cmdlet

For cmdlets written in C#, add a **HelpUri** attribute to the **Cmdlet** class. The value of the attribute must be a URI that begins with `http` or `https`.

The following code shows the **HelpUri** attribute of the `Get-History` cmdlet class.

C#

```
[Cmdlet(VerbsCommon.Get, "History", HelpUri = "https://go.microsoft.com/fwlink/?
LinkID=001122")]
```

Adding a HelpUri property to an advanced function

For advanced functions, add a **HelpUri** property to the **CmdletBinding** attribute. The value of the property must be a URI that begins with "http" or "https".

The following code shows the **HelpUri** attribute of the `New-Calendar` function

PowerShell

```
function New-Calendar {
    [CmdletBinding(SupportsShouldProcess=$true,
    HelpUri="https://go.microsoft.com/fwlink/?LinkID=01122")]
```

Adding a HelpUri attribute to a cim command

For CIM commands, add a **HelpUri** attribute to the **CmdletMetadata** element in the CDXML file. The value of the attribute must be a URI that begins with `http` or `https`.

The following code shows the HelpUri attribute of the `Start-Debug` CIM command

XML

```
<CmdletMetadata Verb="Debug" HelpUri="https://go.microsoft.com/fwlink/?
LinkID=001122"/>
```

Adding a HelpUri attribute to a workflow

For workflows that are written in the PowerShell language, add an `.EXTERNALHELP` comment keyword to the workflow code. The value of the keyword must be a URI that begins with `http` or `https`.

> **Note**
>
> The HelpUri property isn't supported for XAML-based workflows in PowerShell.

The following code shows the `.EXTERNALHELP` keyword in a workflow file.

PowerShell

```
# .EXTERNALHELP "https://go.microsoft.com/fwlink/?LinkID=138338"
```

Last updated on 05/20/2025

How to add dynamic parameters to a provider help topic

> **Note**
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

This section explains how to populate the **DYNAMIC PARAMETERS** section of a provider help topic.

Dynamic parameters are parameters of a cmdlet or function that are available only under specified conditions.

The dynamic parameters that are documented in a provider help topic are the dynamic parameters that the provider adds to the cmdlet or function when the cmdlet or function is used in the provider drive.

Dynamic parameters can also be documented in custom cmdlet help for a provider. When writing both provider help and custom cmdlet help for a provider, include the dynamic parameter documentation in both documents.

If a provider doesn't implement any dynamic parameters, the provider help topic contains an empty `DynamicParameters` element.

To add dynamic parameters

1. In the `<AssemblyName>.dll-help.xml` file, within the `providerHelp` element, add a `DynamicParameters` element. The `DynamicParameters` element should appear after the `Tasks` element and before the `RelatedLinks` element.

   For example:

   ```
   XML
   ```

```
<providerHelp>
    <Tasks>
    </Tasks>
    <DynamicParameters>
    </DynamicParameters>
    <RelatedLinks>
    </RelatedLinks>
</providerHelp>
```

If the provider doesn't implement any dynamic parameters, the `DynamicParameters` element can be empty.

2. Within the `DynamicParameters` element, for each dynamic parameter, add a `DynamicParameter` element.

   For example:

   XML

   ```
   <DynamicParameters/>
       <DynamicParameter>
       </DynamicParameter>
   </DynamicParameters>
   ```

3. In each `DynamicParameter` element, add a `Name` and `CmdletSupported` element.

   - Name - Specifies the parameter name
   - CmdletSupported - Specifies the cmdlets in which the parameter is valid. Type a comma-separated list of cmdlet names.

   For example, the following XML documents the `Encoding` dynamic parameter that the Windows PowerShell FileSystem provider adds to the `Add-Content`, `Get-Content`, `Set-Content` cmdlets.

   XML

   ```
   <DynamicParameters/>
       <DynamicParameter>
           <Name> Encoding </Name>
           <CmdletSupported> Add-Content, Get-Content, Set-Content
   </CmdletSupported>
   </DynamicParameters>
   ```

4. In each `DynamicParameter` element, add a `Type` element. The `Type` element is a container for the `Name` element which contains the .NET type of the value of the dynamic parameter.

   For example, the following XML shows that the .NET type of the `Encoding` dynamic parameter is the FileSystemCmdletProviderEncoding enumeration.

   XML

```
<DynamicParameters/>
    <DynamicParameter>
        <Name> Encoding </Name>
        <CmdletSupported> Add-Content, Get-Content, Set-Content
</CmdletSupported>
        <Type>
            <Name>
Microsoft.PowerShell.Commands.FileSystemCmdletProviderEncoding </Name>
        <Type>
...
</DynamicParameters>
```

5. Add the `Description` element, which contains a brief description of the dynamic parameter. When composing the description, use the guidelines prescribed for all cmdlet parameters in How to Add Parameter Information.

   For example, the following XML includes the description of the `Encoding` dynamic parameter.

   XML

```
<DynamicParameters/>
    <DynamicParameter>
        <Name> Encoding </Name>
        <CmdletSupported> Add-Content, Get-Content, Set-Content
</CmdletSupported>
        <Type>
            <Name>
Microsoft.PowerShell.Commands.FileSystemCmdletProviderEncoding </Name>
        <Type>
        <Description> Specifies the encoding of the output file that contains
the content. </Description>
...
</DynamicParameters>
```

6. Add the `PossibleValues` element and its child elements. Together, these elements describe the values of the dynamic parameter. This element is designed for enumerated

values. If the dynamic parameter doesn't take a value, such as is the case with a `[switch]` parameter, or the values can't be enumerated, add an empty `PossibleValues` element.

The following table lists and describes the `PossibleValues` element and its child elements.

- PossibleValues - This element is a container. Its child elements are described below. Add one `PossibleValues` element to each provider help topic. The element can be empty.
- PossibleValue - This element is a container. Its child elements are described below. Add one `PossibleValue` element for each value of the dynamic parameter.
- Value - Specifies the value name.
- Description - This element contains a `Para` element. The text in the `Para` element describes the value that's named in the `Value` element.

For example, the following XML shows one `PossibleValue` element of the `Encoding` dynamic parameter.

XML

```
<DynamicParameters/>
    <DynamicParameter>
...
        <Description> Specifies the encoding of the output file that contains
the content. </Description>
        <PossibleValues>
            <PossibleValue>
                <Value> ASCII </Value>
                <Description>
                    <para> Uses the encoding for the ASCII (7-bit) character
set. </para>
                </Description>
            </PossibleValue>
...
        </PossibleValues>
</DynamicParameters>
```

Example

The following example shows the `DynamicParameters` element of the `Encoding` dynamic parameter.

XML

```
<DynamicParameters/>
    <DynamicParameter>
```

```
        <Name> Encoding </Name>
        <CmdletSupported> Add-Content, Get-Content, Set-Content </CmdletSupported>
        <Type>
            <Name> Microsoft.PowerShell.Commands.FileSystemCmdletProviderEncoding
</Name>
        <Type>
        <Description> Specifies the encoding of the output file that contains the
content. </Description>
        <PossibleValues>
            <PossibleValue>
                <Value> ASCII </Value>
                <Description>
                    <para> Uses the encoding for the ASCII (7-bit) character set.
</para>
                </Description>
            </PossibleValue>
            <PossibleValue>
                <Value> Unicode </Value>
                <Description>
                    <para> Encodes in UTF-16 format using the little-endian byte
order. </para>
                </Description>
            </PossibleValue>
        </PossibleValues>
</DynamicParameters>
```

Last updated on 04/08/2026

How to Add a See Also Section to a Provider Help Topic

> ⓘ Note
>
> Manual authoring of XML-based help is very difficult. The **PlatyPS** module allows you to write help in Markdown and then convert it to XML-based help. This makes it much easier to write and maintain help. **PlatyPS** can also create the Updateable Help packages for you. For more information, see Create XML-based help using PlatyPS.

This section explains how to populate the **SEE ALSO** section of a provider help topic.

The **SEE ALSO** section consists of a list of topics that are related to the provider or might help the user better understand and use the provider. The topic list can include cmdlet help, provider help and conceptual ("about") help topics in Windows PowerShell. It can also include references to books, paper, and online topics, including an online version of the current provider help topic.

When you refer to online topics, provide the URI or a search term in plain text. The `Get-Help` cmdlet doesn't link or redirect to any of the topics in the list. Also, the `Online` parameter of the `Get-Help` cmdlet doesn't work with provider help.

The **See Also** section is created from the `RelatedLinks` element and the tags that it contains. The following XML shows how to add the tags.

To Add SEE ALSO Topics

1. In the `<AssemblyName>.dll-help.xml` file, within the `providerHelp` element, add a `RelatedLinks` element. The `RelatedLinks` element should be the last element in the `providerHelp` element. Only one `RelatedLinks` element is permitted in each provider help topic.

   For example:

   XML

   ```
   <providerHelp>
       <RelatedLinks>
   ```

```
    </RelatedLinks>
</providerHelp>
```

2. For each topic in the **SEE ALSO** section, within the `RelatedLinks` element, add a `navigationLink` element. Then, within each `navigationLink` element, add one `linkText` element and one `uri` element. If you aren't using the `uri` element, you can add it as an empty element (<uri/>).

   For example:

   XML

   ```
   <providerHelp>
       <RelatedLinks>
           <navigationLink>
               <linkText> </linkText>
               <uri> </uri>
           </navigationLink>
       </RelatedLinks>
   </providerHelp>
   ```

3. Type the topic name between the `linkText` tags. If you are providing a URI, type it between the `uri` tags. To indicate the online version of the current provider help topic, between the `linkText` tags, type "Online version:" instead of the topic name. Typically, the "Online version:" link is the first topic in the SEE ALSO topic list.

   The following example include three SEE ALSO topics. The first refer to the online version of the current topic. The second refers to a Windows PowerShell cmdlet help topic. The third refers to another online topic.

   XML

   ```
   <providerHelp>
       <RelatedLinks>
           <navigationLink>
               <linkText> Online version: </linkText>
               <uri>http://www.fabrikam.com/help/myFunction.htm</uri>
           </navigationLink>
           <navigationLink>
               <linkText> about_functions </linkText>
               <uri/>
           </navigationLink>
           <navigationLink>
               <linkText> Windows PowerShell Getting Started Guide </linkText>
               <uri>https://go.microsoft.com/fwlink/?LinkID=89597<uri/>
           </navigationLink>
   ```

```
    </RelatedLinks>
</providerHelp>
```

Last updated on 05/20/2025

Extended Type System Overview

PowerShell uses its **PSObject** object to extend the types of objects in two ways. First, the **PSObject** object provides a way to show different views of specific object types. This is referred to as showing an adapted view of an object. Second, the **PSObject** object provides a way to add members to existing object. Together, by wrapping an existing object, referred to as the base object, the **PSObject** object provides an extended type system (ETS) that script and cmdlet developers can use to manipulate .NET objects within the shell.

Cmdlet and Script Development Issues

ETS resolves two fundamental issues:

First, some .NET Objects do not have the necessary default behavior for acting as the data between cmdlets.

- Some .NET objects are "meta" objects (for example: WMI Objects, ADO objects, and XML objects) whose members describe the data they contain. However, in a scripting environment it is the contained data that is most interesting, not the description of the contained data. ETS resolves this issue by introducing the notion of Adapters that adapt the underlying .NET object to have the expected default semantics.
- Some .NET Object members are inconsistently named, provide an insufficient set of public members, or provide insufficient capability. ETS resolves this issue by introducing the ability to extend the .NET object with additional members.

Second, the PowerShell scripting language is *typeless* in that a variable does not need to be declared of a particular type. That is, the variables a script developer creates are by nature *typeless*. However, the PowerShell system is "type-driven" in that it depends on having a type name to operate against for basic operations such as outputting results or sorting.

Therefore a script developer must have the ability to state the type of one of their variables and build up their own set of dynamically typed "objects" that contain properties and methods and can participate in the type-driven system. ETS solves this problem by providing a common object for the scripting language that has the ability to state its type dynamically and to add members dynamically.

Fundamentally, ETS resolves the issue mentioned previously by providing the **PSObject** object, which acts as the basis of all object access from the scripting language and provides a standard

abstraction for the cmdlet developer.

Cmdlet Developers

For the cmdlet developers, ETS provides the following support:

- The abstractions to work against objects in a generic way using the **PSObject** object. ETS also provides the ability to drill past these abstractions if required.
- The mechanisms to create a default behavior for formatting, sorting, serialization, and other system manipulations of their object type using a well-known set of extended members.
- The means to operate against any object using the same semantics as the script language using a LanguagePrimitives object.
- The means to dynamically "type" a hash table so that the rest of the system can operate against it effectively.

Script Developers

For the script developers, ETS provides the following support:

- The ability to reference any underlying object type using the same syntax (`$a.x`).
- The ability to access beyond the abstraction provided by the **PSObject** object (such as accessing only adapted members, or accessing the base object itself).
- The ability to define well-known members that control the formatting, sorting, serialization, and other manipulations of an object instance or type.
- The means to name an object as a specific type and thus control the inheritance of its type-based members.
- The ability to add, remove, and modify extended members.
- The ability to manipulate the **PSObject** object itself if required.

The PSObject class

The **PSObject** object is the basis of all object access from the scripting language and provides a standard abstraction for the cmdlet developer. It contains a base-object (a .NET object) and any instance members (members, specifically extended members, that are present on a particular object instance while not necessarily on other objects of the same type). Depending on the type of the base-object, the **PSObject** object might also provide implicit and explicit access to adapted members as well as any type-based extended members.

The **PSObject** object provides the following mechanisms:

- The ability to construct a **PSObject** with or without a base-object.
- The ability to access of all members of each constructed **PSObject** object through a common lookup algorithm and the ability to override that algorithm when required.
- The ability to get and set the type-names of the constructed **PSObject** objects so that scripts and cmdlets can reference similar **PSObject** objects by the same type-name, regardless of the type of their base-object.

How to Construct a PSObject

The following list describes ways to create a **PSObject** object:

- Calling the **PSObject** .#ctor constructor creates a new **PSObject** object with a base-object of PSCustomObject. A base-object of this type indicates that the **PSObject** object has no meaningful base-object. However, a **PSObject** object with this type of base-object does provide a property bag that cmdlet developers can find helpful by adding extended-members.

Developers can also specify the object type-name, which allows this object to share its extended-members with other **PSObject** objects of the same type-name.

- Calling the **PSObject** .#ctor(System.Object) constructor creates a new **PSObject** object with a base-object of type System.Object.

  In this case, the type-name for the created object is a collection of the derivation hierarchy of the base-object. For example, the type-name for the **PSObject** that contains a ProcessInfo base-object would include the following names:
  - System.Diagnostics.Process
  - System.ComponentModel.Component
  - System.MarshalByRefObject
  - System.Object

- Calling the **PSObject** .AsPSObject(System.Object) method creates a new **PSObject** object based on a supplied object.

  If the supplied object is of type System.Object, the supplied object is used as the base-object for the new **PSObject** object. If the supplied object is already a **PSObject** object, the supplied object is returned as is.

Base, adapted, and extended members

Conceptually, ETS uses the following terms to show the relationship between the original members of the base-object and those members added by PowerShell. For more information about the specific types of members that are used by the **PSObject** object, see PSObject class.

Base-object members

If the base-object is specified when constructing the **PSObject** objects, then the members of the base-object are made available through the Members property.

Adapted members

When a base-object is a meta-object, one that contains data in a generic fashion whose properties "describe" their contained data, ETS adapts those objects to a view that allows for direct access to the data through adapted members of the **PSObject** object. Adapted members and base-object members are accessed through the Members property.

Extended members

In addition to the members made available from the base-object or those adapted members created by PowerShell, a **PSObject** may also define extended members that extend the original base-object with additional information that is useful in the scripting environment.

For example, all the core cmdlets provided by PowerShell, such as the Get-Content and Set-Content cmdlets, take a Path parameter. To ensure that these cmdlets, and others, can work against objects of different types, a Path member can be added to those objects so that they all state their information in a common way. This extended Path member ensures that the cmdlets can work against all those types even though there base class might not have a Path member.

Extended members, adapted members, and base-object members are all accessed through the Members property.

Last updated on 05/20/2025

Extended Type System class members

ETS refers to a number of different kinds of members whose types are defined by the **PSMemberTypes** enumeration. These member types include properties, methods, members, and member sets that are each defined by their own CLR type. For example, a **NoteProperty** is defined by its own **PSNoteProperty** type. These individual CLR types have both their own unique properties and common properties that are inherited from the PSMemberInfo class.

The PSMemberInfo class

The **PSMemberInfo** class serves as a base class for all ETS member types. This class provides the following base properties to all member CLR types.

- **Name** property: The name of the member. This name can be defined by the base-object or defined by PowerShell when adapted members or extended members are exposed.
- **Value** property: The value returned from the particular member. Each member type defines how it handles its member value.
- **TypeNameOfValue** property: This is the name of the CLR type of the value that is returned by the Value property.

Accessing members

Collections of members can be accessed through the **Members**, **Methods**, and **Properties** properties of the **PSObject** object.

ETS properties

ETS properties are members that can be treated as a property. Essentially, they can appear on the left-hand side of an expression. They include alias properties, code properties, PowerShell properties, note properties, and script properties. For more information about these types of properties, see ETS properties.

ETS methods

ETS methods are members that can take arguments, may return results, and cannot appear on the left-hand side of an expression. They include code methods, PowerShell methods, and script methods. For more information about these types of methods, see ETS methods.

Last updated on 05/20/2025

ETS member sets

Member sets allow you to partition the members of the **PSObject** object into two subsets so that the members of the subsets can be referenced together by their subset name. The two types of subsets include property sets and member sets. For example of how PowerShell uses member sets, there is a specific property set named **DefaultDisplayPropertySet** that is used to determine, at runtime, which properties to display for a given **PSObject** object.

Property Sets

Property sets can include any number of properties of a **PSObject** type. In general, a property set can be used whenever a collection of properties (of the same type) is needed. The property set is created by calling the `PSPropertySet(System.String,System.Collections.Generic.IEnumerable{System.String})` constructor with the name of the property set and the names of the referenced properties. The created **PSPropertySet** object can then be used as an alias that points to the properties in the set. The PSPropertySet class has the following properties and methods.

- **IsInstance** property: Gets a **Boolean** value that indicates the source of the property.
- **MemberType** property: Gets the type of properties in the property set.
- **Name** property: Gets the name of the property set.
- **ReferencedPropertyNames** property: Gets the names of the properties in the property set.
- **TypeNameOfValue** property: Gets a **PropertySet** enumeration constant that defines this set as a property set.
- **Value** property: Gets or sets the **PSPropertySet** object.
- `PSPropertySet.Copy` method: Makes an exact copy of the **PSPropertySet** object.
- `PSMemberSet.ToString` method: Converts the **PSPropertySet** object to a string.

Member Sets

Member sets can include any number of extended members of any type. The member set is created by calling the `PSMemberSet(System.String,System.Collections.Generic.IEnumerable{System.Management.Automation.PSMemberInfo})` constructor with the name of the member set and the names of the referenced members. The created **PSPropertySet** object can then be used as an alias that

points to the members in the set. The PSMemberSet class has the following properties and methods.

- **IsInstance** property: Gets a **Boolean** value that indicates the source of the member.
- **Members** property: Gets all the members of the member set.
- **MemberType** property: Gets a **MemberSet** enumeration constant that defines this set as a member set.
- **Methods** property: Gets the methods included in the member set.
- **Properties** property: Gets the properties included in the member set.
- **TypeNameOfValue** property: Gets a **MemberSet** enumeration constant that defines this set as a member set.
- **Value** property: Gets the **PSMemberSet** object.
- `PSMemberSet.Copy` method: Makes an exact copy of the **PSMemberSet** object.
- `PSMemberSet.ToString` method: Converts the **PSMemberSet** object to a string.

Last updated on 05/20/2025

ETS properties

Properties are members that can be treated as a property. Essentially, they can appear on the left-hand side of an expression. The properties that are available include alias, code, note, and script properties.

Alias Property

An alias property is a property that references another property that the **PSObject** object contains. It is used primarily to rename the referenced property. However, it may also be used to convert the referenced property's value to another type. With respect to ETS, this type of property is always an extended-member and is defined by the PSAliasProperty class. The class includes the following properties.

- **ConversionType** property: The CLR type used to convert the referenced member's value.
- **IsGettable** property: Indicates whether the value of the referenced property can be retrieved. This property is dynamically determined by examining the **IsGettable** property of the referenced property.
- **IsSettable** property: Indicates whether the value of the referenced property can be set. This property is dynamically determined by examining the **IsSettable** property of the referenced property.
- **MemberType** property: An **AliasProperty** enumeration constant that defines this property as an alias property.
- **ReferencedMemberName** property: The name of the referenced property that this alias refers to.
- **TypeNameOfValue** property: The full name of the CLR type of the referenced property's value.
- **Value** property: The value of the referenced property.

Code Property

A code property is a property that is a getter and setter that is defined in a CLR language. In order for a code property to become available, a developer must write the property in some CLR language, compile, and ship the resultant assembly. This assembly must be available in the runspace where the code property is desired. With respect to ETS, this type of property is

always an extended-member and is defined by the PSCodeProperty class. The class includes the following properties.

- **GetterCodeReference** property: The method used to get the value of the code property.
- **IsGettable** property: Indicates whether the value of the code property can be retrieved, that the **SetterCodeReference** property: The method used to set the value of the code property.
- **IsSettable** property: Indicates whether the value of the code property can be set, that the **SetterCodeReference** property is not null.
- **MemberType** property: A **CodeProperty** enumeration constant that defines this property as a code property.
- **SetterCodeReference** property: The method used to get the value of the code property.
- **TypeNameOfValue** property: The CLR type of the code property value that is returned by the properties get operation.
- **Value** property: The value of the code property. When this property is retrieved, the getter code in the GetterCodeReference property is invoked, passing the current **PSObject** object and returning the value returned by the invocation. When this property is set, the setter code in the **SetterCodeReference** property is invoked, passing the current **PSObject** object as the first argument and the object used to set the value as the second argument.

Note Property

A Note property is a property that has a name/value pairing. With respect to ETS, this type of property is always an extended-member and is defined by the PSNoteProperty class. The class includes the following properties.

- **IsGettable** property: Indicates whether the value of the note property can be retrieved.
- **IsSettable** property: Indicates whether the value of the note property can be set.
- **MemberType** property: A **NoteProperty** enumeration constant that defines this property as a note property.
- **TypeNameOfValue** property: The fully-qualified type name of the object returned by the note property's get operation.
- **Value**: The value of the note property.

PowerShell property

A PowerShell property is a property defined on the base object or a property that is made available through an adapter. It can refer to both CLR fields as well as CLR properties. With respect to ETS, this type of property can be either a base-member or an adapter-member and is defined by the PSProperty class. The class includes the following properties.

- **IsGettable** property: Indicates whether the value of the base or adapted property can be retrieved.
- **IsSettable** property: Indicates whether the value of the base or adapted property can be set.
- **MemberType** property: A Property enumeration constant that defines this property as a PowerShell property.
- **TypeNameOfValue** property: The fully-qualified name of the property value type. For example, for a property whose value is a string, its property value type is **System.String**.
- **Value** property: The value of the property. If the get or set operation is called on a property that does not support that operation, a **GetValueException** or **SetValueException** exception is thrown

PowerShell Script property

A Script property is a property that has getter and setter scripts. With respect to ETS, this type of property is always an extended-member and is defined by the PSScriptProperty class. The class includes the following properties.

- **GetterScript** property: The script used to retrieve the script property value.
- **IsGettable** property: Indicates whether the **GetterScript** property exposes a script block.
- **IsSettable** property: Indicates whether the **SetterScript** property exposes a script block.
- **MemberType** property: A ScriptProperty enumeration constant that identifies this property as a script property.
- **SetterScript** property: The script used to set the script property value.
- **TypeNameOfValue** property: The fully-qualified type name of the object returned by the getter script. In this case **System.Object** is always returned.
- **Value** property: The value of the script property. A get invokes the getter script and returns the value provided. A set invokes the setter script.

Last updated on 05/20/2025

ETS class methods

ETS methods are members that can take arguments, may return results, and cannot appear on the left-hand side of an expression. The methods that are available within ETS include code, Windows PowerShell, and script methods.

> **Note**
>
> From scripts, methods are accessed using the same syntax as other members with the addition of parenthesis at the end of the method name.

Code Methods

A code method is an extended member that is defined in a CLR language. It provides similar functionality to a method defined on a base object; however, a code method may be added dynamically to an **PSObject** object. In order for a code method to become available, a developer must write the property in some CLR language, compile, and ship the resultant assembly. This assembly must be available in the runspace where the code method is desired. Be aware that a code method implementation must be thread safe. Access to these methods is done through PSCodeMethod objects that provides the following public methods and properties.

- `PSCodeMethod.Copy` method: Makes an exact copy of the **PSCodeMethod** object.
- `PSCodeMethod.Invoke(System.Object[])` method: Invokes the underlying code method.
- `PSCodeMethod.ToString` method: Converts the **PSCodeMethod** object to a string.
- `PSCodeMethod.CodeReference` property: Gets the underlying method that the code method is based on.
- **PSMemberInfo.IsInstance** property: Gets a **Boolean** value that indicates the source of the member.
- **PSCodeMethod.MemberType** property: Gets an **PSMemberTypes.CodeMethod** enumeration constant that identifies this method as a code method.
- **PSMemberInfo.Name** property: Gets the name of the underlying code method.
- **PSCodeMethod.OverloadDefinitions** property: Gets a definition of all the overloads of the underlying code method.
- **PSCodeMethod.TypeNameOfValue** property: Gets the full name of the code method.
- **PSMemberInfo.Value** property: Gets the **PSCodeMethod** object.

Windows PowerShell Methods

A PowerShell method is a CLR method defined on the base object or is made accessible through an adapter. Access to these methods is done through **PSMethod** objects that provides the following public methods and properties.

- `PSMethod.Copy` method: Makes an exact copy of the **PSMethod** object.
- `PSMethod.Invoke(System.Object[])` method: Invokes the underlying method.
- `PSMethod.ToString` method: Converts the **PSMethod** object to a string.
- **PSMemberInfo.IsInstance** property: Gets a **Boolean** value that indicates the source of the member.
- **PSMethod.MemberType** property: Gets an **PSMemberTypes.Method** enumeration constant that identifies this method as a PowerShell method.
- **PSMemberInfo.Name** property: Gets the name of the underlying method.
- **PSMethod.OverloadDefinitions** property: Gets the definitions of all the overloads of the underlying method.
- **PSMethod.TypeNameOfValue** property: Gets the ETS type of this method.
- **PSMemberInfo.Value** property: Gets the **PSMethod** object.

Script Methods

A script method is an extended member that is defined in the PowerShell language. It provides similar functionality to a method defined on a base object; however, a script method may be added dynamically to an **PSObject** object. Access to these methods is done through PSScriptMethod objects that provides the following public methods and properties.

- `PSScriptMethod.Copy` method: Makes an exact copy of the **PSScriptMethod** object.
- `PSScriptMethod.Invoke(System.Object[])` method: Invokes the underlying script method.
- `PSScriptMethod.ToString` method: Converts the **PSScriptMethod** object to a string.
- **PSMemberInfo.IsInstance** property: Gets a **Boolean** value that indicates the source of the member.
- **PSScriptMethod.MemberType** property: Gets a **PSMemberTypes.ScriptMethod** enumeration constant that identifies this method as a script method.
- **PSMemberInfo.Name** property: Gets the name of the underlying code method.
- **PSScriptMethod.OverloadDefinitions** property: Gets the definitions of all the overloads of the underlying script method.
- **PSScriptMethod.TypeNameOfValue** property: Gets the ETS type of this method.
- **PSScriptMethod.Script** property: Gets the script used to invoke the method.

- **PSMemberInfo.Value** property: Gets the **PSScriptMethod** object.

Last updated on 05/20/2025

ETS type converters

ETS uses two basic types of type converters when a call is made to the `LanguagePrimitives.ConvertTo(System.Object, System.Type)` method. When this method is called, PowerShell attempts to perform the type conversion using its standard PowerShell language converters or a custom converter. If PowerShell cannot perform the conversion, it throws an **PSInvalidCastException** exception.

Standard Windows PowerShell Language Converters

These standard conversions are checked before any custom conversions and cannot be overridden. The following table lists the type conversions performed by PowerShell when the `ConvertTo(System.Object, System.Type)` method is called. Be aware that references to the **valueToConvert** and **resultType** parameters refer to parameters of the `ConvertTo(System.Object,System.Type)` method.

Expand table

From (valueToConvert)	To (resultType)	Returns
Null	String	""
Null	Char	'\0'
Null	Numeric	`0` of the type specified in the **resultType** parameter.
Null	Boolean	Results of call to the `IsTrue(System.Object)(Null)` method.
Null	PSObject	New object of type **PSObject**.
Null	Non-value-type	Null.
Null	Nullable<T>	Null.
Derived Class	Base class	**valueToConvert**
Anything	Void	**AutomationNull.Value**
Anything	String	Calls `ToString` mechanism.

From (valueToConvert)	To (resultType)	Returns
Anything	Boolean	`IsTrue(System.Object) (valueToConvert)`
Anything	PSObject	Results of call to the `AsPSObject(System.Object) (valueToConvert)` method.
Anything	Xml Document	Converts **valueToConvert** to string, then calls **XMLDocument** constructor.
Array	Array	Attempts to convert each element of the array.
Singleton	Array	`Array[0]` equals **valueToConvert** that is converted to the element type of the array.
IDictionary	Hash table	Results of call to Hashtable(valueToConvert).
String	Char[]	`valueToConvert.ToCharArray`
String	RegEx	Results of call to `Regx(valueToConvert)`.
String	Type	Returns the appropriate type using the **valueToConvert** parameter to search **RunspaceConfiguration.Assemblies**.
String	Numeric	If **valueToConvert** is "", returns `0` of the **resultType**. Otherwise the culture "culture invariant" is used to produce a numeric value.
Integer	System.Enum	Converts the integer to the constant if the integer is defined by the enumeration. If the integer is not defined an **PSInvalidCastException** exception is thrown.

Custom conversions

If PowerShell cannot convert the type using a standard PowerShell language converter, it then checks for custom converters. PowerShell looks for several types of custom converters in the order described in this section. Be aware that references to the **valueToConvert** and **resultType** parameters refer to parameters of the `ConvertTo(System.Object, System.Type)` method. If a custom converter throws an exception, then no further attempt is made to convert the object and that exception is wrapped in a **PSInvalidCastException** exception which is then thrown.

PowerShell type converter

PowerShell type converters are used to convert a single type or a family of types, such as all types that derive from the **System.Enum** class. To create a PowerShell type converter you must

implement an PSTypeConverter class and associate that implementation with the target class. There are two ways of associating the PowerShell type converter with its target class.

- Through the type configuration file
- By applying the **TypeConverterAttribute** attribute to the target class

PowerShell type converters, derived from the PSTypeConverter abstract class, provide methods for converting an object to a specific type or from a specific type. If the **valueToConvert** parameter contains an object that has a PowerShell Type converter associated with it, PowerShell calls the `PSTypeConverter.ConvertTo(System.Object, System.Type,System.IFormatProvider, System.Boolean)` method of the associated converter to convert the object to the type specified by the **resultType** parameter. If the **resultType** parameter references a type that has a PowerShell type converter associated with it, PowerShell calls the `PSTypeConverter.ConvertFrom(System.Object,System.Type, System.IFormatProvider, System.Boolean)` method of the associated converter to convert the object from the type specified by the **resultType** parameter.

System type converter

System type converters are used to convert a specific target class. This type of converter cannot be used to convert a family of classes. To create an system type converter you must implement an TypeConverter class and associate that implementation with the target class. There are two ways of associating the system type converter with its target class.

- Through the type configuration file
- By applying the TypeConverterAttribute attribute to the target class

Parse converter

If the **valueToConvert** parameter is a string, and the object type of the **resultType** parameter has a `Parse` method, then the `Parse` method is called to convert the string.

Constructor converter

If the object type of the **resultType** parameter has a constructor that has a single parameter that is the same type as the object of the **valueToConvert** parameter, then this constructor is called.

Implicit cast operator converter

If the **valueToConvert** parameter has an implicit cast operator that converts to **resultType**, then its cast operator is called. If the **resultType** parameter has an implicit cast operator that converts from **valueToConvert**, then its cast operator is called.

Explicit cast operator converter

If the **valueToConvert** parameter has an explicit cast operator that converts to **resultType**, then its cast operator is called. If the **resultType** parameter has an explicit cast operator that converts from **valueToConvert**, then its cast operator is called.

Last updated on 05/20/2025

Errors and exceptions in the Extended Type System

Errors can occur in ETS during the initialization of type data and when accessing a member of an **PSObject** object or using one of the utility classes such as **LanguagePrimitives**.

Runtime errors

With one exception, when casting, all exceptions thrown from ETS during runtime are either an **ExtendedTypeSystemException** exception or an exception derived from the **ExtendedTypeSystemException** class. This allows script developers to trap these exceptions using the `trap` statement in their script.

Errors getting member values

All errors that occur when getting the value of an ETS member (property, method, or parameterized property) cause a **GetValueException** or **GetValueInvocationException** exception to be thrown. When ETS recognizes that an error occurred a **GetValueException** exception is thrown. When the underlying getter of a referenced member recognizes that an error occurred, a **GetValueInvocationException** exception is thrown that may or may not include the inner exception that caused the get invocation error.

Errors setting member values

All errors that occur when setting the value of an ETS property cause a **SetValueException** or **SetValueInvocationException** exception to be thrown. When ETS recognizes that an error occurred a **SetValueException** exception is thrown. When the underlying setter of a referenced property recognizes that an error occurred, a **SetValueInvocationException** exception is thrown that may or may not include the inner exception that caused the set invocation error.

Errors invoking a method

All errors that occur when invoking an ETS method cause a **MethodException** or **MethodInvocationException** exception to be thrown. When ETS recognizes that an error occurred a **MethodException** exception is thrown. When the referenced method recognizes

that an error occurred, a **MethodInvocationException** exception is thrown that may or may not include the inner exception that caused the invocation error.

Casting errors

When an invalid cast is attempted, an **PSInvalidCastException** is thrown. Because this exception derives from **System.InvalidCastException**, it is not able to be directly trapped from script. Be aware that the entity attempting the cast would need to wrap **PSInvalidCastException** in an **PSRuntimeException** for this to be trappable by scripts. If an attempt is made to set the value of an **PSPropertySet**, **PSMemberSet**, **PSMethodInfo**, or a member of the **ReadOnlyPSMemberInfoCollection`1**, a **NotSupportedException** is thrown.

Common runtime errors

Any other common runtime errors that occur are of type **ExtendedTypeSystemException** exception with no additional specific exception types.

Initialization errors

Errors may occur when initializing `types.ps1xml`. Typically, these errors are displayed when the PowerShell runtime starts. However, they can also be displayed when a module is loaded.

Last updated on 05/20/2025

The Monad Manifesto

07/07/2025

Jeffrey Snover, the inventor of PowerShell, wrote the Monad Manifesto to explain his vision for PowerShell and how it would change the way we manage systems. You can read more about why he wrote it in his Monad Manifesto blog post.

In that post, he explained how hard it was to communicate the vision for PowerShell to widely distributed development teams. The development teams didn't understand the vision. Writing the Manifesto forced him to be clear about what problem was being addressed, the core principles of the design, how to address the issue, who would benefit from it, and why.

Use the following link to download a copy of the Manifesto.

- Monad Manifesto .

This document is historically significant to PowerShell. It's a version of the original Monad Manifesto, which articulated the long-term vision and initiated the development effort that became PowerShell. PowerShell has delivered on many of the elements described in this document.

Contributing to PowerShell documentation

Thank you for your support of PowerShell!

The Contributor's Guide is a collection of articles that describe the tools and processes we use to create documentation at Microsoft. Some of these guides cover information common to any documentation set published to `learn.microsoft.com`. Other guides are specific to how we write documentation for PowerShell.

The common articles are available in our centralized Contributor's Guide. The PowerShell-specific guides are available here.

Ways to contribute

There are two ways to contribute. Both contributions are valuable to us.

- Filing issues helps us identify problems and gaps in our documentation. Sometimes the issues are difficult to resolve, requiring more investigation and research. The issue process allows us to have a conversation about the problem and develop a satisfactory resolution.
- Submitting a pull request to add or change content is a more involved process. The following information outlines the tools, processes, and standards for submitting content to the documentation.

Prepare to make a contribution

Contributing to the documentation requires a GitHub account. Use the following checklist to install and configure the tools you need to make contributions.

1. Sign up for GitHub
2. Install Git and Markdown tools
3. Install the Docs Authoring Pack
4. Install Posh-Git - not required but recommended
5. Set up a local Git repository
6. Review Git and GitHub fundamentals

Get started writing docs

There are two ways to contribute changes to the documentation:

1. Quick edits to existing docs - Minor corrections, fixing typos, or small additions
2. Full GitHub workflow for docs - large changes, multiple versions, adding or changing images, or contributing new articles

Also, read the Writing essentials section of the centralized Contributor's Guide. Another excellent resource is the Microsoft Writing Style Guide.

Minor corrections or clarifications to documentation and code examples in public repositories are covered by the learn.microsoft.com Terms of Use.

Use the full GitHub workflow when you're making significant changes. If you're not an employee of Microsoft, our PR validation system adds a comment to the pull request asking you to sign the online Contribution Licensing Agreement (CLA). You must complete this step before we can review or accept your pull request. Signing the CLA is only required the first time you submit a PR in the repository. You might be asked to sign the CLA for each time you contribute to a new repository.

Code of conduct

All repositories that publish to Microsoft Learn adhere to the Microsoft Open Source Code of Conduct or the .NET Foundation Code of Conduct. For more information, see the Code of Conduct FAQ.

Next steps

The following articles cover information specific to PowerShell documentation. Where there's overlap with the guidance in the centralized Contributor's Guide, we call out how those rules differ for the PowerShell content.

Review the following documents:

- Get started writing docs
- Markdown best practices
- PowerShell-Docs style guide
- How to file an issue
- Submitting a pull request

Additional resources

- Editorial checklist
- How we manage issues
- How we manage pull requests

Last updated on 11/20/2025

Get started contributing to PowerShell documentation

This article is an overview of how to get started as a contributor to the PowerShell documentation.

PowerShell-Docs structure

There are three categories of content in the PowerShell-Docs repository:

- reference content
- conceptual content
- metadata and configuration files

Reference content

The reference content is the PowerShell cmdlet reference for the cmdlets that ship in PowerShell. The cmdlet reference is collected in versioned folders (like 5.1, 7.4, 7.5, and 7.6), which contain the reference for the modules that ship with PowerShell. This content is also used to create the help information displayed by the `Get-Help` cmdlet.

Conceptual content

The conceptual documentation isn't organized by version. All articles are displayed for every version of PowerShell.

> **Note**
>
> Anytime a conceptual article is added, removed, or renamed, the TOC must be updated. Any deleted or renamed files must be redirected.

Metadata files

This project contains several types of metadata files. The metadata files control the behavior of our build tools and the publishing system. Only PowerShell-Docs maintainers and approved contributors are allowed to change these files. If you think that a meta file should be changed, open an issue to discuss the needed changes.

Meta files in the root of the repository

- `.*` - configuration files in the root of the repository
- `*.md` - Project documentation in the root of the repository
- `*.yml` - Project documentation in the root of the repository
- `.devcontainer/*` - devcontainer configuration files
- `.github/**/*` - GitHub templates, actions, and other meta files
- `.vscode/**/*` - VS Code extension configurations
- `assets/*` - contains downloadable files linked in the documentation
- `redir/*` - contain redirection mapping files
- `tests/*` - test tools used by the build system
- `tools/*` - other tools used by the build system

Meta files in the documentation set

- `reference/**/*.json` - docset configuration files
- `reference/**/*.yml` - TOC and other structured content files
- `reference/bread/*` - breadcrumb navigation configuration
- `reference/includes/*` - markdown include files
- `reference/mapping/*` - version mapping configuration
- `reference/**/media/**` - image files used in documentation
- `reference/module/*` - Module Browser page configuration

Creating new articles

A GitHub issue must be created for any new document you want to contribute. Check for existing issues to make sure you're not duplicating efforts. Assigned issues are considered to be `in progress`. If you wish to collaborate on an issue, contact the person assigned to the issue.

Similar to the PowerShell RFC process, create an issue before you write the content. The issue ensures you don't waste time and effort on work that gets rejected by the PowerShell-Docs team. The issue allows us to consult with you on the scope of the content and where it fits in the PowerShell documentation. All articles must be included in the Table of Contents (TOC). The proposed TOC location should be included in the issue discussion.

> **Note**

The publishing system autogenerates the TOC for reference content. You don't have to update the TOC.

Updating existing articles

Where applicable, cmdlet reference articles are duplicated across all versions of PowerShell maintained in this repository. When reporting an issue about a cmdlet reference or an `About_` article, list the versions of the article that have the problem.

Apply the appropriate change to each version of the file.

Localized content

The PowerShell documentation is written in English and translated into 17 other languages. The English content is stored in the GitHub repository named `MicrosoftDocs/PowerShell-Docs`. Issues found in the translated content should be submitted to this repository.

All translations start from the English content first. We use both human and machine translation.

Expand table

Translation method	Languages
Human translation	de-DE, es-ES, fr-FR, it-IT, ja-JP, ko-KR, pt-BR, ru-RU, zh-CN, zh-TW
Machine translation	cs-CZ, hu-HU, nl-NL, pl-PL, pt-PT, sv-SE, tr-TR

The content translated by machine translation might not always result in correct word choices and grammar. If you find an error in translation for any language, rather than in the technical details of the article, open an issue explaining why you think the translation is wrong.

Some translation issues can be fixed by changing the English source files. However, some issues can require updates to our internal translation system. For those cases, we must submit the issue to our internal localization team for review and response.

Next steps

There are two common ways of submitting changes in GitHub. Both methods are described in the central Contributor's Guide:

1. You can make quick edits to existing documents in the GitHub web interface.
2. Use the full GitHub workflow for adding new articles, updating multiple files, or other large changes.

Before starting any changes, you should create a fork of the PowerShell-Docs repository. The changes should be made in a working branch in your copy of the PowerShell-Docs. If you're using the **quick edit** method in GitHub, these steps are handled for you. If you're using the **full GitHub workflow**, you must be set up to work locally.

Both methods end with the creation of a Pull Request (PR). For more information and best practices, see Submitting a pull request.

Last updated on 11/20/2025

Contribute using GitHub Codespaces

Article • 03/30/2025

GitHub has a feature called Codespaces that you can use to contribute to the PowerShell documentation without having to install or configure any software locally. When you use a codespace, you get the same authoring tools the team uses for writing and editing.

You can use a codespace in your browser, making your contributions in VS Code hosted over the internet. If you have VS Code installed locally, you can connect to the codespace there too.

Available tools

When you use a codespace to contribute to the PowerShell documentation, your editor has these tools already available for you:

- Markdownlint for checking your Markdown syntax.
- cSpell for checking your spelling.
- Vale for checking your prose.
- The Learn Authoring Pack for inserting platform-specific syntax, previewing your contribution, and more.
- The Reflow Markdown extension for wrapping your Markdown as needed, making reading and editing easier.
- The Table Formatter extension for making your tables more readable without having to manually align columns.
- The change-case extension for converting the casing of your headings and prose.
- The GitLens extension for reviewing historical file changes.
- The PowerShell extension for interacting authoring PowerShell examples.
- The Gremlins tracker for Visual Studio Code for finding problematic characters in your Markdown.

Cost

GitHub Codespaces can be used for free up to 120 core-hours per month. The monthly usage is calculated across all your repositories, not just documentation.

For more information about pricing, see About billing for GitHub Codespaces.

> **Tip**
>
> If you're comfortable using containers and Docker, you can get the same experience by using the devcontainer defined for the PowerShell documentation repositories. There's no cost associated with using devcontainers. For more information, see the **Dev Containers tutorial**.

Creating your GitHub Codespace

To create your GitHub Codespace for contributing to PowerShell documentation, follow these steps:

1. Open https://github.com/codespaces in your browser.
2. Select the "New codespace" button in the top right of the page.
3. In the "Create a new codespace" page, select the "Select a repository" button and type the name of the repository you want to contribute to, like `MicrosoftDocs/PowerShell-Docs`.
4. Leave all other settings as their default.
5. Select the "Create codespace" button.

After following these steps, GitHub creates a new codespace for that repository and sets it up for you. When the codespace is ready, the page refreshes and shows the web editor UI for the codespace. The UI is based on VS Code and works the same way.

Opening your GitHub Codespace

To open your GitHub Codespace in the browser, follow these steps:

1. Open https://github.com/codespaces in your browser.
2. The page lists your Codespaces. Find the created codespace for the repository you want to contribute to and select it.

After you select your codespace, GitHub opens it in the same window. From here, you're ready to contribute.

To open your GitHub Codespace in VS Code, follow the steps in Using GitHub Codespaces in Visual Studio Code.

Authoring in your GitHub Codespace

Once you have your GitHub Codespace open in your browser or VS Code, contributing to the documentation follows the same process.

The rest of this article describes a selection of tasks you can do in your GitHub Codespace while writing or editing your contribution.

Extract a reference link

When you want to turn an inline link, like `[some text](destination.md)`, into a reference link like `[some text][01]`, select the link destination in the editor. Then you can either:

1. Right-click on the link destination and select "Refactor..." in the context menu.
2. Press `Ctrl` + `Shift` + `R`.

Either action raises the refactoring context menu. To replace the `(destination.md)` in the link with `[def]`, select **Extract to link definition** in the context menu. You can rename the definition by typing a name in.

For the PowerShell documentation, we use two-digit numerical reference link definitions, like `[01]` or `[31]`. Only use reference link definitions in about articles and conceptual documentation. Don't use reference link definitions in cmdlet reference documentation.

Fix prose style violations

When you review an article in your codespace, Vale automatically checks the article when you first open it and every time you save it. If Vale finds any style violations, it highlights them in the document with colored squiggles.

Hover over a violation to see more information about it.

To open a web page that explains the rule, select the rule's name in the hover information. To open the rule and review its implementation, select the rule's filename (ending in `.yml`).

If the rule supports a quick fix, you can select "Quick Fix..." in the hover information for the violation and apply one of the suggested fixes by selecting it from the context menu. You can also press `Ctrl` + `.` when your cursor is on a highlighted problem to apply a quick fix if the rule supports it.

If the rule doesn't support quick fixes, read the rule's message and fix it if you can. If you're not sure how to fix it, the editors can make a suggestion when reviewing your PR.

Fix syntax problems

When you review an article in your codespace, Markdownlint automatically checks the article when you open it and as you type. If Markdownlint finds any syntax problems, it highlights them in the document with colored squiggles.

Hover over a violation to see more information about it. To open a web page that explains the rule, select the rule's ID in the hover information.

If the rule supports a quick fix, you can select "Quick Fix..." in the hover information for the violation and apply one of the suggested fixes by selecting it from the context menu. You can also press `Ctrl` + `.` when your cursor is on a highlighted problem to apply a quick fix if the rule supports it.

If the rule doesn't support quick fixes, read the rule's message and fix it if you can. If you're not sure how to fix it, the editors can make a suggestion when reviewing your PR.

You can also apply fixes to all syntax violations in an article. To do so, open the command palette and type `Fix all supported markdownlint violations in the document`. As you type, the command palette filters the available commands. Select the "Fix all supported markdownlint violations in the document" command. When you do, Markdownlint updates the document to resolve any violations it has a quick fix for.

Format a table

To format a Markdown table, place your cursor in or on the table in your Markdown. Open the Command Palette and search for the `Table: Format Current` command. When you select that command, it updates the Markdown for your table to align and pad the table for improved readability.

It converts a table defined like this:

markdown

```
| foo | bar | baz |
|:--:|:--|-:|
| a | b | c |
```

Into this:

markdown

```
|  foo  | bar  | baz  |
| :---: | :--- | ---: |
|   a   | b    |    c |
```

Insert an alert

The documentation uses alerts to make information more notable to a reader.

To insert an alert, you can, open the Command Palette and search for the `Learn: Alert` command. When you select that command, it opens a context menu. Select the alert type you want to add. When you do, the command inserts the alert's Markdown at your cursor in the document.

Make a heading use sentence casing

To convert a heading's casing, highlight the heading's text except for the leading `#` symbols, which set the heading level. When you have the text highlighted, open the Command Palette and search for the `Change case sentence` command. When you select that command, it converts the casing of the highlighted text.

You can also use the casing commands for any text in the document.

Open the Command Palette

You can use VS Code's Command Palette to run many helpful commands.

To open the Command Palette in the UI, select "View" in the top menu bar. Then select "Command Palette..." in the context menu.

To open the Command Palette with your keyboard, press the key combination for your operating system:

- Windows and Linux: `Ctrl` + `Shift` + `P`
- macOS: `Cmd` + `Shift` + `P`

Preview your contribution

To preview your contribution, open the Command Palette and search for the `Markdown: Open Preview` command. When you select that command, VS Code opens a preview of the active document. The preview's style matches the Learn platform.

> **Note**
>
> Site-relative and cross-reference links don't work in the preview.

Reflow your content

To limit the line lengths for a paragraph in a document, place your cursor on the paragraph. Then open the Command Palette and search for the `Reflow Markdown` command. When you select the command, it updates the current paragraph's line lengths to the configured length. For our repositories, that length is 99 characters.

When using this command for block quotes, make sure the paragraph in the block quote you're reflowing is surrounded by blank lines. Otherwise, the command reflows every paragraph together.

> **⊗ Caution**
>
> Don't use this command when editing about articles. The lines in those documents must not be longer than 80 characters. There's currently no way for a repository to configure different line lengths by folder or filename, so reflow doesn't work for about article documents.

Review all problems in a document

To review all syntax and style rule violations in a document, open the Problems View.

To open the Problems View in the UI, select "View" in the top menu bar. Then select "Problems" in the context menu.

To open the Problems View with your keyboard, press the key combination for your operating system:

- Windows and Linux: `Ctrl` + `Shift` + `M`
- macOS: `Cmd` + `Shift` + `M`

The Problems View displays all errors, warnings, and suggestions for the open document. Select a problem to scroll to it in the document.

You can filter the problems by type or text matching.

Updating the ms.date metadata

To update the `ms.date` metadata for an article, open the Command Palette and search for the `Learn: Update "ms.date" Metadata Value` command. When you select the command, it updates the metadata to the current date.

Additional resources

The tasks and commands described in this article don't cover everything you can do with VS Code or the installed extensions.

For more information on using VS Code, see these articles:

- Visual Studio Code Tips and Tricks
- Basic Editing
- Using Git source control in VS Code
- Markdown and Visual Studio Code

For more information about the installed extensions, see their documentation:

- change-case
- GitLens
- Gremlins tracker for Visual Studio Code
- Learn Authoring Pack
- markdownlint
- Reflow Markdown
- Table Formatter

Markdown best practices

Article • 04/29/2025

This article provides specific guidance for using Markdown in our documentation. It isn't a tutorial for Markdown. If you need a tutorial for Markdown, see this Markdown cheatsheet.

The build pipeline that builds our documentation uses markdig to process the Markdown documents. Markdig parses the documents based on the rules of the latest CommonMark specification. OPS follows the CommonMark specification and adds some extensions for platform-specific features, such as tables and alerts.

The CommonMark specification is stricter about the construction of some Markdown elements. Pay close attention to the details provided in this document.

We use the markdownlint extension in VS Code to enforce our style and formatting rules. This extension is installed as part of the **Learn Authoring Pack**.

Blank lines, spaces, and tabs

Blank lines also signal the end of a block in Markdown.

- Put a single blank between Markdown blocks of different types; for example, between a paragraph and a list or header.
- Don't use more than one blank line. Multiple blank lines render as a single blank line in HTML, therefore the extra blank lines are unnecessary.
- Don't use put multiple consecutive blank lines in a code block, consecutive blank lines break the code block.

Spacing is significant in Markdown.

- Remove extra spaces at the end of lines. Trailing spaces can change how Markdown renders.
- Always use spaces instead of tabs (hard tabs).

Titles and headings

Use ATX headings only (`#` style, as opposed to `=` or `-` style headers).

- Use sentence case - only proper nouns and the first letter of a title or header should be capitalized
- Include a single space between the `#` and the first letter of the heading
- Surround headings with single blank line

- Don't use more than one H1 per document
  - It should be the first header
  - It should match the title of the article
- Increment header levels by one - don't skip levels
- Limit depth to H3 or H4
- Avoid using bold or other markup in headers

Limit line length to 100 characters

For conceptual articles and cmdlet reference, limit lines to 100 characters. For `about_` articles, limit the line length to 79 characters. Limiting the line length improves the readability of `git` diffs and history. It also makes it easier for other writers to make contributions.

Use the Reflow Markdown extension in VS Code to reflow paragraphs to fit the prescribed line length.

Some content types can't be easily reflowed. For example, the code inside of code blocks can also be difficult to reflow, depending on the content and the code language. You can't reflow a table. In these cases, use your best judgment to keep the content as close to the 100-character limit as possible.

Emphasis

For emphasis, Markdown supports bold and italics. Markdown allows you to use either `*` or `_` to mark the emphasis. However, to be consistent and show intent:

- Use `**` for bold
- Use `_` for italics

Following this pattern makes it easier for others to understand the intent of the markup when there's a mix of bold and italics in a document.

Lists

If your list has multiple sentences or paragraphs, consider using a sublevel header rather than a list.

Surround lists with a single blank line.

Unordered lists

- Don't end list items with a period unless they contain multiple sentences.
- Use the hyphen character (`-`) for list item bullets to avoid confusion with emphasis markup that uses the asterisk (`*`).
- To include a paragraph or other elements under a bullet item, insert a line break and align indentation with the first character after the bullet.

For example:

Markdown

```
This is a list that contains child elements under a bullet item.

- First bullet item

  Sentence explaining the first bullet.

  - Child bullet item

    Sentence explaining the child bullet.

- Second bullet item
- Third bullet item
```

This is a list that contains child elements under a bullet item.

- First bullet item

  Sentence explaining the first bullet.

  - Child bullet item

    Sentence explaining the child bullet.

- Second bullet item
- Third bullet item

Ordered lists

- All items in a numbered list should use the number `1.` rather than incrementing each item.
  - Markdown rendering increments the value automatically.
  - This makes reordering items easier and standardizes the indentation of subordinate content.
- To include a paragraph or other elements under a numbered item, align indentation with the first character after the item number.

For example:

```Markdown
1. For the first element, insert a single space after the `1`. Long sentences
should be wrapped to
   the next line and must line up with the first character after the numbered list
marker.

   To include a second element, insert a line break after the first and align
indentations. The
   indentation of the second element must line up with the first character after
the numbered list
   marker.

1. The next numbered item starts here.
```

The resulting Markdown is rendered as follows:

1. For the first element, insert a single space after the `1`. Long sentences should be wrapped to the next line and must line up with the first character after the numbered list marker.

   To include a second element, insert a line break after the first and align indentations. The indentation of the second element must line up with the first character after the numbered list marker.

2. The next numbered item starts here.

Images

The syntax to include an image is:

```Markdown
![[alt text]](<folderPath>)

Example:
![Introduction](./media/overview/Introduction.png)
```

Where `alt text` is a brief description of the image and `<folderPath>` is a relative path to the image.

- Alternate text is required to support screen readers for the visually impaired.
- Images should be stored in a `media/<article-name>` folder within the folder containing the article.

    - Create a folder that matches the filename of your article under the `media` folder. Copy the images for that article to that new folder.
- Don't share images between articles.
    - If an image is used by multiple articles, each folder must have a copy of that image.
    - This prevents a change to an image in one article from affecting another article.

The following image file types are supported: `*.png`, `*.gif`, `*.jpeg`, `*.jpg`, `*.svg`

Markdown extension - Alert boxes

The Learn Authoring Pack contains tools that support features unique to our publishing system. Alerts are a Markdown extension to create blockquotes that render with colors and icons highlighting the significance of the content. The following alert types are supported:

Markdown

```
> [!NOTE]
> Information the user should notice even if skimming.

> [!TIP]
> Optional information to help a user be more successful.

> [!IMPORTANT]
> Essential information required for user success.

> [!CAUTION]
> Negative potential consequences of an action.

> [!WARNING]
> Dangerous certain consequences of an action.
```

These alerts look like this on Microsoft Learn:

Note block

> **Note**
>
> Information the user should notice even if skimming.

Tip block

> **Tip**
>
> Optional information to help a user be more successful.

Important block

> **ⓘ Important**
>
> Essential information required for user success.

Caution block

> **⊗ Caution**
>
> Negative potential consequences of an action.

Warning block

> **⚠ Warning**
>
> Dangerous certain consequences of an action.

Markdown extension - Tables

A table is an arrangement of data with rows and columns consisting of:

- A single header row
- A delimiter row separating the header from the data
- Zero or more data rows

Each row consists of cells containing arbitrary text separated by pipes (`|`). A leading and trailing pipe is also recommended for clarity. Spaces between pipes and cell content are trimmed. Block-level elements can't be inserted in a table. All content of a row must be on a single line.

The delimiter row consists of cells whose only content are hyphens (`-`), and optionally, a leading or trailing colon (`:`), or both, to indicate left, right, or center alignment respectively.

For small tables, consider using a list instead. Lists are:

- Easier to maintain and read
- Can be reflowed to fit within the 100-character line limit
- More accessible for users that use screen readers for visual assistance

For more information, see *Tables* section of Markdown reference for Microsoft Learn.

Hyperlinks

- Hyperlinks must use Markdown syntax `[friendlyname](url-or-path)`.
- The publishing system supports three types of links:
  - URL links
  - File links
  - Cross-reference links
- All URLs to external websites should use HTTPS unless that isn't valid for the target site.
- Links must have a friendly name, usually the title of the linked article
- Avoid using backticks, bold, or other markup inside the brackets of a hyperlink.
- Bare URLs can be used when you're documenting a specific URI but must be enclosed in backticks. For example:

  Markdown

  ```
  By default, if you don't specify this parameter, the DMTF standard resource
  URI
  `http://schemas.dmtf.org/wbem/wscim/1/cim-schema/2/` is used and the class
  name is appended to it.
  ```

- Use link references where allowed. Link references within paragraphs make the paragraphs more readable.

  Link references divide a Markdown link into two parts:
  - the link reference - `[friendlyname][id]`
  - the link definition - `[id]: url-or-path`

URL-type Links

- URL links to other articles on `learn.microsoft.com` must use site-relative paths. The simplest way to create a site-relative link is to copy the URL from your browser then remove `https://learn.microsoft.com/en-us` from the value you paste into markdown.
- Don't include locales in URLs on Microsoft properties (remove `/en-us` from URL) or Wikipedia links.
- Remove any unnecessary query parameters from the URL. Examples that should be removed:
  - `?view=powershell-5.1` - used to link to a specific version of PowerShell

    - `?redirectedfrom=MSDN` - added to the URL when you get redirected from an old article to its new location
- If you need to link to a specific version of a document, you must add the `&preserve-view=true` parameter to the query string. For example: `?view=powershell-5.1&preserve-view=true`
- On Microsoft sites, URL links don't contain file extensions (for example, no `.md`)

File-type links

- A file link is used to link from one reference article to another, or from one conceptual article to another in the same docset.
    - If you need to link from a conceptual article to a reference article you must use a URL link.
    - If you need to link to an article in another docset or across repositories you must use a URL link.
- Use relative filepaths (for example: `../folder/file.md`)
- All file paths use forward-slash (`/`) characters
- Include the file extension (for example, `.md`)

Cross-reference links

Cross-reference links are a special feature supported by the publishing system. You can use cross-reference links in conceptual articles to link to .NET API or cmdlet reference.

- For links to .NET API reference, see Use links in documentation in the central Contributor Guide.

- Links to cmdlet reference have the following format: `xref:<module-name>.<cmdlet-name>`. For example, to link to the `Get-Content` cmdlet in the **Microsoft.PowerShell.Management** module.

  ```
  [Get-Content](xref:Microsoft.PowerShell.Management.Get-Content)
  ```

Deep linking

Deep linking is allowed on both URL and file links.

- The anchor text must be lowercase
- Add the anchor to the end of the target path. For example:
    - `[about_Splatting](about_Splatting.md#splatting-with-arrays)`

    - `[custom key bindings](https://code.visualstudio.com/docs/getstarted/keybindings#_custom-keybindings-for-refactorings)`

For more information, see Use links in documentation.

Code spans

Code spans are used for inline code snippets within a paragraph. Use single backticks to indicate a code span. For example:

```
Markdown
```

```
PowerShell cmdlet names use the `Verb-Noun` naming convention.
```

This example renders as:

PowerShell cmdlet names use the `Verb-Noun` naming convention.

Code blocks

Code blocks are used for command examples, multi-line code samples, query languages, and outputs. There are two ways to indicate a section of text in an article file is a code block: by fencing it in triple-backticks (```) or by indenting it.

Never use indentation because it's too easy to get wrong and it may be difficult for another writer to understand your intent when they need to edit your article.

Fenced code blocks can include an optional tag that indicates the language syntax contained in the block. The publishing platform supports a list of language tags. The language tag is used to provide syntax highlighting when the article is rendered on the webpage. The language tag is not case-sensitive, but you should use lowercase except for a few special cases.

- Code fences without tags can be used for syntax blocks or other types of content where syntax highlighting is not required.
- When showing output from a command, use a tagged code fence with the language tag `Output`. The rendered box is labeled as **Output** and doesn't have syntax highlighting.
- If the output is in a specific supported language, use the appropriate language tag. For example, many commands output JSON, so use the `json` tag.
- If you use an unsupported language tag, the code block will render without syntax highlighting. The language tag becomes the label for the rendered code box on the webpage. Capitalize the tag so that the label is capitalized on the webpage.

Next steps

PowerShell style guide

PowerShell-Docs style guide

Article • 03/30/2025

This article provides style guidance specific to the PowerShell-Docs content. It builds on the information outlined in the Overview.

Formatting command syntax elements

Use the following rules to format elements of the PowerShell language when the elements are used in a sentence.

- Always use the full name for cmdlets and parameters in the proper Pascal case
- Only use an alias when you're specifically demonstrating the alias
- PowerShell keywords and operators should be all lowercase
- The following items should be formatted using **bold** text:
  - Type names
  - Class names
  - Property names
  - Parameter names
    - By default, use the parameter without the hyphen prefix.
    - Use parameter name with the hyphen if you're illustrating syntax. Wrap the parameter in backticks.

    For example:

    ```markdown
    The parameter's name is **Name**, but it's typed as `-Name` when used on
    the command
    line as a parameter.
    ```

- The following items should be formatted using backticks (`):
  - Property and parameter values
  - Type names that use the bracketed style - For example: `[System.Io.FileInfo]`

- Referring to characters by name. For example: Use the asterisk character (`*`) to as a wildcard.
- Language keywords and operators
- Cmdlet, function, and script names
- Command and parameter aliases
- Method names - For example: The `ToString()` method returns a string representation of the object
- Variables
- Native commands
- File and directory paths
- Inline command syntax examples - See Markdown for code samples

  This example shows some backtick examples:

  ```
  markdown

  The following code uses `Get-ChildItem` to list the contents of
  `C:\Windows` and assigns
  the output to the `$files` variable.

  ```powershell
  $files = Get-ChildItem C:\Windows
  ```
  ```

  This example shows command syntax inline:

  ```
  markdown

  To start the spooler service on a remote computer named DC01, you type:
  `sc.exe \\DC01 start spooler`.
  ```

  Including the file extension ensures that the correct command is executed according to PowerShell's command precedence.

Markdown for code samples

Markdown supports two different code styles:

- **Code spans (inline)** - marked by a single backtick (`) character. Used within a paragraph rather than as a standalone block.
- **Code blocks** - a multi-line block surrounded by triple-backtick (```) strings. Code blocks can also have a language label following the backticks. The language label enables syntax highlighting for the contents of the code block.

All code blocks should be contained in a code fence. Never use indentation for code blocks. Markdown allows this pattern but it can be problematic and should be avoided.

A code block is one or more lines of code surrounded by a triple-backtick (```) code fence. The code fence markers must be on their own line before and after the code sample. The opening marker can have an optional language label. The language label enables syntax highlighting on the rendered webpage.

For a full list of supported language tags, see Fenced code blocks in the centralized contributor guide.

Publishing also adds a **Copy** button that can copy the contents of the code block to the clipboard. This allows you to paste the code into a script to test the code sample. However, not all examples are intended to be run as written. Some code blocks are basic illustrations of PowerShell concepts.

There are three types code blocks used in our documentation:

1. Syntax blocks
2. Illustrative examples
3. Executable examples

Syntax code blocks

Syntax code blocks are used to describe the syntactic structure of a command. Don't use a language tag on the code fence. This example illustrates all the possible parameters of the `Get-Command` cmdlet.

markdown

````
```
Get-Command [-Verb <String[]>] [-Noun <String[]>] [-Module <String[]>]
  [-FullyQualifiedModule <ModuleSpecification[]>] [-TotalCount <Int32>] [-Syntax]
  [-ShowCommandInfo] [[-ArgumentList] <Object[]>] [-All] [-ListImported]
  [-ParameterName <String[]>] [-ParameterType <PSTypeName[]>] [<CommonParameters>]
```
````

This example describes the `for` statement in generalized terms:

markdown

```
```
for (<init>; <condition>; <repeat>)
{<statement list>}
```
```

Illustrative examples

Illustrative examples are used to explain a PowerShell concept. Yo`u should Avoid using PowerShell prompts in examples whenever possible. However, illustrative examples aren't meant to be copied and pasted for execution. They're most commonly used for simple examples that are easy to understand. You may include the PowerShell prompt and example output.

Here's a simple example illustrating the PowerShell comparison operators. In this case, we don't intend the reader to copy and run this example. Notice that this example uses `PS>` as a simplified prompt string.

markdown

```
```powershell
PS> 2 -eq 2
True

PS> 2 -eq 3
False

PS> 1,2,3 -eq 2
2

PS> "abc" -eq "abc"
True

PS> "abc" -eq "abc", "def"
False

PS> "abc", "def" -eq "abc"
abc
```
```

Executable examples

Complex examples, or examples that are intended to be copied and executed, should use the following block-style markup:

markdown

```
```powershell
<Your PowerShell code goes here>
```
```

The output displayed by PowerShell commands should be enclosed in an **Output** code block to prevent syntax highlighting. For example:

markdown

````
```powershell
Get-Command -Module Microsoft.PowerShell.Security
```

```Output
CommandType  Name                        Version    Source
-----------  ----                        -------    ------
Cmdlet       ConvertFrom-SecureString    3.0.0.0    Microsoft.PowerShell.Security
Cmdlet       ConvertTo-SecureString      3.0.0.0    Microsoft.PowerShell.Security
Cmdlet       Get-Acl                     3.0.0.0    Microsoft.PowerShell.Security
Cmdlet       Get-AuthenticodeSignature   3.0.0.0    Microsoft.PowerShell.Security
Cmdlet       Get-CmsMessage              3.0.0.0    Microsoft.PowerShell.Security
Cmdlet       Get-Credential              3.0.0.0    Microsoft.PowerShell.Security
Cmdlet       Get-ExecutionPolicy         3.0.0.0    Microsoft.PowerShell.Security
Cmdlet       Get-PfxCertificate          3.0.0.0    Microsoft.PowerShell.Security
Cmdlet       New-FileCatalog             3.0.0.0    Microsoft.PowerShell.Security
Cmdlet       Protect-CmsMessage          3.0.0.0    Microsoft.PowerShell.Security
Cmdlet       Set-Acl                     3.0.0.0    Microsoft.PowerShell.Security
Cmdlet       Set-AuthenticodeSignature   3.0.0.0    Microsoft.PowerShell.Security
Cmdlet       Set-ExecutionPolicy         3.0.0.0    Microsoft.PowerShell.Security
Cmdlet       Test-FileCatalog            3.0.0.0    Microsoft.PowerShell.Security
Cmdlet       Unprotect-CmsMessage        3.0.0.0    Microsoft.PowerShell.Security
```
````

The `Output` code label isn't an official *language* supported by the syntax highlighting system. However, this label is useful because our publishing system adds the **Output** label to the code box frame on the web page. The **Output** code box has no syntax highlighting.

Coding style rules

Avoid line continuation in code samples

Avoid using line continuation characters (`` ` ``) in PowerShell code examples. Backtick characters are difficult to see and can cause problems if there are extra spaces at the end of the line.

- Use PowerShell splatting to reduce line length for cmdlets that have several parameters.

- Take advantage of PowerShell's natural line break opportunities, like after pipe (`|`) characters, opening braces (`{`), parentheses (`(`), and brackets (`[`).

Avoid using PowerShell prompts in examples

Use of the prompt string is discouraged and should be limited to scenarios that are meant to illustrate command-line usage. For most of these examples, the prompt string should be `PS>`. This prompt is independent of OS-specific indicators.

Prompts are required in examples to illustrate commands that alter the prompt or when the path displayed is significant to the scenario. The following example illustrates how the prompt changes when using the Registry provider.

PowerShell

```
PS C:\> cd HKCU:\System\
PS HKCU:\System\> dir

    Hive: HKEY_CURRENT_USER\System

Name                   Property
----                   --------
CurrentControlSet
GameConfigStore        GameDVR_Enabled                         : 1
                       GameDVR_FSEBehaviorMode                 : 2
                       Win32_AutoGameModeDefaultProfile        : {2, 0, 1, 0...}
                       Win32_GameModeRelatedProcesses          : {1, 0, 1, 0...}
                       GameDVR_HonorUserFSEBehaviorMode        : 0
                       GameDVR_DXGIHonorFSEWindowsCompatible : 0
```

Don't use aliases in examples

Use the full name of all cmdlets and parameters unless you're specifically documenting the alias. Cmdlet and parameter names must use the proper Pascal-cased names.

Using parameters in examples

Avoid using positional parameters. To reduce the chance of confusion, you should include the parameter name in an example, even if the parameter is positional.

Formatting cmdlet reference articles

Cmdlet reference articles have a specific structure. PlatyPS defines this structure. PlatyPS generates the cmdlet help for PowerShell modules in Markdown. After you edit the Markdown files, PlatyPS can create the MAML help files used by the `Get-Help` cmdlet.

PlatyPS has a schema that expects a specific structure for cmdlet reference. The PlatyPS schema document describes this structure. Schema violations cause build errors that must be fixed before we can accept your contribution.

- Don't remove any of the ATX header structures. PlatyPS expects a specific set of headers in a specific order.
- The H2 **INPUTS** and **OUTPUTS** headers must have an H3 type. If the cmdlet doesn't take input or return a value, then use the value `None` for the H3.
- Inline code spans can be used in any paragraph.
- Fenced code blocks are only allowed in the **EXAMPLES** section.

In the PlatyPS schema, **EXAMPLES** is an H2 header. Each example is an H3 header. Within an example, the schema doesn't allow code blocks to be separated by paragraphs. The schema only allows the following structure:

```

### Example X - Title sentence

0 or more paragraphs
1 or more code blocks
0 or more paragraphs.
```

Number each example and add a brief title.

For example:

````
markdown

### Example 1: Get cmdlets, functions, and aliases

This command gets the PowerShell cmdlets, functions, and aliases that are
installed on the
computer.

```powershell
Get-Command
```

### Example 2: Get commands in the current session

```powershell
````
````

```
Get-Command -ListImported
```

Formatting About_ files

`About_*` files are written in Markdown but are shipped as plain text files. We use Pandoc to convert the Markdown to plain text. `About_*` files are formatted for the best compatibility across all versions of PowerShell and with the publishing tools.

Basic formatting guidelines:

- Limit paragraph lines to 80 characters
- Limit code blocks to 76 characters
- Limit blockquotes and alerts to 78 characters
- When using these special meta-characters `\`, `$`, and `<`:
  - Within a header, these characters must be escaped using a leading `\` character or enclosed in code spans using backticks (`` ` ``)
  - Within a paragraph, these characters should be put into code spans. For example:

    markdown

    ```
    ### The purpose of the \$foo variable

    The `$foo` variable is used to store ...
    ```

- Markdown tables
  - For `About_*` articles, tables must fit within 76 characters
    - If the content doesn't fit within 76 character limit, use bullet lists instead
  - Use opening and closing `|` characters on each line

Next steps

Editorial checklist

Editor's checklist

Article • 03/30/2025

This article contains a summarized list of rules for writing or editing PowerShell documentation. See other articles in the Contributor's Guide for detailed explanations and examples of these rules.

Metadata

- `ms.date`: must be in the format **MM/DD/YYYY**
  - Change the date when there's a significant or factual update
    - Reorganizing the article
    - Fixing factual errors
    - Adding new information
  - Don't change the date if the update is insignificant
    - Fixing typos and formatting
- `title`: unique string of 43-59 characters long (including spaces)
  - Don't include site identifier (it's autogenerated)
  - Use sentence case - capitalize only the first word and any proper nouns
- `description`: 115-145 characters including spaces - this abstract displays in the search result

Formatting

- Backtick syntax elements that appear, inline, within a paragraph
  - Cmdlet names `Verb-Noun`
  - Variable `$counter`
  - Syntactic examples `Verb-Noun -Parameter`
  - File paths `C:\Program Files\PowerShell`, `/usr/bin/pwsh`
  - URLs that aren't meant to be clickable in the document
  - Property or parameter values
- Use bold for property names, parameter names, class names, module names, entity names, object, or type names
  - Bold is used for semantic markup, not emphasis
  - Bold - use asterisks `**`
- Italic - use underscore `_`
  - Only used for emphasis, not for semantic markup
- Line breaks at 100 columns (or at 80 for **about_Topics**)
- No hard tabs - use spaces only

- No trailing spaces on lines
- PowerShell keywords and operators should be all lowercase
- Use proper (Pascal) casing for cmdlet names and parameters

Headers

- Start with H1 first - only one H1 per article
- Use ATX Headers only
- Use sentence case for all headers
- Don't skip levels - no H3 without an H2
- Limit header depth to H3 or H4
- Add blank lines before and after
- Don't add or remove headers - PlatyPS enforces specific headers in its schema

Code blocks

- Add blank lines before and after
- Use tagged code fences - **powershell**, **Output**, or other appropriate language ID
- Use untagged code fence for syntax blocks
- Put output in separate code block except for basic examples where you don't intend for the reader to use the **Copy** button
- See list of supported languages

Lists

- Indent properly
- Add blank lines before first item and after last item
- Use hyphen (`-`) for bullets not asterisk (`*`) to reduce confusion with emphasis
- Use `1.` for all items in a numbered list

Terminology

- Use *PowerShell* vs. *Windows PowerShell*
- See Product Terminology

Cmdlet reference examples

- Must have at least one example in cmdlet reference
- Examples should be only enough code to demonstrate the usage

- PowerShell syntax
  - Use full names of cmdlets and parameters - no aliases
  - Use splatting for parameters when the command line gets too long
  - Avoid using line continuation backticks - only use when necessary
- Remove or simplify the PowerShell prompt (`PS>`) except where required for the example
- Cmdlet reference example must follow the following PlatyPS schema

  markdown

  ````
  ### Example 1 - Descriptive title

  Zero or more short descriptive paragraphs explaining the context of the
  example followed by one or
  more code blocks. Recommend at least one and no more than two.

  ```powershell
  ... one or more PowerShell code statements ...
  ```

  ```Output
  Example output of the code above.
  ```

  Zero or more optional follow up paragraphs that explain the details of the
  code and output.
  ````

- don't put paragraphs between the code blocks. All descriptive content must come before or after the code blocks.

Linking to other documents

- When linking outside the docset or between cmdlet reference and conceptual
  - Use site-relative URLs when linking to Microsoft Learn (remove `https://learn.microsoft.com/en-us`)
  - don't include locales in URLs on Microsoft properties (remove `/en-us` from URL)
  - All URLs to external websites should use HTTPS unless that's not valid for the target site
- When linking within the docset
  - Use the relative filepath (`../folder/file.md`)
- All paths use forward-slash (`/`) characters
- Image links should have unique alt text

Product terminology and branding guidelines

Article • 03/30/2025

When you write about any product, it's important to use the proper product names and terminology. This guide defines product names and terminology related to PowerShell. Note the capitalization of specific words or use cases.

PowerShell (collective name)

Use **PowerShell** to describe the scripting language and an interactive shell.

PowerShell (product name)

The cross-platform version of PowerShell built on .NET (core), rather than the .NET Framework. PowerShell can be installed on Windows, Linux, and macOS.

PowerShell Core (product deprecated)

The name used for PowerShell v6, built on .NET Core. This name shouldn't be used.

Windows PowerShell (product name)

The version of PowerShell that ships in Windows, which requires the full .NET Framework.

Guidelines

- First mention - use "Windows PowerShell"
- Subsequent mentions - Use "PowerShell" unless the use case requires "Windows PowerShell" to be more specific:

  > In PowerShell, the `Invoke-WebRequest` cmdlet returns **BasicHtmlWebResponseObject**

  > In Windows PowerShell, the `Invoke-WebRequest` cmdlet returns **HtmlWebResponseObject**

PowerShell modules

PowerShell modules are add-ons that contain PowerShell cmdlets to manage specific products or services.

For example:

- Azure PowerShell
- Az.Accounts module
- Windows management module
- Hyper-V module
- Microsoft Graph PowerShell SDK
- Exchange PowerShell

Guidelines

- Always use the collective name or the more specific module name when referring to a PowerShell module
- Never refer to a module as "PowerShell"

Azure PowerShell (collective name)

The branded group of products containing PowerShell modules used to manage Azure.

There are several versions of Azure PowerShell products available. Each product contains multiple named modules.

Guidelines

- Use "Azure PowerShell" as the collective name for the product
- Always use the collective name, never just "PowerShell"
- Use the more specific product name when referring to a specific version

Az PowerShell (product name)

The currently supported collection of modules for managing Azure resources with PowerShell.

AzureRM PowerShell (product name)

The previous generation of modules that use the Azure Resource Manager (ARM) model for managing Azure resources. This product is deprecated, no longer maintained or supported, and not recommended.

Azure Service Management PowerShell (product name)

The earliest collection of modules for managing legacy Azure resources that use Azure Service Manager (ASM) APIs. This legacy PowerShell module isn't recommended when creating new resources since ASM is scheduled for retirement.

Azure-related PowerShell modules

These products are used to manage Azure resources but aren't part of the Azure PowerShell collective product. They should never be described using the "Azure PowerShell" collective name.

- Azure Information Protection PowerShell
- Azure Deployment Manager PowerShell
- Azure Elastic Database Jobs PowerShell
- Azure Service Fabric PowerShell
- Azure Stack PowerShell
- Microsoft Graph PowerShell SDK
- Microsoft Entra PowerShell

Guidelines

- Always use the full proper name of the product or the specific PowerShell module name

Other PowerShell-related products

Visual Studio Code (VS Code)

This is Microsoft's free, open source editor.

Guidelines

- First mention - use the full name
- Subsequent mentions - you can use "VS Code"
- Never use "VSCode"

PowerShell Extension for Visual Studio Code

The extension turns VS Code into the preferred IDE for PowerShell.

Guidelines

- First mention - use the full name
- Subsequent mentions - you can use "PowerShell extension"

How to file a PowerShell-Docs issue

Article • 03/30/2025

There are two ways to create an issue:

1. Use the feedback controls at the bottom of the page.
2. File an issue in GitHub directly

Using the feedback controls

For a full description of the feedback controls, see the Docs Team blog announcing this feature.

At the bottom of most pages on `learn.microsoft.com`, there are two feedback buttons. One is a link for product feedback and one is for documentation feedback. The documentation feedback requires a GitHub account. Clicking the button takes you in GitHub and presents an issue template. Enter your feedback and submit the form.

> **Note**
>
> The feedback tool not a support channel. It's a way to ask questions to clarify documentation or to report errors and omissions. If you need technical support, see Community resources.

Filing issues on GitHub

To file a GitHub issue directly, you can select the New issue button in the PowerShell-Docs repository. Select the **Get started** button for the issue you want to create. The GitHub issue template helps you provide the information needed to address the problem you're reporting.

To avoid duplication, search the existing issues to see if someone else has already reported it. If you find an existing issue, you can add your comments, related questions, or answers.

Next steps

See Get started writing.

Additional resources

How we manage issues

How to submit pull requests

Article • 03/30/2025

To make changes to content, submit a pull request (PR) from your fork. A pull request must be reviewed before it can be merged. For best results, review the editorial checklist before submitting your pull request.

Using git branches

The default branch for PowerShell-Docs is the `main` branch. Changes made in working branches are merged into the `main` branch before then being published. The `main` branch is merged into the `live` branch every weekday at 3:00 PM (Pacific Time). The `live` branch contains the content that is published to `learn.microsoft.com`.

Before starting any changes, create a working branch in your local copy of the PowerShell-Docs repository. When working locally, be sure to synchronize your local repository before creating your working branch. The working branch should be created from an up-to-date copy of the `main` branch.

All pull requests should target the `main` branch. Don't submit changes to the `live` branch. Changes made in the `main` branch get merged into `live`, overwriting any changes made to `live`.

Make the pull request process work better for everyone

The simpler and more focused you can make your PR, the faster it can be reviewed and merged.

Avoid pull requests that update large numbers of files or contain unrelated changes

Avoid creating PRs that contain unrelated changes. Separate minor updates to existing articles from new articles or major rewrites. Work on these changes in separate working branches.

Bulk changes create PRs with large numbers of changed files. Limit your PRs to a maximum of 50 changed files. Large PRs are difficult to review and are more prone to contain errors.

Renaming or deleting files

There must be an issue associated with the PR when you rename or delete files. That issue must discuss the need to rename or delete the files.

Avoid mixing content additions or changes with file renames and deletes. Any file that you rename or delete must be added to the appropriate redirection file. When possible, update any files that link to the renamed or deleted content, including any TOC files.

Avoid editing repository configuration files

Avoid modifying repository configuration files. Limit your changes where possible to the Markdown content files and any supporting image files needed for the content.

Incorrect modifications to repository configuration files can break the build, introduce vulnerabilities or accessibility issues, or violate organizational standards. Repository configuration files are any files that match one or more of these patterns:

- `*.yml`
- `.github/**`
- `.localization-config`
- `.openpublishing*`
- `LICENSE*`
- `reference/docfx.json`
- `reference/mapping/**`
- `tests/**`
- `ThirdPartyNotices`
- `tools/**`

For safety and security, don't change these files. If you think one of these files should be changed, file an issue. After the maintainers triage the issue, they'll make the appropriate changes.

Use the PR template

When you create a PR, a template is automatically inserted into the PR body for you. It looks like this:

Markdown

```
# PR Summary

<!--
    Delete this comment block and summarize your changes and list
    related issues here. For example:
```

```
    This changes fixes problem X in the documentation for Y.

    - Fixes #1234
    - Resolves #1235
-->

## PR Checklist

<!--
    These items are mandatory. For your PR to be reviewed and merged,
    ensure you have followed these steps. As you complete the steps,
    check each box by replacing the space between the brackets with an
    x or by clicking on the box in the UI after your PR is submitted.
-->

- [ ] **Descriptive Title:** This PR's title is a synopsis of the changes it
proposes.
- [ ] **Summary:** This PR's summary describes the scope and intent of the change.
- [ ] **Contributor's Guide:** I have read the [contributors guide][contrib].
- [ ] **Style:** This PR adheres to the [style guide][style].

<!--
    If your PR is a work in progress, please mark it as a draft or
    prefix it with "(WIP)" or "WIP:"

    This helps us understand whether or not your PR is ready to review.
-->

[contrib]: /powershell/scripting/community/contributing/overview
[style]: /powershell/scripting/community/contributing/powershell-style-guide
```

In the "PR Summary" section, write a short summary of your changes and list any related issues by their issue number, like `#1234`. If your PR fixes or resolves the issue, use GitHub's autoclose feature so the issue is automatically closed when your PR is merged.

Review the items in the "PR Checklist" section and check them off as you complete each one. You must follow the directions and check each item for the team to approve your PR.

If your PR is a work-in-progress, set it to draft mode or prefix your PR title with `WIP`.

Expectations Comment

After you submit your PR, a bot will comment on your PR. The comment provides resources and sets expectations for the rest of the process. We might update this comment periodically, so always review the comment, even if this isn't your first contribution.



github-actions bot commented 9 days ago

Expectations

Thanks for your submission! Here's a quick note to provide you with some context for what to expect from the docs team and the process now that you've submitted a PR. Even if you've contributed to this repo before, we strongly suggest reading this information; it might have changed since you last read it.

To see our process for reviewing PRs, please read our editor's checklist and process for managing pull requests in particular. Below is a brief, high-level summary of what to expect, but our contributor guide has expanded details.

The docs team begins to review your PR if you request them to or if your PR meets these conditions:

- It is not a draft PR.
- It does not have a `WIP` prefix in the title.
- It passes validation and build steps.
- It does not have any merge conflicts.
- You have checked every box in the PR Checklist, indicating you have completed all required steps and marked your PR as **Ready to Merge**.

You can **always** request a review at any stage in your authoring process, the docs team is here to help! You do not need to submit a fully polished and finished draft; the docs team can help you get content ready for merge.

While reviewing your PR, the docs team may make suggestions, write comments, and ask questions. When all requirements are satisfied, the docs team marks your PR as **Approved** and merges it. Once your PR is merged, it is included the next time the documentation is published. For this project, the documentation is published daily at 3 p.m. Pacific Standard Time (PST).

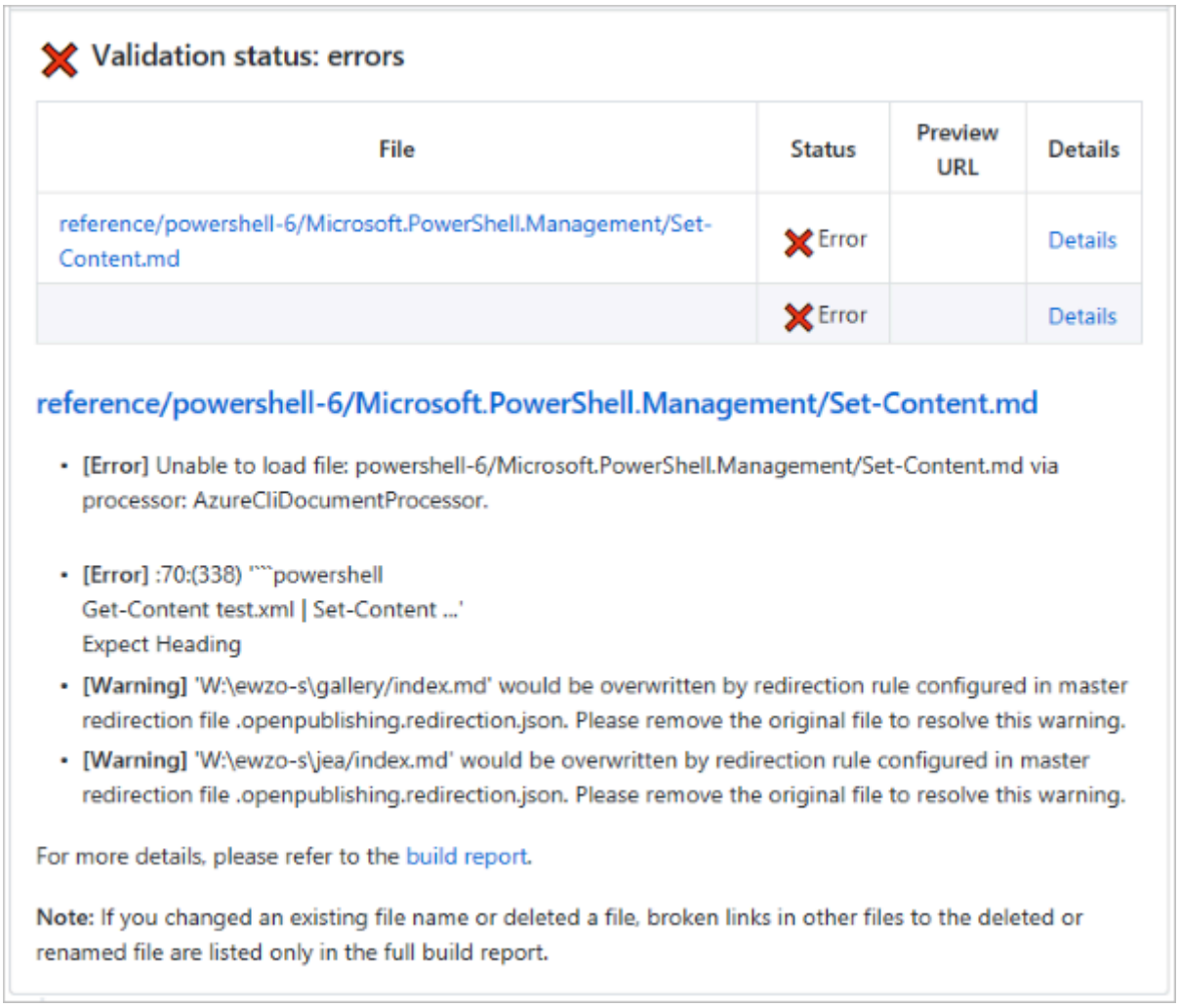
Docs PR validation service

The Docs PR validation service is a GitHub app that runs validation rules on your changes. You must fix any errors or warnings reported by the validation service.

The following steps outline the validation behavior:

1. You submit a PR.

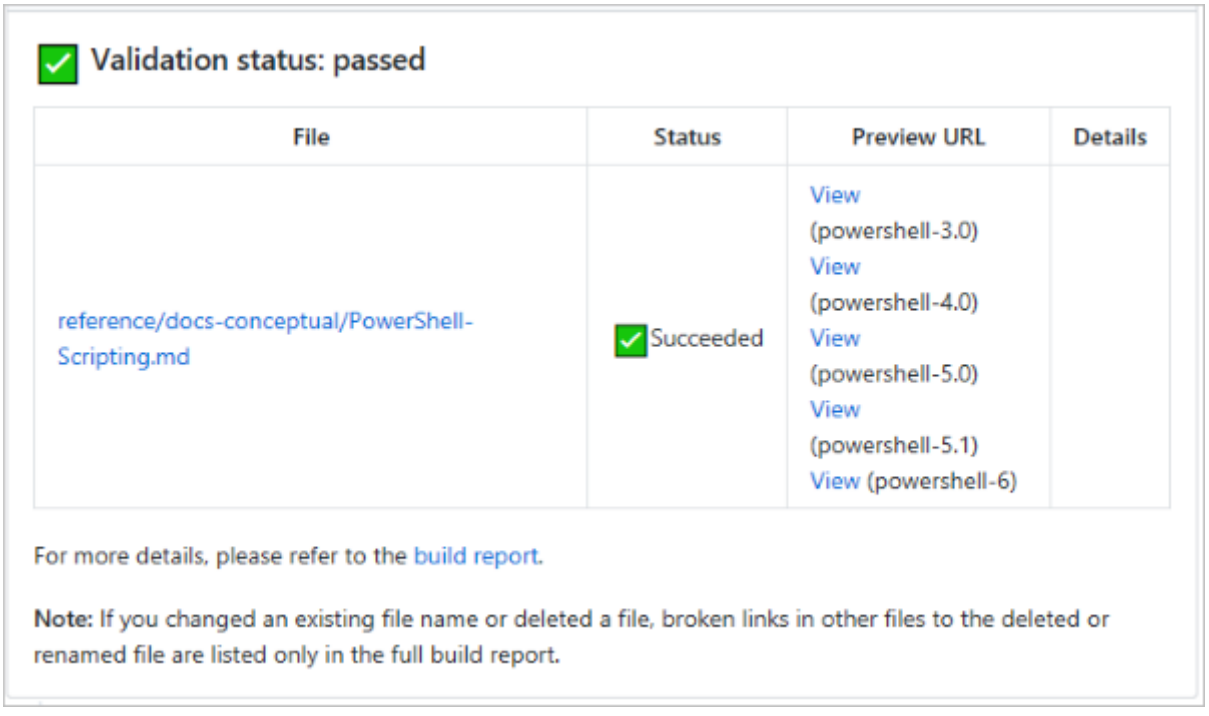
2. In the GitHub comment that indicates the status of the "checks" enabled on the repository. In this example, there are two checks enabled, "Commit Validation" and "OpenPublishing.Build":



The build can pass even if commit validation fails.

3. Select **Details** for more information. The **Details** page shows all the validation checks that failed and includes information about how to fix the issues.

4. When validation succeeds, the following comment is added to the PR:



> ⓘ Note
>
> If you're an external contributor (not a Microsoft employee), you don't have access to the detailed build reports or preview links.

When the PR is reviewed, you might be asked to make changes or fix validation warning messages. The PowerShell-Docs team can help you understand validation errors and editorial requirements.

GitHub Actions

Several different GitHub Actions run against your changes to validate and provide context for you and the reviewers.

Checklist verification

If your PR isn't in draft mode and isn't prefixed with `WIP`, a GitHub Action inspects your PR to verify that you selected every item in the PR template's checklist. The maintainers won't review or merge your PR until you complete the checklist. The checklist items are mandatory.

Authorization verification

If your PR targets the `live` branch or modifies any repository configuration files, a GitHub Action checks your permissions to verify that you're authorized to submit those changes.

Only repository administrators are authorized to target the `live` branch or modify repository configuration files.

Versioned content change reporting

If your PR adds, removes, or modifies any versioned content a GitHub Action analyzes your changes and writes a report summarizing the types of changes made to versioned content.

This report can show if there are other versions of the files that you need to update in this PR.

To find the versioned content report for your PR:

1. Selecting the "Checks" tab on your PR page.
2. Select the "Reporting" job from the list of jobs.
3. Select the "..." button in the top right.
4. Select "View job summary."

Versioned Content summary

Versioned Content Change Report

> Pull Request:
>
> ```
> Owner:  MicrosoftDocs
> Repo:   PowerShell-Docs
> Number: 9065
> ```

This report is organized by base folder (a folder containing multiple versions of content) with child folders beneath. Each child folder section enumerates all of the changes to files in that folder across the versions in tables.

Each table includes the version-relative path to the files and the change status of those files for each version. The version-relative path is the remainder of the path from the version folder. For example, the file `reference/5.1/Foo/Bar/Baz.md` is in the `reference` base folder and has a version-relative path of `Foo/Bar/Baz.md`. For change status, every versioned file will have one of the following statuses:

- **Added:** The file was added. Git status `A`.
- **Changed:** The file's type was changed. Git status `T`.
- **Copied:** The file was copied. Git status `C`.
- **Modified:** The file's contents were changed. Git status `M`.
- **Removed:** The file was deleted. Git status `D`.
- **Renamed:** The file was renamed. Git status `R`.
- **Unchanged:** The file was not modified.
- **N/A:** The file does not exist for this version.

In cases where the change status of a file is not the same across versions, the Pull Request author and reviewer should both double check the file to understand if the discrepancy is intentional or accidental.

Base Folder: `reference`

Changed **1** versioned files in `reference`. See below for details per folder.

Change Sets for `Microsoft.PowerShell.LocalAccounts`

Version-Relative Path	5.1	7.0	7.2	7.3
`Set-LocalUser.md`	Modified	N/A	N/A	N/A

Job summary generated at run-time

Next steps

PowerShell-Docs style guide

Additional resources

How we manage pull requests

Contributing quality improvements

Summarize this article for me

You don't have to be a subject matter expert or a documentation expert to contribute. If you see an opportunity to improve the documentation, submit a pull request with your proposed improvement. This guide outlines several simple ways that anyone can contribute quality improvements to the documentation.

We're focusing on these quality areas:

- Formatting code samples
- Formatting command syntax
- Link References
- Markdown linting
- Spelling

Formatting code samples

All code samples should follow the style guidelines for PowerShell content. Those rules are repeated in abbreviated form here for convenience:

- All code blocks should be contained in a triple-backtick code fence (```).
- Line length for code blocks is limited to `90` characters for cmdlet reference articles.
- Line length for code blocks is limited to `76` characters for `about_*` articles.
- Avoid using line continuation characters (`` ` ``) in PowerShell code examples.
  - Use splatting or natural line break opportunities, like after pipe (`|`) characters, opening braces (`{`), parentheses (`(`), and brackets (`[`) to limit line length.
- Only include the PowerShell prompt for illustrative examples where the code isn't intended for copy-pasting.
- Don't use aliases in examples unless you're specifically documenting the alias.
- Avoid using positional parameters. Use the parameter name, even if the parameter is positional.

Formatting command syntax

All prose should follow the style guidelines for PowerShell content. Those rules are repeated here for convenience:

- Always use the full name for cmdlets and parameters. Avoid using aliases unless you're specifically demonstrating the alias.

- Property, parameter, object, type names, class names, class methods should be **bold**.
  - Property and parameter values should be wrapped in backticks (`` ` ``).
  - When referring to types using the bracketed style, use backticks. For example: `[System.Io.FileInfo]`
- PowerShell module names should be **bold**.
- PowerShell keywords and operators should be all lowercase.
- Use proper (Pascal) casing for cmdlet names and parameters.
- When you refer to a parameter by name, the name should be **bold**.
- Use parameter name with the hyphen if you're illustrating syntax. Wrap the parameter in backticks.
- When you show example usage of an external command, the example should be wrapped in backticks. Always include the file extension of the external command.

Link references

For maintainability and readability of the markdown source for our documentation, we're converting our conceptual documentation to use link references instead of inline links.

For example, instead of:

Markdown

```
The [Packages tab](https://www.powershellgallery.com/packages) displays all
available
packages in the PowerShell Gallery.
```

It should be:

Markdown

```
The [Packages tab][01] displays all available packages in the PowerShell Gallery.
```

> **Note**
>
> When you replace an inline link, reflow the content to wrap at 100 characters. You can use the reflow-markdown VS Code extension to quickly reflow the paragraph.

At the bottom of the file, add a markdown comment before the definition of the links.

Markdown

```
<!-- Link references -->
[01]: https://www.powershellgallery.com/packages
```

Make sure that:

1. Every link points to the correct location.
2. Don't duplicate link reference definitions. If a link reference definition already exists for a URL, reuse the existing reference instead of creating a new one.
3. The link reference definitions are at the bottom of the file after the markdown comment and are in the numeric order.

Markdown linting

For any article in one of the participating repositories, opening the article in VS Code displays linting warnings. Address any of these warnings you find, except:

- MD022/blanks-around-headings/blanks-around-headers for the `Synopsis` header in cmdlet reference documents. For those items, the rule violation is intentional to ensure the documentation builds correctly with PlatyPS.

Make sure of the line lengths and use the Reflow Markdown extension to fix any long lines.

Spelling

Despite our best efforts, typos and misspellings get through and end up in the documentation. These mistakes make documentation harder to follow and localize. Fixing these mistakes makes the documentation more readable, especially for non-English speakers who rely on accurate translations.

Last updated on 02/12/2026

Hacktoberfest and other hack-a-thon events

Article • 05/29/2025

Hacktoberfest is an annual worldwide event held during October. The event encourages open source developers to contribute to repositories through pull requests (PR). GitHub hosts many open source repositories that contribute to Microsoft Learn content. Several repositories actively participate in Hacktoberfest.

How to contribute

Before you can contribute to an open source repo, you must first configure your account to contribute to Microsoft Learn. If you're new to this process, start by signing up for a GitHub account. Be sure to install Git and the Markdown tools.

To get credit for participation, register with Hacktoberfest and read their participation guide.

Find a repo that needs your help

The PowerShell-Docs team is supporting Hacktoberfest contributions for several PowerShell documentation repositories. We defined a set of cleanup tasks designed to be simple for first time contributors. Full information can be found in the Hacktoberfest meta-issue.

To be successful with these tasks, you should:

- Have a general understanding of PowerShell syntax
- Have an understanding of splatting
- Be able to read and follow the PowerShell-Docs style guide and Editorial checklist
- Have basic familiarity with Markdown

Before contributing should read the meta-issue. When you're ready to start, open a new Hacktoberfest using one of the following links:

- MicrosoftDocs/PowerShell-Docs
- MicrosoftDocs/PowerShell-Docs-DSC
- MicrosoftDocs/PowerShell-Docs-Modules
- MicrosoftDocs/windows-powershell-docs
- MicrosoftDocs/azure-docs-powershell

Quality expectations

To have a successful contribution to an open source Microsoft Learn repository, create a meaningful and impactful PR. The following examples from the official Hacktoberfest site are considered ***low-quality contributions***:

- PRs containing bulk automated changes
  - Example: scripted PRs to remove whitespace, fix common spelling, or optimize images
  - Submit an issue first describing the automated changes you want to make
- PRs deemed disruptive (for example, taking someone else's branch or commits and making a PR)
- PRs deemed a hindrance vs. helping
- PRs that are clearly an attempt to increment your PR count for October

Open a PR

A *PR* provides a convenient way for a contributor to propose a set of changes. Successful PRs have these common characteristics:

- The PR adds value.
- The contributor is receptive to feedback.
- The intended changes are well articulated.
- The changes are related to an existing issue.

If you're proposing a PR without a corresponding issue, create an issue first. For more information, see GitHub: About pull requests.

See also

- Git and GitHub essentials for Microsoft Learn documentation
- Official Hacktoberfest site

How we manage issues

Article • 03/30/2025

This article documents how we manage issues in the PowerShell-Docs repository. This article is designed to be a job aid for members of the PowerShell-Docs team. We publish this information here to provide process transparency for our public contributors.

Sources of issues

- Community contributors
- Internal contributors
- Transcriptions of comments from social media channels
- Feedback via the Docs feedback form

Response time targets

80% of new issues are closed within 3 business days.

- Triaged - 1 business day
- Fix or change - 10 business days

Labeling & Milestones

Label Types

- **Area** - Identifies the part of PowerShell or the docs that the issue is discussing
- **Issue** - The type of issue: like bug, feedback, or idea
- **Priority** - The priority of the issue; value range 0-3 (high-low)
- **Quality** - The quality improvement effort the issue commits to resolving
- **Status** - The status of the work item or why it was closed
- **Tag** - Used to for additional classification like availability or doc-a-thons
- **Waiting** - Shows that we're waiting on some external person or event

For more information on specific labels, see Labeling.

Milestones

Issues and PRs should be tagged with the appropriate milestone. If the issue doesn't apply to a specific version, then no milestone is used. PRs and related issues for changes that have yet to

be merged into the PowerShell code base should be assigned to the **Future** milestone. After you merge the change, update the milestone to the appropriate version.

Expand table

Milestone	Description
7.0.0	Work items related to PowerShell 7.0
7.2.0	Work items related to PowerShell 7.2
7.3.0	Work items related to PowerShell 7.3
Future	Work items a future version of PowerShell

Triage process

PowerShell docs team members review the issues daily and triage new issues as they arrive. The team meets weekly to discuss difficult issues need triage and prioritize the work.

Misplaced product feedback

- Enter a comment redirecting the customer to the correct feedback channel.
- Optional: Copy the issue to the appropriate product feedback location, add a link to the copied item, and close the issue.

  The default location for PowerShell issues is https://github.com/PowerShell/PowerShell/issues/new/choose.

Support requests

- If the support question is simple, answer it politely and close the issue.
- If the question is more complicated, or the submitter replies with more questions, redirect them to forums and support channels. Suggested text for redirecting to forums:

  Markdown

  ```
  > This is not the right forum for these kinds of questions. Try posting your
  question in a
  > community support forum. For a list of community forums see:
  > https://learn.microsoft.com/powershell/scripting/community/community-
  support
  ```

Code of conduct violations

- Edit the issue to remove any offensive content, if necessary
- Enter a comment indicating the issue is spam, close the issue, and then lock it to prevent further comments
- Discuss each violation in the regular triage meeting to determine the need for further action

Managing pull requests

Article • 03/30/2025

This article documents how we manage pull requests in the PowerShell-Docs repository. This article is designed to be a job aid for members of the PowerShell-Docs team. We publish this information here to provide process transparency for our public contributors.

Best practices

- Request a review. The person submitting the PR shouldn't merge the PR without a peer review.
- Assign the peer reviewer when the PR is submitted. Early assignment allows the reviewer to respond sooner with editorial remarks.
- Use comments to describe the nature of the change being submitted. For example, if the change is minor, explain the change and that you don't need a full technical review. Be sure to @mention the reviewer.
- Use the comment suggestion feature to make it easier for the author to accept the suggested change. For more information, see Reviewing proposed changes in a pull request.

PR Process steps

1. Writer: Create PR

    - Fill out the PR template
    - Link any issues resolved by the PR
    - Use GitHub's autoclose feature to close the issue
    - Work through and check off each item in the checklist

2. Writer: Assign peer reviewer
3. Reviewer: proofreads and comments (as necessary)
4. Writer: Incorporate review feedback
5. Both: Review preview rendering
6. Both: Review validation report - fix warnings and errors
7. Reviewer: Mark review "Approved"
8. Repo Maintainer: Merge PR

Content Reviewer Checklist

See the editorial checklist for a more comprehensive list.

- Proofread for grammar, style, concision, technical accuracy
- Ensure examples still apply for the target version
- Check Preview rendering
- Check metadata - ms.date, remove ms.assetid, ensure required fields
- Validate markdown correctness
  - See style guide for content-specific formatting rules
- Reorganize examples as follows:
  - Intro paragraph
  - Code and output
  - Detailed explanation of code (as necessary)
- Check hyperlinks for accuracy
  - Replace or remove TechNet/MSDN links
  - Ensure minimum number of redirects to target
  - Ensure HTTPS
  - Correct link type
    - File links for local files
    - URL links for files outside of the docset
  - Remove locales from URLs
  - Simplify URLs pointing to `learn.microsoft.com`
- Verify versioned content is correct across all versions
  - Review the versioned content change report

Branch Merge Process

The `main` branch is the only branch that should be merged into `live`. Merges from short-lived (working) branches should be squashed before merging into `main`.

Expand table

Merge from/to	***release-branch***	***main***	***live***
working-branch	squash and merge	squash and merge	Not allowed
release-branch	—	merge	Not allowed
main	rebase	—	merge

PR Merger checklist

- Content review complete
- Correct target branch for the change

- No merge conflicts
- All validation and build step pass
  - Warnings and suggestions should be fixed (see Notes for exceptions)
  - No broken links
  - The Checklist action ran and passed
  - If an Authorization check was triggered, it passed
- Merge according to table

Notes

The following warnings can be ignored:

```
Can't find service name for `<version>/<modulepath>/About/About.md`
```

```
Metadata with following name(s) are not allowed to be set in YAML header, or as
file level
metadata in docfx.json, or as global metadata in docfx.json: `locale`. They are
generated by
Docs platform, so the values set in these 3 places will be ignored. Please remove
them from all
3 places to resolve the warning.
```

When a PR is merged, the HEAD of the target branch is changed. Any open PRs that were based on the previous HEAD are now outdated. A Maintainer has the right required to override the merge warnings and merge the outdated PR in GitHub. Merging an outdated PR is safe if the previously merged PRs didn't touch the same files.

To update the PR, select the **Update Branch** button. Choose **Update with rebase** option. For more information, see Updating your pull request branch.

Publishing to Live

Periodically, the changes accumulated in the `main` branch need to be published to the live website.

- The `main` branch is merged to `live` each weekday at 3pm PST.
- The `main` branch should be merged to `live` after any significant change.
  - Changes to 50 or more files
  - After merging a release branch

- Changes to repo or docset configurations (docfx.json, OPS configs, build scripts, etc.)
- Changes to the redirection file
- Changes to the TOC
- After merging a "project" branch (content reorg, bulk update, etc.)

Labeling in GitHub

Article • 03/30/2025

This article documents how we label issues and pull requests in the PowerShell-Docs repository. This article is designed to be a job aid for members of the PowerShell-Docs team. We publish this information here to provide process transparency for our public contributors.

Labels always have a name and a description that is prefixed with their type.

Area labels

Area labels identify the parts of PowerShell or the documentation that the issue relates to.

Expand table

Label	Related Content
`area-about`	The `about_*` articles.
`area-archive`	The Microsoft.PowerShell.Archive module.
`area-cim`	The CimCmdlets module.
`area-community`	Community-facing projects, including the contributor's guide and monthly updates.
`area-conceptual`	Conceptual articles (not cmdlet reference).
`area-console`	The console host
`area-core`	The Microsoft.PowerShell.Core module.
`area-crescendo`	The Crescendo module.
`area-debugging`	Debugging PowerShell.
`area-diagnostics`	The Microsoft.PowerShell.Diagnostics module.
`area-dsc`	PowerShell Desired State Configuration.
`area-editorsvcs`	The PowerShell editor services.
`area-engine`	The PowerShell engine.
`area-error-handling`	Error handling in PowerShell

Label	Related Content
`area-experimental`	PowerShell's experimental features
`area-gallery`	The PowerShell Gallery.
`area-helpsystem`	The Help services, including the pipeline and `*-Help` cmdlets.
`area-host`	The Microsoft.PowerShell.Host module.
`area-ise`	The PowerShell ISE.
`area-jea`	The Just Enough Administration feature.
`area-language`	The PowerShell syntax and keywords.
`area-learn`	The structured training content for PowerShell.
`area-localaccounts`	The Microsoft.PowerShell.LocalAccounts module.
`area-localization`	Localization problems or opportunities for the content.
`area-management`	The Microsoft.PowerShell.Management module.
`area-native-cmds`	Using native commands in PowerShell.
`area-omi`	Open Management Infrastructure & CDXML.
`area-ops-issue`	Building and rendering the content on the site.
`area-other`	Miscellaneous modules.
`area-overview`	The overview section in the conceptual content.
`area-packagemanagement`	The PackageManagement module.
`area-parallelism`	Content covering parallel processing, such as using `ForEach-Object` or PowerShell Jobs.
`area-platyps`	The PlatyPS module.
`area-portability`	Cross-platform compatibility.
`area-powershellget`	The PowerShellGet module.
`area-providers`	PowerShell providers.
`area-psreadline`	The PSReadLine module.
`area-release-notes`	The PowerShell release notes.

Label	Related Content
`area-remoting`	The PowerShell remoting feature and cmdlets.
`area-scriptanalyzer`	The PSScriptAnalyzer module.
`area-sdk-docs`	The conceptual documentation for the PowerShell SDK.
`area-sdk-ref`	The .NET API reference documentation for the PowerShell SDK.
`area-security`	The Microsoft.PowerShell.Security module and security concepts in general.
`area-setup`	Installing and configuring PowerShell.
`area-threadjob`	The ThreadJob module.
`area-utility`	The Microsoft.PowerShell.Utility module.
`area-versions`	Issues with the versioning of the documentation.
`area-vscode`	The VS Code PowerShell extension.
`area-wincompat`	The Windows Compatibility feature.
`area-wmf`	The Windows Management Framework.
`area-workflow`	The Windows PowerShell Workflow feature.

Issue labels

Issue labels distinguish issues by purpose.

Expand table

Label	Issue Category
`issue-doc-bug`	Errors or ambiguities in the content
`issue-doc-idea`	Requests for new content
`issue-kudos`	Praise, positive feedback, or thanks rather than work items
`issue-product-feedback`	Feedback or problems with the product itself
`issue-question`	Support questions

Priority labels

Priority labels rank which work items need to be worked on before others. These labels are only used when needed to manage large sets of work items.

Expand table

Label	Priority Level
`Pri0`	Highest
`Pri1`	High
`Pri2`	Medium
`Pri3`	Low

Project Labels

Project labels indicate what ongoing GitHub Project a work item is related to. These labels are used for automatically adding work items to a project on creation.

Expand table

Label	Project
`project-quality`	The quality improvement project

Quality labels

Quality labels categorize work items for the quality improvement effort.

Expand table

Label	Improvement
`quality-aliases`	Ensure cmdlet aliases are documented
`quality-format-code-samples`	Ensure proper casing, line length, and other formatting in code samples
`quality-format-command-syntax`	Ensure proper casing and formatting for command syntax
`quality-link-references`	Ensure links in conceptual docs are defined as numbered references

Label	Improvement
`quality-markdownlint`	Ensure content follows markdownlint rules
`quality-spelling`	Ensure proper casing and spelling for words

Status labels

Status labels indicate why a work item was closed or shouldn't be merged. Issues are only given status labels when they're closed without a related PR.

Expand table

Label	Status
`resolution-answered`	Closed by existing documentation
`resolution-duplicate`	Closed as duplicate issue
`resolution-external`	Closed by customer or outside resource
`resolution-no-repro`	Unable to reproduce the reported issue
`resolution-refer-to-support`	Closed and referred to community or product support
`resolution-wont-fix`	Closed as won't fix

Tag labels

Tag labels add independent context for work items.

Expand table

Label	Purpose
`in-progress`	Someone is actively working on the item
`go-live`	The work item is related to a specific release
`doc-a-thon`	The work item is related to a doc-a-thon
`up-for-grabs`	Any contributor can volunteer to resolve the work item
`hacktoberfest-accepted`	The PR is accepted for inclusion in #hacktoberfest
`hacktoberfest-candidate`	The PR is a candidate for inclusion in #hacktoberfest

Label	Purpose
`needs-triage`	The issue must be triaged by the team before it's ready to be worked
`code-of-conduct`	Closed for spam, trolling, or code of conduct violations
`do-not-merge`	The PR isn't meant to be merged

Waiting labels

Waiting labels indicate that a work item can't be resolved until an external condition is met.

Expand table

Label	Waiting For
`hold-for-pr`	Upstream PR to be merged
`hold-for-release`	Upstream product to release
`needs-investigation`	Waiting for team member to verify or research
`needs-more-info`	Additional details or clarification from work item author
`needs-response`	Response from work item author
`review-shiproom`	Shiproom discussion with the PowerShell team

PowerShell Support Lifecycle

There are multiple versions of PowerShell 7 that can be installed.

- **Stable release** - A stable release is a release that occurs between LTS releases. Stable releases can contain critical fixes, innovations, and new features. Microsoft supports a Stable release for about six months after the next LTS release.

  The current Stable release is PowerShell 7.5.10.

- **Long Term Servicing (LTS) release** - An LTS release of PowerShell is an LTS release of .NET. Updates to an LTS release only contain critical security updates and servicing fixes that are designed to minimize impact on existing workloads.

  The current LTS release is PowerShell 7.6.5. The previous LTS release, PowerShell 7.4.19, is still supported until 10-Nov-2026.

- **Preview release** - A preview release is a version of PowerShell that's currently in development. Preview releases can contain breaking changes, bug fixes, new features, and experiments. Preview releases might contain bugs and might not be stable. For that reason, you shouldn't use them in production environments. Preview versions aren't officially supported. They allow you to test out new features and provide feedback. Your feedback is important and can influence the features that get released.

  PowerShell 7.7-preview.3 is the current preview release.

PowerShell follows the Microsoft Modern Lifecycle Policy. The end-of-support dates follow the .NET Support Policy for the version of .NET that the release of PowerShell was built upon. Both LTS and Stable releases of PowerShell receive security updates and bug fixes. Microsoft only supports the latest update version of a release.

> **Note**
>
> This document is about support for PowerShell, not Windows PowerShell. Windows PowerShell is a component of the Windows operating system and is subject to the Windows support lifecycle. For more information, see Product and Services Lifecycle Information.

Support options

Microsoft provides support for PowerShell on a best-effort basis. Support for Windows PowerShell 5.1 is provided through Windows support channels. You can use the standard paid support channels to get support for PowerShell.

- Support for business
- Contact support

There are many free support options available from the PowerShell community. The most active community support channels are available through **Discord** or **Slack**. The discussion channels are mirrored on both platforms, so you can choose the platform that you prefer. These channels can help you troubleshoot issues, answer questions, and provide guidance on how to use PowerShell.

If you think that you found a bug, you can file an issue on GitHub. The PowerShell team can't provide support through GitHub, but they welcome bug reports. The community support page provides links to the most popular community support channels.

Supported platforms

PowerShell runs on multiple operating systems (OS) and processor architecture platforms. The platform must meet the following criteria:

- The target platform (OS version and processor architecture) is supported by .NET
- Microsoft has tested and approved PowerShell on the target platform
- The OS version is supported by the distributor for at least one year
- The OS version isn't an interim release or equivalent
- The OS version is currently supported by the distributor

Support for PowerShell ends when either of the following conditions are met:

- The target platform reaches end-of-life as defined by the platform owner
- The specific version of PowerShell reaches end-of-life

After a version of PowerShell reaches end-of-life, no further updates, including security updates, are provided. Microsoft encourages customers to upgrade to a supported version of PowerShell to continue receiving updates and support.

Windows

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of Windows reaches end-of-support.

The Docker images for the .NET SDK contain the latest versions of PowerShell. These images are available from the Microsoft Artifact Registry. These images may not have the latest security updates. Microsoft recommends that you update the OS packages to the latest version to ensure the latest security updates are applied. These images are provided for testing purposes. If you need a Docker image for a production workload, you should build and maintain your own image.

> **Note**
>
> Support for a specific version of Windows is determined by the Microsoft Support Lifecycle policies. For more information, see:
>
> - Windows client lifecycle FAQ
> - Modern Lifecycle Policy FAQ

macOS

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of macOS reaches end-of-support.

The following versions of macOS are supported:

- macOS 26 (Tahoe) x64 and Arm64
- macOS 15 (Sequoia) x64 and Arm64
- macOS 14 (Sonoma) x64 and Arm64

Apple determines the support lifecycle of macOS. For more information, see the following:

- macOS release notes
- Apple Security Updates

Alpine Linux

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of Alpine reaches end-of-life.

Support for these versions of Alpine ends on the following dates:

- Alpine 3.24 - 2028-06-01
- Alpine 3.23 - 2027-11-01
- Alpine 3.22 - 2027-05-01
- Alpine 3.21 - 2026-11-01

The Docker images for the .NET SDK contain the latest versions of PowerShell. These images are available from the Microsoft Artifact Registry.

These images are built from official operating system (OS) images provided by the OS distributor. These images may not have the latest security updates. Microsoft recommends that you update the OS packages to the latest version to ensure the latest security updates are applied.

These images are provided for testing purposes. If you need a Docker image for a production workload, you should build and maintain your own.

Debian Linux

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of Debian reaches end-of-life.

Support for these versions of Debian ends on the following dates:

- Debian 13 - 2028-08-09

Install package files (`.deb`) are also available from https://packages.microsoft.com/.

The Docker images for the .NET SDK contain the latest versions of PowerShell. These images are available from the Microsoft Artifact Registry.

These images are built from official operating system (OS) images provide by the OS distributor. These images may not have the latest security updates. Microsoft recommends that you update the OS packages to the latest version to ensure the latest security updates are applied.

These images are provided for testing purposes. If you need a Docker image for a production workload, you should build and maintain your own.

Red Hat Enterprise Linux (RHEL)

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of RHEL reaches end-of-support.

Support for these versions of RHEL ends on the following dates:

- RHEL 10 - 2035-05-31
- RHEL 9 - 2032-05-31
- RHEL 8 - 2029-05-31

Install package files (`.rpm`) are also available from https://packages.microsoft.com/.

PowerShell is tested on Red Hat Universal Base Images (UBI). For more information, see the UBI information page.

Ubuntu Linux

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of Ubuntu reaches end-of-support.

Support for these versions of Ubuntu ends on the following dates:

- Ubuntu 26.04 (Resolute Raccoon) - 2031-04-30
- Ubuntu 24.04 (Noble Numbat) - 2029-05-31
- Ubuntu 22.04 (Jammy Jellyfish) - 2024-09-30

Install package files (`.deb`) are also available from https://packages.microsoft.com/.

The Docker images for the .NET SDK contain the latest versions of PowerShell. You can download these images from the Microsoft Artifact Registry.

These images are built from official operating system (OS) images provide by the OS distributor. These images may not have the latest security updates. Microsoft recommends that you update the OS packages to the latest version to ensure the latest security updates are applied.

These images are provided for testing purposes. If you need a Docker image for a production workload, you should build and maintain your own.

> **Note**
>
> Ubuntu 25.10 (Questing Quokka) is an interim release. Microsoft doesn't test or support interim releases of Ubuntu. For more information, see Community supported distributions.

Support for PowerShell modules

The support lifecycle for PowerShell doesn't cover modules that ship outside of the PowerShell release package. For example, using the `ActiveDirectory` module that ships as part of Windows Server is supported under the Windows Support Lifecycle.

Support for experimental features

Experimental features aren't intended to be used in production environments. We appreciate feedback on experimental features and we provide best-effort support for them.

Notes on licensing

PowerShell is released under the MIT license. Under this license, and without a paid support agreement, users are limited to community support. With community support, Microsoft makes no guarantees of responsiveness or fixes.

PowerShell end-of-support dates

The PowerShell support lifecycle follows the support lifecycle of .NET. The following table lists the end-of-support dates for the current versions of PowerShell:

Expand table

Version	Release Date	End-of-support	.NET Version
PowerShell 7.7 (preview)			.NET 11.0
PowerShell 7.6 (LTS)	18-Mar-2026	14-Nov-2028	.NET 10.0
PowerShell 7.5	23-Jan-2025	10-Nov-2026	.NET 9.0
PowerShell 7.4 (LTS)	16-Nov-2023	10-Nov-2026	.NET 8.0

The following table lists the end-of-support dates for retired versions of PowerShell:

Expand table

Version	Release Date	End-of-support	.NET Version
PowerShell 7.3	09-Nov-2022	08-May-2024	.NET 7.0
PowerShell 7.2 (LTS)	08-Nov-2021	08-Nov-2024	.NET 6.0
PowerShell 7.1	11-Nov-2020	08-May-2022	.NET 5.0
PowerShell 7.0 (LTS)	04-Mar-2020	03-Dec-2022	.NET Core 3.1
PowerShell 6.2	29-Mar-2019	04-Sep-2020	.NET Core 2.1
PowerShell 6.1	13-Sep-2018	28-Sep-2019	.NET Core 2.1
PowerShell 6.0	20-Jan-2018	13-Feb-2019	.NET Core 2.0

Windows PowerShell release history

The following table contains a historical timeline of the major releases of Windows PowerShell. Microsoft no longer supports Windows PowerShell versions lower than 5.1.

Expand table

Version	Release Date	Note
Windows PowerShell 5.1	Aug-2016	Released in Windows 10 Anniversary Update and Windows Server 2016, WMF 5.1
Windows PowerShell 5.0	Feb-2016	Released in Windows Management Framework (WMF) 5.0
Windows PowerShell 4.0	Oct-2013	Released in Windows 8.1 and with Windows Server 2012 R2, WMF 4.0
Windows PowerShell 3.0	Oct-2012	Released in Windows 8 and with Windows Server 2012 WMF 3.0
Windows PowerShell 2.0	Jul-2009	Released in Windows 7 and Windows Server 2008 R2, WMF 2.0
Windows PowerShell 1.0	Nov-2006	Released as optional component of Windows Server 2008

Run the following command to see the full version number of .NET used by the version of PowerShell you're running:

PowerShell

```
[System.Runtime.InteropServices.RuntimeInformation]::FrameworkDescription
```

Last updated on 08/13/2026